

MÓDULO 02

Diseño de Features

Aprende a descomponer cualquier feature en papel y lápiz antes de escribir código. El arte perdido de pensar primero, codificar después.

METODOLOGÍA

**FADER + Mapeo +
Contratos + Flujo**

BACKEND

**Supabase (BaaS) · REST
API**

NIVEL

**Principiante a
Avanzado**

TIEMPO ESTIMADO

**25-35 horas (con
prácticas)**

Clean Architecture

Flutter

Supabase

Contract-First

Contenido

18 capítulos · 18 archivos · ~25 min por feature

PARTE 1 · FUNDAMENTOS

00	Alcance de la Feature (Paso 0)
01	Descomposición de Features: Framework FADER
01a	Práctica: Descomponer Feature Carrito de Compras

PARTE 2 · MAPEO A CAPAS Y CONTRATOS

02	Mapeo a Capas de Clean Architecture
02a	Práctica: Mapear Carrito de Compras a Clean Architecture
03	Contract-First Design: Primero los Contratos
03a	Práctica: Contratos del Carrito de Compras

PARTE 3 · FLUJO DE DATOS

04	Flujo de Datos entre Capas
04a	Práctica: Flujo de Datos del Carrito de Compras

PARTE 4 · CONTRATO CON EL BACKEND Y CALIDAD

05e	Diseño Supabase por Feature
05f	Criterios de Aceptación y Matriz de Trazabilidad

PARTE 5 · CASOS INTEGRADORES

05	Caso Completo: Sistema de Reservas
05b	Caso Práctico: Plataforma de Cursos Online (E-Learning)
05c	Caso Práctico: Sistema de Facturación
05d	Caso Práctico: App de Delivery

15 15: Estimación de Complejidad de Features

PL Plantilla de Diseño de Feature

BI Bibliografía y Fuentes

Alcance de la Feature (Paso 0)

Antes de aplicar FADER, define qué entra y qué NO entra en la feature. Una feature puede estar bien descompuesta y aun así tener un alcance ambiguo que la haga crecer mientras la implementas.

¿Por qué definir alcance primero?

El error clásico no es descomponer mal: es descomponer **bien la feature equivocada**.

Sin alcance, cada revisión añade un "y de paso también...":

- "La pantalla de compradores" → alguien pregunta "*¿y qué pasa con las notificaciones?*" → la feature crece.
- El FADER se rehace 3 veces porque el problema real nunca quedó acotado.
- El sprint termina con una feature a medias y otra empezada.

El alcance responde antes que cualquier otra cosa:

¿Qué resuelve esta feature? ¿Qué NO resuelve? ¿Qué necesita de los demás?

FADER te dice *cómo* hacer la feature. El alcance te dice *cuál* es la feature.

Flujo del diseño:

- | | |
|---------------|--|
| 0. ALCANCE | ← este archivo |
| 1. FADER | ← 01-descomposicion-feature.md |
| 2. MAPEO | ← 02-mapeo-capas.md |
| 3. CONTRATOS | ← 03-contratos-primero.md |
| 4. FLUJO | ← 04-flujo-datos.md |
| 5. BACKEND | ← 05e-diseno-supabase.md (Supabase / REST API) |
| 6. CRITERIOS | ← 05f-criterios-acceptacion-trazabilidad.md |
| 7. ESTIMACIÓN | ← 15-estimacion-complejidad.md |

Las 5 secciones del Alcance

1. Incluye

Todo lo que la feature **sí** cubre. Sé explícito y concreto. No escribas "gestión de compradores" — escribe las operaciones.

Incluye:

- Listar compradores.
- Buscar por nombre o teléfono.
- Aprobar tickets.
- Liberar tickets.

2. No incluye

Todo lo que la feature **no** cubre, aunque parezca relacionado. Es la sección que evita el *scope creep*.

No incluye:

- Enviar notificaciones.
- Procesar pagos.
- Editar datos del comprador.
- Historial de cambios.

Regla: si algo que *podría* estar relacionado no está en "Incluye", debe estar en "No incluye". No dejes espacio para la ambigüedad.

3. Dependencias

Qué necesita que exista **antes** o **durante**. Pueden ser otras features, servicios, tablas o APIs.

Dependencias:

- Feature de autenticación (para saber quién es el organizador).
- Tabla `raffles` existente en Supabase.
- RLS configurado para `organizer\_id`.

4. Suposiciones

Cosas que asumes como verdaderas sin verificar. Documentarlas permite que se discutan en vez de descubrirse en producción.

Suposiciones:

- Un comprador pertenece a una sola rifa.
- "Aprobar" significa marcar el ticket como ganador.
- El organizador es el único que gestiona su rifa.

5. Preguntas abiertas

Lo que todavía no sabes y necesitas responder antes o durante el diseño. Si una pregunta puede cambiar el alcance, resuélvela antes de FADER.

Preguntas abiertas:

- ¿Puede aprobarse un ticket que ya fue aprobado?
- ¿La aprobación debe ser atómica con la liberación?
- ¿Hay límite de tickets aprobados por rifa?

Ejemplo completo: Buyers

Aplicado a una feature real de una app de rifas:

Feature: Gestión de compradores (Buyers)

Incluye:

- Listar compradores.
- Buscar por nombre o teléfono.
- Aprobar tickets seleccionados.
- Liberar tickets no seleccionados.

No incluye:

- Enviar notificaciones.
- Procesar pagos.
- Editar datos del comprador.
- Historial de aprobaciones.

Dependencias:

- Autenticación (identidad del organizador).
- Feature de rifas (tabla raffles).
- RLS en tickets por organizer_id.

Suposiciones:

- Un comprador pertenece a una sola rifa.
- "Aprobar" = marcar ticket como ganador.
- Solo el organizador de la rifa gestiona sus compradores.

Preguntas abiertas:

- ¿Puede aprobarse un ticket ya aprobado?
 - ¿La aprobación debe ser atómica (aprobar + liberar)?
 - ¿Existe límite de tickets aprobados por rifa?
-

Plantilla de Alcance

Feature: [Nombre]

Alcance

Incluye

- []
- []

No incluye

- []
- []

Dependencias

- []
- []

Suposiciones

- []
- []

Preguntas abiertas

- []
- []

Errores comunes

Error	Síntoma	Solución
"Incluye" vago	"Gestión de compradores"	Lista operaciones concretas
"No incluye" vacío	La feature crece durante la implementación	Anticipa lo que podría parecer relacionado
Ocultar supuestos	Se descubren en producción	Documenta todo lo que asumes
Preguntas sin resolver	Rediseños constantes	Resuélvelas antes de FADER si cambian el alcance
Alcance y FADER desalineados	Operaciones que no están en "Incluye"	Cada operación de FADER debe estar en "Incluye"

Checklist de autoevaluación

- ☐ Cada operación de FADER aparece en "Incluye"
- ☐ Nada de "No incluye" se filtra al diseño
- ☐ Cada dependencia está identificada (features, tablas, servicios)

- ☐ Las suposiciones son explícitas y verificables
 - ☐ Las preguntas abiertas críticas se resolvieron antes de empezar
-

Siguiente paso

Con el alcance definido, aplica **FADER** para descomponer la feature dentro de los límites que acabas de fijar.

Tiempo estimado: 10-15 minutos por feature

Herramientas: Papel y lápiz

Descomposición de Features: Framework FADER

Aprende a diseccionar cualquier feature en piezas atómicas antes de escribir una línea de código.

Paso 0: Define el Alcance

Antes de descomponer, fija los límites de la feature. Una feature puede estar perfectamente descompuesta y aun así crecer sin control si el alcance es ambiguo.

Feature: Gestión de compradores (Buyers)

Incluye:

- Listar compradores.
- Buscar por nombre o teléfono.
- Aprobar tickets seleccionados.
- Liberar tickets no seleccionados.

No incluye:

- Enviar notificaciones.
- Procesar pagos.
- Editar datos del comprador.

Dependencias:

- Autenticación (identidad del organizador).
- Feature de rifas (tabla raffles).

Suposiciones:

- Un comprador pertenece a una sola rifa.

Preguntas abiertas:

- ¿Puede aprobarse un ticket ya aprobado?
- ¿La aprobación debe ser atómica?

Regla: cada operación que descubras en FADER debe caber en "Incluye". Si descubres algo que no cabe, o amplía el alcance (y se negocia), o es otra feature.

Consulta la teoría completa y la plantilla en [00-alcance-feature.md](#).

¿Por qué descomponer?

Cuando te dan una feature como "agregar carrito de compras", el 80% del trabajo no es escribir el código del carrito. El trabajo real es entender:

- ¿Qué es un carrito en este negocio? (no todos los carritos funcionan igual)
- ¿Qué puede hacer el usuario con él?
- ¿Qué reglas de negocio aplican?
- ¿Qué bordes y excepciones existen?

La descomposición sistemática responde todo eso **antes** de abrir el editor.

Framework FADER

F A D E R

Formular → Actorizar → Descomponer → Entidades → Reglas

De lo general → A lo específico

De lo externo → A lo interno

Paso 1: Formular

Define el problema con la mayor precisión posible. Una feature bien formulada es media solución.

Formato:

Como [actor], quiero [acción] para [beneficio/valor].

Contraejemplos:

- ❌ "Hay que hacer un carrito de compras"
- ❌ "El usuario necesita agregar productos"

Bien formulados:

- ✅ "Como **cliente registrado**, quiero **agregar productos a un carrito** para **revisarlos antes de comprar**."
- ✅ "Como **cliente**, quiero **ver el total actualizado del carrito** para **saber cuánto gastaré**."

Plantilla de preguntas:

Pregunta	Lo que descubres
¿Qué necesidad real resuelve?	El propósito de la feature
¿Qué pasa si NO existe?	La urgencia/impacto real
¿Quién se beneficia?	Los actores involucrados
¿Cómo se usa hoy sin esta feature?	El workaround actual
¿Qué expectativas tiene el usuario?	Criterios de éxito

Paso 2: Actorizar

Identifica **quién** interactúa con la feature. Cada actor tiene intereses y permisos distintos.

Tipos de actores:

Actor	Definición	Ejemplo
Usuario primario	Quien inicia la acción	Cliente que compra
Usuario secundario	Quien provee soporte	Administrador que gestiona inventario
Sistema externo	API, servicio, BD	Pasarela de pagos, Supabase
Sistema interno	Otra feature de la app	Módulo de notificaciones, módulo de usuarios

Ejercicio: hoja de actores

Feature: Carrito de Compras

ACTORES

- 1. Cliente (usuario primario)
 - Puede ver su carrito
 - Puede agregar/quitar productos
 - Puede modificar cantidades
 - Puede iniciar el checkout
- 2. Administrador (usuario secundario)
 - Puede ver carritos abandonados
 - Puede ajustar precios en carrito
- 3. Pasarela de Pagos (sistema externo)
 - Recibe el total del carrito
 - Devuelve confirmación/rechazo
- 4. Módulo de Inventario (sistema interno)
 - Verifica stock disponible
 - Reserva productos temporalmente

Reglas de oro:

- Cada actor tiene una **razón de ser** (no inventes actores)
- Un actor puede ser una persona o un sistema
- Un mismo usuario puede ser múltiples actores (ej: cliente y admin)

Paso 3: Descomponer

Divide la feature en **operaciones atómicas**. Cada operación debe ser:

- **Independiente** (se puede probar sola)

- **Concreta** (hace una sola cosa)
- **Valiosa** (aporta algo al usuario o al sistema)

Técnica: la tormenta de post-its

Imagina que tienes post-its y escribes cada acción mínima en uno. Luego los agrupas.

Carrito de Compras → Operaciones Atómicas:

Agregar producto al carrito

Quitar producto del carrito

Actualizar cantidad de un producto

Ver resumen del carrito

Aplicar cupón de descuento

Calcular total con impuestos

Validar stock antes del checkout

Limpiar carrito (after checkout)

Clasificación de operaciones:

Tipo	Definición	Ejemplo
Create	Crear un nuevo recurso	Agregar producto al carrito
Read	Leer/consultar datos	Ver resumen del carrito
Update	Actualizar un recurso	Cambiar cantidad
Delete	Eliminar un recurso	Quitar producto
Validación	Verificar reglas de negocio	Validar stock, validar cupón
Cálculo	Procesar datos para obtener resultado	Calcular total con impuestos
Transición	Cambiar estado	Iniciar checkout

Paso 4: Entidades

Identifica los **conceptos del mundo real** que aparecen en tu feature. No pienses en tablas de BD ni en clases de Dart. Piensa en objetos de negocio.

Ejercicio: tarjeta de entidad

Para cada entidad, llena esta tarjeta:

ENTIDAD: Producto

Descripción:

Un bien o servicio que se puede comprar

Atributos esenciales:

- id: string
- nombre: string
- precio: double
- stock: int

NO es atributo:

- cantidadEnCarrito (eso es del Carrito, no del Producto)

Se relaciona con:

- Carrito (a través de ItemCarrito)
- Categoría (clasificación)

Entidades típicas para un Carrito de Compras:

ENTIDADES DEL DOMINIO

Producto

- ├ id: String
- ├ nombre: String
- ├ precio: double
- ├ stock: int
- └ categoria: Categoria

ItemCarrito

- ├ producto: Producto
- ├ cantidad: int
- └ precioUnitario: double (precio al momento de agregar)

Carrito

- ├ id: String
- ├ items: List<ItemCarrito>
- ├ totalBruto: double
- ├ descuento: double?
- ├ impuesto: double
- └ totalNeto: double

CuponDescuento

- ├ codigo: String
- ├ tipo: Porcentaje | MontoFijo
- ├ valor: double
- └ expiracion: DateTime

Pregunta clave: ¿Esta entidad existe en el mundo real del negocio o solo existe en mi base de datos?

- ☒ **Producto** → Existe en el mundo real
- ☐ **ProductoTable** → Existe solo en la BD (eso va en la capa DATA)

Paso 5: Reglas

Captura todas las **reglas**. Una regla es una condición que debe cumplirse siempre, independientemente de la interfaz de usuario o la tecnología.

Clasifica las reglas en 3 tipos. No todas pertenecen al dominio:

Tipo	Código	Qué es	Ejemplo	Dónde vive
Negocio	RN	Regla del negocio, independiente de la tecnología	Un ticket aprobado no vuelve a aprobarse	UseCase (dominio)
Técnica	RT	Requisito de implementación con el stack	La búsqueda debe estar paginada	Contrato con el backend / DataSource
Seguridad	RS	Restricción de acceso y permisos	Impedir acceder a rifas ajenas	Autorización del servidor (RLS en Supabase / middleware en REST)

Categorías de reglas de negocio (RN):

Categoría	Ejemplo
Restricción	No se pueden agregar más de 50 productos distintos al carrito
Cálculo	El impuesto es del 16% sobre el total bruto menos descuentos
Validación	El cupón de descuento no puede estar expirado
Flujo	Si el stock es menor a la cantidad solicitada, mostrar error
Consistencia	Si se aplica un cupón, el descuento no puede exceder el 50% del total

Reglas técnicas (RT) y de seguridad (RS):

RT001 - La consulta de productos debe estar paginada

RT002 - La búsqueda debe tener debounce

RT003 - La operación de aprobar+liberar debe ser atómica (RPC)

RS001 - Solo el organizador puede leer su rifa (RLS)

RS002 - Solo el organizador puede modificar sus tickets

Estas reglas NO van al dominio ni a la UI. Se diseñan en [05e-diseno-supabase.md](#) y se implementan en DATA (DataSource + migración SQL).

Formato para documentar reglas:

RN001 - Restricción de cantidad máxima

Descripción: Un carrito no puede tener más de 50 items distintos

Actor: Cliente

Severidad: Error

Mensaje: "Has alcanzado el límite de 50 productos por carrito"

RN002 - Validación de stock

Descripción: No se puede agregar un producto si el stock es 0

Actor: Cliente

Severidad: Error

Mensaje: "El producto {nombre} no tiene stock disponible"

RN003 - Cálculo de impuesto

Descripción: El impuesto se calcula como 16% del subtotal

Actor: Sistema

Severidad: Información

RN004 - Cupón no expirado

Descripción: No se puede aplicar un cupón cuya fecha de expiración haya pasado

Actor: Cliente

Severidad: Error

Mensaje: "El cupón {codigo} ha expirado"

RN005 - Descuento máximo

Descripción: El descuento total no puede exceder el 50% del subtotal

Actor: Sistema

Severidad: Error

Mensaje: "El descuento supera el límite permitido"

La hoja FADER completa

Así se ve una feature completamente descompuesta:

FEATURE: Carrito de Compras

[O] ALCANCE:

Incluye: gestionar items, cupones y resumen

No incluye: pagos, envíos, fidelización

Dependencias: catálogo de productos, inventario

Suposiciones: cliente con sesión iniciada

Preguntas abiertas: ¿cupones combinables?

[F]ormular:

Como cliente, quiero gestionar productos en un carrito para revisarlos antes de comprar.

[A]ctorizar:

1. Cliente (primario)
2. Admin (secundario)
3. Pasarela de Pagos (externo)
4. Módulo de Inventario (interno)

[D]escomponer:

- Agregar producto (C)
- Quitar producto (D)
- Actualizar cantidad (U)
- Ver resumen (R)
- Aplicar cupón (Validación)
- Calcular total (Cálculo)
- Validar stock (Validación)

[E]ntidades:

- Producto, ItemCarrito, Carrito, CuponDescuento

[R]eglas:

- RN001: Máximo 50 items por carrito
- RN002: Stock > 0 para agregar
- RN003: Impuesto 16% del subtotal
- RN004: Cupón no expirado
- RN005: Descuento máximo 50%
- RT001: Búsqueda paginada
- RS001: RLS impide carritos ajenos

Técnicas Complementarias

FADER no vive solo. Existen otras técnicas de descomposición que puedes usar dentro de cada paso. No son competencia, son herramientas que enriquecen pasos específicos.

Event Storming — Alberto Brandolini

Qué es: Una dinámica de grupo donde se escriben eventos del negocio en post-its naranjas y se colocan en una línea de tiempo.

Dónde encaja en FADER:

Paso FADER	Cómo ayuda Event Storming
Descomponer	Los eventos ("Producto agregado al carrito", "Cupón aplicado") son operaciones atómicas
Reglas	Los eventos revelan restricciones ("No se puede agregar si no hay stock")
Entidades	Los objetos que aparecen en los eventos son candidatos a entidad

Cómo usarlo individualmente:

1. Escribe todos los eventos que pueden ocurrir:
"Producto agregado al carrito"
"Cantidad actualizada"
"Cupón aplicado"
"Stock validado"
"Carrito limpiado después del checkout"

2. Ordénalos en una línea de tiempo:
Antes del checkout → [Agregar → Actualizar → Validar stock → Aplicar cupón]
Checkout → [Calcular total → Procesar pago]
Después → [Limpiar carrito → Confirmar pedido]

3. Cada post-it es una operación atómica → lo pasas a tu [D]escomposición

Ventaja: Revela operaciones que no habías considerado (ej: "Carrito abandonado después de 24h" es un evento que no está en los requisitos originales).

User Story Mapping — Jeff Patton

Qué es: Técnica para organizar historias de usuario en un mapa de dos dimensiones: actividades principales (eje horizontal) y pasos específicos (eje vertical).

Dónde encaja en FADER:

Paso FADER	Cómo ayuda User Story Mapping
Formular	Las actividades principales del mapa son tus enunciados "Como... quiero..."
Actorizar	Cada columna del mapa suele corresponder a un actor
Descomponer	Los pasos verticales son tus operaciones atómicas

Ejemplo para Carrito:

ANTES

Explorar productos

Ver catalogo
Buscar prod
Filtrar

DURANTE

Gestionar carrito

Agregar prod
Quitar prod
Actualizar
Aplicar cupon

DESPUES

Finalizar compra

Ir a checkout
Confirmar Pago
Confirmacion

Example Mapping — Cucumber/BDD

Qué es: Técnica para descomponer una historia en ejemplos concretos usando tarjetas de colores.

Dónde encaja en FADER:

Paso FADER	Como ayuda Example Mapping
Reglas	Cada regla se ilustra con un ejemplo concreto de entrada -> salida

Formato:

Historia: Aplicar cupon de descuento

REGLA: Cupon debe estar vigente

-> Ejemplo valido: Cupon "DESC10", fecha expiracion > hoy
Resultado: Descuento aplicado

-> Ejemplo invalido: Cupon "DESC10", fecha expiracion < hoy
Resultado: Error "El cupon ha expirado"

Ventaja: Los ejemplos concretos son la mejor manera de validar que entendiste la regla. Si no puedes escribir 3 ejemplos de una regla, no la entiendes bien.

Criterio de Granularidad

Uno de los problemas mas comunes al aplicar FADER es **no saber cuando parar**. Usa el test de la atomicidad.

El test de la atomicidad

Una operacion atomica debe pasar **las 3 pruebas**:

#	Prueba	Pregunta	Si falla...
1	Un solo verbo	La operacion hace una sola cosa?	Dividela
2	Un solo resultado	Produce un unico cambio en el sistema?	Dividela
3	Valor independiente	Tiene sentido por si sola para el usuario?	Combinala o eliminala

Ejemplos:

"Gestionar carrito" -> Falla prueba 1 (tiene multiples verbos implicitos)

Correcto: "Agregar producto", "Quitar producto", "Actualizar cantidad"

"Agregar producto y aplicar cupon" -> Falla prueba 1 y 2

Correcto: "Agregar producto" y "Aplicar cupon" son operaciones separadas

"Validar formato del codigo del cupon" -> Falla prueba 3

Sin valor para el usuario por si sola, es parte de "Aplicar cupon"

Guia por tipo de feature

Tipo de feature	Tamano tipico de [D]escomposicion	Ejemplo
Hotfix / bug	1-2 operaciones	Corregir calculo de impuesto
Feature pequena	3-6 operaciones	Agregar filtro de busqueda
Feature mediana	7-15 operaciones	Carrito de compras
Feature grande	16-30 operaciones	Sistema de facturacion recurrente
Epic	30+ (dividir en sub-features)	Modulo completo de pagos

Regla practica: Si tu [D]escomposicion supera las 20 operaciones, probablemente no es una feature sino un conjunto de features. Dividelo.

Senales de granularidad incorrecta

Demasiado grueso:

- "Gestionar usuarios" -> no sabes que incluye
- "Procesar pedido" -> puede ser 10 operaciones distintas

Demasiado fino:

- "Validar que el campo email no este vacio" -> es parte de un UseCase
- "Convertir string a mayusculas" -> detalle tecnico, no operacion de negocio

El punto justo:

- "Registrar usuario" -> operacion completa con validaciones + persistencia
- "Crear pedido" -> operacion que puede fallar pero se entiende como unidad

Antipatrones de FADER

A veces es mas facil reconocer lo que esta mal. Aqui tienes FADER bien hecho vs mal hecho para la misma feature.

Feature: Filtro de busqueda de productos

FADER MAL HECHO:

[F] Como usuario, quiero buscar productos (demasiado vago)

[A] Cliente, Admin (el admin aparecio porque si, no tiene accion real)

[D] - Buscar productos (una sola operacion que lo abarca todo)
- Ver resultados (no es una operacion, es el resultado de buscar)

[E] Producto (solo esta entidad, pero faltan Categoria, Marca, etc.)

[R] - El producto debe existir (regla tecnica RT, no de negocio)
- Mostrar resultados (eso no es una regla, es un comportamiento)

Problemas:

- No distingue tipos de busqueda (texto libre, por categoria, por precio)
- Actor inventado sin proposito
- Operaciones demasiado gruesas
- Reglas que no son reglas de negocio (o no están clasificadas: las técnicas y de seguridad van como RT/RS)
- Entidades incompletas

FADER BIEN HECHO:

[F] F1: Como cliente, quiero buscar productos por nombre o categoria para encontrar lo que necesito rapidamente.
F2: Como cliente, quiero filtrar productos por rango de precio para ajustarme a mi presupuesto.

[A] 1. Cliente (primario) -> busca, filtra, navega resultados
2. Sistema de inventario (interno) -> provee datos de stock

[D] Cliente:
- [R] Buscar por texto libre (nombre del producto)
- [R] Filtrar por categoria
- [R] Filtrar por rango de precio
- [R] Ordenar resultados (precio, nombre, popularidad)
- [R] Ver paginacion de resultados

[E] Producto (id, nombre, precio, categoriaId, marcaId, stock)
Categoria (id, nombre)
Marca (id, nombre)
FiltroBusqueda (V0: texto, categoriaId?, precioMin?, precioMax?, ordenarPor?, pagina)

[R] RN001: Busqueda por texto libre coincide con nombre o descripcion
RN002: Los filtros se aplican en AND (categoria Y precio Y texto)
RN003: Sin resultados -> mensaje "No se encontraron productos"
RN004: Producto sin stock aparece al final con marca "Agotado"
RT001: Resultados paginados de a 20 items
RS001: Solo clientes autenticados pueden buscar (RLS/catálogo público)

Aciertos:

- Dos formulaciones para dos necesidades distintas
- Actor con proposito claro

- Operaciones con clasificacion CRUD
- Value Object explicito para el filtro
- Reglas especificas y bordes contemplados

Second Pass: Iterar el FADER

El primer FADER nunca es perfecto. La descomposicion es un proceso iterativo.

Cuando visitar cada paso

Paso	Disparador de revision
Formular	Cuando encuentras un actor que no formulaste. Ej: "Ah, el admin tambien necesita aprobar cupones"
Actorizar	Cuando una operacion no tiene actor claro. Ej: "Quien dispara el calculo de impuestos?"
Descomponer	Cuando una regla revela operaciones que faltan. Ej: "RN004 dice que el cupon expira... -> falta 'Validar vigencia de cupon'"
Entidades	Cuando una regla menciona conceptos sin entidad. Ej: "RN005 habla de 'limite de descuento' -> falta entidad LimiteDescuento"
Reglas	Cuando modelas entidades y ves bordes. Ej: "Que pasa si el precio es 0? y si es negativo?"

Ciclo de refinamiento recomendado

```
1er pase: F A D E R (rapido, 15 min)
|
Revisas [D]escomponer -> falta algo?
|
Revisas [R]eglas -> revelan nuevas operaciones o entidades?
|
2do pase: ajustas donde hizo falta (10 min)
|
Revision cruzada: cada regla tiene operacion?
                  cada operacion tiene actor?
                  cada entidad se usa en alguna regla?
```

Señales de que necesitas un second pass:

- Una regla menciona un concepto que no esta en tus entidades
- Una operacion no tiene a quien asignarle la responsabilidad
- Descubres que dos actores hacen lo mismo pero con reglas distintas
- Al escribir el UseCase, te das cuenta de que la operacion deberia ser dos

FADER para Features Grandes vs Pequeñas

No todas las features merecen el mismo nivel de descomposicion. Aplicar FADER completo a un hotfix es sobreingenieria. Aplicar solo dos pasos a un epic es negligencia.

Matriz de esfuerzo

	Hotfix (1-2 op)	Pequena (3-6 op)	Mediana (7-15 op)	Grande (16-30 op)	Epic (30+ op)
[F]ormular	Rapido	Completo	Completo	Completo	Multiples
[A]ctorizar	Opcional	Rapido	Completo	Completo	Completo
[D]escomponer	Rapido	Completo	Completo	Completo	Por feature
[E]ntidades	Opcional	Rapido	Completo	Completo	Completo
[R]eglas	Las clave	Las clave	Completo	Completo	Completo

Donde esta el limite?

- **Hotfix:** Solo formula y las reglas clave. 5 minutos. No hagas una tesis.
- **Feature pequena:** FADER completo pero rapido. 15-20 minutos.
- **Feature mediana:** FADER completo con second pass. 30-45 minutos.
- **Feature grande:** FADER completo mas ADR. 1-2 horas.
- **Epic:** Divide en features medianas y aplica FADER a cada una.

Ejemplo: Hotfix vs Feature completa

Hotfix: "Corregir que el calculo de IVA use 16% en vez de 21%"

```
[F] Como sistema, quiero usar la tasa de IVA correcta (16%)
    para cumplir con la normativa fiscal.

[R] RN001: El IVA para productos nacionales es 16%
    (era 21% por error)

-- Esto es suficiente. No necesitas actores, entidades, ni descomposicion.
-- Arreglas la constante y listo.
```

Feature completa: "Modulo de configuracion de impuestos por pais"

```
[F] Como administrador, quiero configurar tasas de impuesto
    por pais y tipo de producto para cumplir con regulaciones locales.

[A] Admin, Sistema de Facturacion, API de impuestos externa

[D] - [C] Crear configuracion de impuesto
    - [R] Listar configuraciones
    - [U] Editar tasa de impuesto
    - [D] Eliminar configuracion (si no tiene facturas asociadas)
    - [Validacion] Validar que la tasa este dentro del rango legal

[E] ConfiguracionImpuesto (pais, tipoProducto, porcentaje,
    fechaInicio, fechaFin, codigoFiscal)
    HistorialCambio (configuracionId, fecha, usuario, valorAnterior, valorNuevo)

[R] RN001: La tasa de IVA no puede ser negativa
    RN002: No se puede eliminar una configuracion con facturas emitidas
    RN003: Los cambios quedan registrados en historial
    RT001: El calculo de impuestos se hace en un RPC (precision decimal)
```

Conexion con Ceremonias Agiles

FADER no es solo para vos en tu escritorio. Funciona en dinamicas de equipo.

En Refinement

```
Antes: "Vamos a ver la historia #42: Carrito de compras"
    -> discusion vaga de 30 minutos sin conclusion
```

```
Con FADER: "Vamos a aplicar FADER a la historia #42"
    Paso 1: Todos escriben [F]ormular en post-its (5 min)
    Paso 2: Comparten y consolidan (5 min)
    Paso 3: [D]escomponer en pizarron (10 min)
    Paso 4: Identificar [R]eglas dudosas (5 min)
```

```
Resultado: 25 minutos, todos alineados, lista de operaciones clara
```

En Planning

Usa la [D]escomposicion como fuente de truth para estimar:

Historia: Carrito de compras

Operaciones (de FADER):

- Agregar producto -> 2 puntos
- Quitar producto -> 1 punto
- Actualizar cantidad -> 1 punto
- Ver resumen -> 2 puntos
- Aplicar cupon -> 3 puntos
- Calcular total -> 2 puntos
- Validar stock -> 2 puntos
- Limpiar carrito -> 1 punto

Total: 14 puntos (feature mediana)

Cada operacion se estima individualmente, no la feature entera. Esto elimina las estimaciones "adivinadas".

En un Spike tecnico

Cuando no sabes si algo es posible, FADER te ayuda a definir el alcance del spike:

[F] Queremos saber si podemos integrar la pasarela de pagos X

[D] Dentro del spike:

- Investigar documentacion de la API
- Crear prototipo de un pago exitoso
- Probar manejo de errores (rechazo, timeout, expirado)
- Medir latencia promedio

[R] Reglas del spike:

- Maximo 2 dias de investigacion
- Al terminar: decision de seguir o no + riesgos identificados

Validacion del FADER

Checklist de autoevaluacion

Al terminar tu hoja FADER, responde estas preguntas:

Sobre el [0] ALCANCE:

- ☐ Definiste Incluye, No incluye, Dependencias, Suposiciones y Preguntas abiertas
- ☐ Cada operacion de FADER cabe en "Incluye"
- ☐ Las preguntas abiertas criticas se resolvieron antes de descomponer

Sobre [F]ormular:

- ☐ Cada enunciado sigue "Como [actor], quiero [accion] para [valor]"
- ☐ No hay dos enunciados que digan lo mismo
- ☐ Sabes cual es la diferencia entre lo que SI hace la feature y lo que NO hace

Sobre [A]ctorizar:

- ☐ Cada actor tiene al menos una operacion que le pertenece
- ☐ Ningun actor esta ahi "porque sí"
- ☐ Los sistemas externos estan identificados
- ☐ Sabes donde terminan los permisos de cada actor

Sobre [D]escomponer:

- ☐ Cada operacion pasa el test de atomicidad (1 verbo, 1 resultado, valor independiente)
- ☐ No hay operaciones con "y" en el nombre
- ☐ Las operaciones estan clasificadas (CRUD, Validacion, Calculo, Transicion)
- ☐ Las dependencias entre operaciones estan claras

Sobre [E]ntidades:

- ☐ Cada entidad existe en el mundo real del negocio
- ☐ Los atributos son esenciales, no tecnicos
- ☐ Las relaciones entre entidades estan definidas
- ☐ No hay atributos que pertenezcan a otra entidad

Sobre [R]eglas:

- ☐ Cada regla tiene un codigo unico (RN001, RT001, RS001...)
- ☐ Clasificaste cada regla: RN (negocio), RT (tecnica), RS (seguridad)
- ☐ No hay reglas tecnicas o de seguridad mezcladas con las de negocio
- ☐ No hay reglas "obvias" sin escribir
- ☐ Cada regla RN tiene una categoria (Restriccion, Calculo, Validacion, Flujo, Consistencia)
- ☐ Las reglas tienen mensaje de error donde aplica

Revision cruzada:

- ☐ Cada regla se relaciona con al menos una operacion
- ☐ Cada operacion tiene un actor responsable
- ☐ Cada entidad aparece en al menos una regla
- ☐ No hay contradicciones entre reglas

Ejercicio: Autoevaluacion de un FADER real

Toma la hoja FADER del Carrito de Compras y aplica este checklist. Marca lo que cumple y lo que no. Si algo no cumple, revisa si falta ajustar.

Feature: Carrito de Compras

[F] "Como cliente, quiero gestionar productos en un carrito para revisarlos antes de comprar."

-> Cumple: "Como [cliente], quiero [gestionar productos en un carrito] para [revisarlos antes de comprar]."

-> NO cumple: no diferencia entre agregar, quitar, y actualizar (el verbo "gestionar" es muy amplio para [F]ormular)

Mejora: dividir en 3 enunciados mas especificos

Nota: El checklist es para que lo uses, no para que lo cumplas al 100% siempre. Un hotfix no necesita todas las preguntas. Una feature critica sí.

Errores comunes

Error	Por qué duele	Cómo evitarlo
Empezar sin alcance	La feature crece durante la implementación	Define Incluye/No incluye antes (Paso 0)
Pensar en código demasiado pronto	Te casas con una implementación antes de entender el problema	Termina FADER completo antes de pensar en clases
Mezclar actores	Lógica de admin + cliente en el mismo lugar	Separa por actor desde el principio
Reglas implícitas	"Obviamente" el stock se valida... hasta que en producción falla	Escribe CADA regla, aunque te parezca obvia
Mezclar tipos de reglas	Paginación/RLS entran como reglas de negocio	Clasifica RN / RT / RS
Entidades hinchadas	Producto termina con 30 campos que no necesita	Solo atributos esenciales del negocio
Descomposición vaga	"Gestionar carrito" no es una operación atómica	Si tiene "y" en el nombre, divídelo



Siguiente paso

Ahora que entiendes FADER, ve a la [práctica de descomposición](#) y aplica el framework a una feature real de Carrito de Compras. Cuando termines el FADER, conviértelo en pruebas verificables con los [criterios de aceptación](#) y la [matriz de trazabilidad](#).

Tiempo estimado de lectura: 40 minutos

Tiempo estimado de práctica: 30-40 minutos

Herramientas: Papel y lápiz

Práctica: Descomponer Feature Carrito de Compras

Aplica el framework FADER a una feature real de Carrito de Compras. Papel y lápiz en mano.

Instrucciones

1. Consigue una **hoja de papel y un lápiz** (o un pizarrón, o una tablet)
2. No abras ningún editor de código. **Cero código.**
3. Sigue cada paso FADER en orden
4. Al final, compara tus respuestas con las soluciones sugeridas

Enunciado

Somos parte del equipo de una tienda online. El equipo de producto nos pide:

Feature: Los clientes deben poder agregar productos a un carrito de compras, ver el resumen con precios actualizados, y aplicar cupones de descuento antes de iniciar el proceso de pago.

Paso a Paso

Paso 0: Alcance

Escribe en tu hoja (consulta [00-alcance-feature.md](#)):

1. **Incluye:** ¿qué cubre el carrito?
2. **No incluye:** ¿qué NO cubre? (pagos, envíos, fidelización...)
3. **Dependencias:** ¿qué necesita que exista antes?
4. **Suposiciones:** ¿qué das por hecho?
5. **Preguntas abiertas:** ¿qué no sabes todavía?

Paso 1: Formular

Escribe en tu hoja:

1. ¿Cuál es la necesidad real del cliente?
2. ¿Qué pasa si esta feature no existe?
3. Tres enunciados tipo "Como [actor]..."

Reflexiona:

- No es lo mismo un carrito "de tienda física" donde puedes tocar los productos, que uno "de e-commerce"
- No es lo mismo un carrito "para una tienda de zapatos" que "para un supermercado mayorista"
- ¿Qué matices aplican a nuestro caso?

Paso 2: Actorizar

Dibuja una tabla de actores:

1. Identifica al menos 3 actores
2. Para cada actor, escribe qué puede y qué NO puede hacer
3. Define los límites: ¿dónde termina el carrito y empieza otro sistema?

Pregúntate:

- ¿El administrador puede modificar el carrito de un cliente?
- ¿El sistema de pagos necesita conocer los items del carrito o solo el total?

Paso 3: Descomponer

Enumera todas las operaciones atómicas usando post-its imaginarios:

1. Opera por operación, pregúntate: ¿esto hace una sola cosa?
2. Clasifícalas en: CRUD, Validación, Cálculo
3. Agrúpalas por actor
4. Identifica dependencias: ¿qué operaciones deben ejecutarse antes que otras?

Paso 4: Entidades

Dibuja las tarjetas de entidad:

1. ¿Qué objetos de negocio existen?
2. ¿Cuáles son sus atributos esenciales?
3. ¿Cómo se relacionan entre sí?
4. ¿Qué NO pertenece a cada entidad?

Paso 5: Reglas

Enuncia al menos 5 reglas de negocio:

1. Usa el formato RN001, RN002... (clasifica técnicas y de seguridad como RT/RS)
2. Clasifica cada una: Restricción, Cálculo, Validación, Flujo

3. Define el mensaje de error para el usuario donde aplique
4. Piensa en bordes: ¿qué pasa si el precio es 0? ¿si la cantidad es negativa?

Solución Sugerida

Compara después de hacerlo tú mismo. No mires antes de terminar.

✓ Alcance

Feature: Carrito de Compras

Incluye:

- Agregar/quitar productos y actualizar cantidades.
- Ver resumen con subtotal, impuesto y total.
- Aplicar cupones de descuento.

No incluye:

- Procesar pagos.
- Envíos y seguimiento.
- Programa de fidelización.

Dependencias:

- Catálogo de productos (stock).
- Autenticación (identidad del cliente).

Suposiciones:

- El cliente tiene sesión iniciada.
- Un carrito pertenece a un solo cliente.

Preguntas abiertas:

- ¿Los cupones son combinables entre sí?
- ¿Hay límite de items por carrito? (RN001)

✓ Formular

Como **cliente con sesión iniciada**, quiero **gestionar mi carrito de compras (agregar, quitar, modificar productos, aplicar cupones y ver el resumen económico)** para **revisar mi pedido antes de pagar**.

Tres enunciados:

- "Como cliente, quiero agregar productos al carrito para acumular mi pedido."
- "Como cliente, quiero ver el total actualizado del carrito para saber cuánto gastaré."
- "Como cliente, quiero aplicar un cupón de descuento para obtener un mejor precio."

✓ Actorizar

Actor	Rol	Permisos
Cliente	Primario	CRUD items, aplicar cupón, ver resumen
Admin	Secundario	Ver carritos abandonados, ajustar precios
Módulo de Inventario	Sistema interno	Validar stock
Pasarela de Pagos	Sistema externo	Solo recibe total y items del checkout

✓ Descomponer

Cliente:

- [C] Agregar producto al carrito
- [D] Quitar producto del carrito
- [U] Actualizar cantidad de un producto
- [R] Ver resumen del carrito (items + totales)
- [Validación] Aplicar cupón de descuento
- [Cálculo] Calcular subtotal
- [Cálculo] Calcular impuesto
- [Cálculo] Calcular total neto

Sistema (automático):

- [Validación] Validar stock antes de agregar
- [Validación] Validar cupón (vigencia, aplicabilidad)
- [Acción] Limpiar carrito después del checkout

Flujo de dependencias:

1. Validar stock → Agregar producto
2. Agregar producto → Calcular subtotal → Calcular impuesto → Calcular total
3. Aplicar cupón → Validar cupón → Recalcular descuento → Recalcular total

✓ Entidades

Producto:

- id, nombre, precio, stock, categoria

ItemCarrito:

- producto (ref Producto), cantidad, precioUnitario

Carrito:

- id, items (List<ItemCarrito>), cuponAplicado, subtotal, descuento, impuesto, total

CuponDescuento:

- codigo, tipo (porcentaje/montoFijo), valor, fechaExpiracion, usoMaximo, usosActuales

✓ Reglas

Código	Descripción	Tipo	Mensaje
RN001	Máximo 50 items distintos por carrito	Restricción	"Has alcanzado el límite de 50 productos"

Código	Descripción	Tipo	Mensaje
RN002	Stock debe ser > 0 para agregar	Validación	"Producto {nombre} sin stock disponible"
RN003	Cantidad a agregar debe ser > 0	Validación	"La cantidad debe ser mayor a 0"
RN004	Impuesto = 16% (subtotal - descuento)	Cálculo	—
RN005	Cupón debe estar vigente (no expirado)	Validación	"El cupón {codigo} ha expirado"
RN006	Descuento máximo 50% del subtotal	Restricción	"El descuento supera el límite permitido"
RN007	Precio unitario se congela al agregar	Consistencia	—
RT001	El carrito se lee con la sesión (sin pasar userId por parámetro)	Técnica	—
RS001	RLS: solo el dueño del carrito puede leerlo/escribirlo	Seguridad	—

¿Qué sigue?

Antes de seguir, revisa tu hoja. ¿Identificaste algo que no está en la solución? ¿La solución tiene algo que se te escapó?

Si encuentras diferencias, no significa que esté mal. El diseño no tiene una única respuesta correcta. Lo importante es el **proceso de pensamiento**.

Cuando estés satisfecho con tu descomposición, continúa con [02-mapeo-capas.md](#) para traducir esto a Clean Architecture.

Tiempo: 30-40 minutos

Material: Papel, lápiz, goma de borrar

Mapeo a Capas de Clean Architecture

Cómo traducir tu descomposición FADER a las capas de Clean Architecture. El puente entre el problema y la solución técnica.

La Matriz de Responsabilidades

Cada pieza de tu descomposición FADER cae en una capa específica. Esta matriz te dice dónde va cada cosa:

MAPEO FADER → CLEAN		
FADER	CAPA	EJEMPLO
Entidades	DOMAIN (entities/)	Producto, Carrito
Reglas de negocio (RN)	DOMAIN (usecases/)	RN001, RN002, RN004
Operaciones (casos de uso)	DOMAIN (usecases/)	AgregarProducto CalcularTotal
Contratos (interfaces)	DOMAIN (repositories/)	CarritoRepository PagoService
Reglas técnicas (RT) y de seguridad (RS)	DATA (datasources/, migrations)	RT001: paginación, RT003: RPC atómico, RS001: RLS
Fuentes de datos	DATA (datasources/)	CarritoRemoteDS CarritoLocalDS
Modelos (DTOs/JSON)	DATA (models/)	CarritoModel ProductoModel
Implementación de contratos	DATA (repositories/)	CarritoRepositoryImpl
Estados de UI	PRESENTATION (cubit/)	CarritoState
Widgets	PRESENTATION (pages/)	CarritoPage ItemCarritoTile

Las reglas técnicas (RT) y de seguridad (RS) viven en DATA (datasource + migración SQL + RLS), no en el dominio. Se diseñan en [05e-diseno-supabase.md](#).

Regla Fundamental

Cada **responsabilidad** tiene una **capa propietaria**. Otras capas pueden *reforzarla* o *representarla*, pero no deben convertirse en una segunda fuente de verdad.

× MAL: "La validación de stock se hace en el UseCase y también en el Widget"
→ La lógica de negocio vive en DOMAIN. La UI solo representa el resultado.

✓ BIEN: "La validación de stock ocurre en el UseCase (fuente de verdad).
El Widget muestra el error si ocurre, pero no decide la regla."

Esto no significa que cada regla exista en un solo lugar del sistema. Una regla de negocio puede estar **reforzada** en varias capas sin que eso sea duplicación incorrecta — es **defensa en profundidad**:

RN004 "Un ticket aprobado no vuelve a aprobarse":

DOMAIN	UseCase valida la regla (fuente de verdad del negocio)
SUPABASE	Constraint + RPC verifican el estado antes de escribir
DATA (RLS)	RS001 impide modificar tickets de otras rifas
UI	El botón "Aprobar" se deshabilita si el ticket ya está aprobado

Cada capa *refuerza* la misma regla, pero solo el UseCase es la **fuentes de verdad** del negocio. Las demás no la redefinen: la protegen o la representan.

Para verificar el diseño completo (que ninguna regla quede sin implementar y ninguna operación sin probar), usa la [matriz de trazabilidad](#).

Proceso de Mapeo

Paso 1: Entidades y Reglas → Domain

Todo lo que es **regla de negocio** o **concepto del mundo real** va a DOMAIN. Esta capa no depende de nada externo.

DOMAIN

entities/

- └─ product.dart ← De tu tarjeta Producto
- └─ cart.dart ← De tu tarjeta Carrito
- └─ cart_item.dart ← De tu tarjeta Item
- └─ coupon.dart ← De tu tarjeta Cupón

usecases/

- └─ add_product_to_cart ← RN002, RN003
- └─ remove_product_from_cart ← sin reglas extra
- └─ update_product_qty ← RN003
- └─ get_cart_summary ← RN004
- └─ apply_coupon ← RN005, RN006
- └─ validate_stock ← RN002

repositories/ ← INTERFACES (contratos)

- └─ cart_repository.dart

services/ ← INTERFACES (contratos)

- └─ payment_service.dart

core/ ← Dependencias transversales del dominio

- └─ failures.dart ← Errores del dominio
- └─ session/ ← Interfaz UserSession
 - └─ user_session.dart

UserSession es una dependencia transversal. Su interface vive en `domain/core/session/` (o `core/session/` a nivel de proyecto) y es inyectada en los repositorios que necesitan saber quién es el usuario autenticado. Ej: `CartRepositoryImpl` recibe `UserSession` para obtener el `userId` del carrito sin que el UseCase tenga que pasarlo como parámetro.

Pregunta de validación: ¿Podría este código vivir en un proyecto Dart sin Flutter?

- ☒ Sí → Va en DOMAIN
- ☐ No (usa contexto de Flutter, depends de paquetes externos) → Va en otra capa

Paso 2: Contratos → Domain (interfaces)

Los **repositorios y servicios** se definen como interfaces abstractas en DOMAIN. Esto es clave: DOMAIN define qué necesita, DATA lo implementa.

```
// DOMAIN define el contrato
abstract class CartRepository {
  Future<Either<Failure, Cart>> getCart(String userId);
  Future<Either<Failure, Cart>> addProduct(Product product, int quantity);
  Future<Either<Failure, Cart>> removeProduct(String productId);
  Future<Either<Failure, Cart>> updateQuantity(String productId, int quantity);
  Future<Either<Failure, Cart>> applyCoupon(String couponCode);
}
```

Regla: El `CartRepository` es una interface. DOMAIN no sabe ni le importa si los datos vienen de Supabase, Firebase, SQLite o un archivo JSON.

Paso 3: Fuentes de Datos → Data

Lo que en FADER identificaste como **sistemas externos** (APIs, BD local, etc.) se convierte en DataSources en la capa DATA.

```
DATA

datasources/
├── cart_remote_data_source.dart ← API/Supabase
└── cart_local_data_source.dart ← SQLite/Hive

models/ ← DTOs que mapean desde/hacia JSON
├── cart_model.dart
├── product_model.dart
└── coupon_model.dart

repositories/ ← Implementaciones
└── cart_repository_impl.dart
```

Regla: DATA implementa las interfaces de DOMAIN. DATA depende de DOMAIN, no al revés.

Paso 4: Estados y Widgets → Presentation

Lo que en FADER identificaste como **interacción del usuario** se mapea a Presentation.

```
PRESENTATION

cubit/
├── cart_cubit.dart
└── cart_state.dart ← Estados de la UI

pages/
├── cart_page.dart
└── cart_summary_page.dart

widgets/
├── cart_item_tile.dart
├── coupon_input.dart
└── cart_total_card.dart
```

Ejemplo Completo de Mapeo

FADER: "Como cliente, quiero agregar un producto al carrito"

DOMAIN

entity: Product (id, name, price, stock)

entity: CartItem (product, quantity, unitPrice)

entity: Cart (items, subtotal, discount, tax, total)

usecase: AddProductToCart

- Llama a CartRepository.addProduct()
- Valida RN002 (stock)
- Valida RN003 (cantidad > 0)

interface: CartRepository

- Future<Either<Failure, Cart>> addProduct(Product, int qty)

failure: AddProductFailure

DATA

model: CartModel (fromJson, toJson)

model: ProductModel (fromJson, toJson)

datasource: CartRemoteDataSource

- Llama a Supabase REST API

repository: CartRepositoryImpl

- Implementa CartRepository usando CartRemoteDataSource

PRESENTATION

state: CartState (loading, loaded, error, addingProduct)

cubit: CartCubit

- addProduct() → Llama a AddProductToCart → Emite nuevo estado

page: CartPage

- Botón "Agregar" → CartCubit.addProduct()

widget: ProductCard

- onTap → CartCubit.addProduct(product)

Errores comunes de mapeo

Error	Síntoma	Solución
Entidad con lógica de serialización	<code>Producto.fromJson()</code> en DOMAIN	<code>fromJson</code> va en Model (DATA)
UseCase que llama a la API directamente	<code>supabaseClient.from(...)</code> en UseCase	El UseCase no sabe de dónde vienen los datos. Usa Repository.
Estado del Cubit que contiene widgets	<code>buttonEnabled</code> , <code>showSuccessDialog</code> en Estado	El estado describe datos, no widgets
Regla de negocio en el Widget	<code>if(product.stock == 0)</code> dentro de un Text	Las reglas van en UseCase, no en UI
Regla técnica/seguridad en el dominio	RT001/RS001 dentro de un UseCase	Las RT/RS van a DATA (datasource; RLS en Supabase o contrato de endpoint en REST)

Error	Síntoma	Solución
Interfaz de Repository en DATA	<code>abstract class CartRepository</code> en DATA	La interface pertenece a DOMAIN
Pasar userId como parámetro en cada método	<code>getCart(String userId)</code> , <code>addProduct(String userId, ...)</code> , etc.	Injectar <code>UserSession</code> en el <code>RepositoryImpl</code> ; el UseCase no necesita saber de dónde viene el userId

Plantilla de Mapeo Rápido

Usa esta tabla para mapear cualquier feature:

Elemento FADER	Capa	Archivo destino
Entidad tal	DOMAIN entities/	<code>entidad.dart</code>
Operación tal	DOMAIN usecases/	<code>operacion.dart</code>
Regla de negocio (RN)	DOMAIN usecases/	(dentro de su UseCase)
Regla técnica (RT) / seguridad (RS)	DATA datasources/ + migración SQL	(datasource + RLS + RPC)
Interface de repositorio	DOMAIN repositories/	<code>repo.dart</code>
Modelo/DTO	DATA models/	<code>modelo_model.dart</code>
Fuente de datos remota	DATA datasources/	<code>remote_ds.dart</code>
Fuente de datos local	DATA datasources/	<code>local_ds.dart</code>
Implementación de repo	DATA repositories/	<code>repo_impl.dart</code>
Estado de UI	PRESENTATION cubit/	<code>feature_state.dart</code>
Lógica de UI	PRESENTATION cubit/	<code>feature_cubit.dart</code>
Pantalla	PRESENTATION pages/	<code>feature_page.dart</code>
Componente	PRESENTATION widgets/	<code>componente_widget.dart</code>

Después de mapear, registra cada operación, regla y contrato en la [matriz de trazabilidad](#).

Siguiendo paso

Ve a la [práctica de mapeo](#) y traduce tu descomposición del Carrito a la estructura de carpetas de Clean Architecture.

Tiempo estimado de lectura: 20 minutos

Tiempo estimado de práctica: 30-40 minutos

Práctica: Mapear Carrito de Compras a Clean Architecture

Traduce tu descomposición FADER del Carrito a la estructura de carpetas y capas de Clean Architecture.

Instrucciones

1. Toma la hoja FADER que completaste en la práctica anterior
2. Sin abrir el editor, dibuja el árbol de carpetas en papel
3. Identifica qué archivos van en cada capa
4. Al final, compara con la solución sugerida

Enunciado

Usando la descomposición FADER del Carrito de Compras, mapea cada elemento a las capas de Clean Architecture.

Paso a Paso



Paso 1: Identifica qué va en DOMAIN

Revisa tu hoja FADER. Separa:

- **Entidades:** ¿Cuáles son los conceptos de negocio puros?
- **UseCases (operaciones + reglas):** ¿Cada operación atómica de la descomposición es un UseCase?
- **Interfaces (repositories/services):** ¿Qué contratos necesita tu dominio?

Dibuja el árbol de la carpeta `domain/`.

Pregúntate:

- ¿Esta entidad necesita saber de Flutter? → No → Va en DOMAIN ✓
- ¿Este UseCase podría testearse sin conexión a internet? → Sí → Va en DOMAIN ✓
- ¿Esta interface expone lo necesario sin atarse a una tecnología? → Sí → Va en DOMAIN ✓

Paso 2: Identifica qué va en DATA

Revisa los actores externos de tu FADER:

- **Fuentes de datos:** ¿De dónde vienen los datos? (API, BD local, cache)
- **Modelos:** ¿Cómo se serializan/deserializan las entidades?
- **Implementaciones:** ¿Quién implementa los contratos de DOMAIN?

Dibuja el árbol de la carpeta `data/`.

Pregúntate:

- ¿Esto cambia si cambio de Supabase a Firebase? → Sí → Va en DATA ✓
- ¿Esto es un detalle de implementación? → Sí → Va en DATA ✓

Paso 3: Identifica qué va en PRESENTATION

Revisa los actores humanos de tu FADER:

- **Estados:** ¿Qué estados visuales puede tener la feature? (cargando, datos, error, vacío)
- **Cubits/BLoCs:** ¿Quién orquesta la lógica de UI?
- **Widgets:** ¿Qué pantallas y componentes necesita el usuario?

Dibuja el árbol de la carpeta `presentation/`.

Pregúntate:

- ¿Esto es específico de Flutter? → Sí → Va en PRESENTATION ✓
 - ¿Esto cambia si paso de mobile a web? → Sí → Va en PRESENTATION ✓
-

Solución Sugerida

✓ Estructura DOMAIN

```
lib/
├── cart/
│   └── domain/
│       ├── entities/
│       │   ├── product.dart
│       │   ├── cart_item.dart
│       │   ├── cart.dart
│       │   └── coupon.dart
│       ├── usecases/
│       │   ├── add_product_to_cart.dart      ← RN002, RN003
│       │   ├── remove_product_from_cart.dart
│       │   ├── update_product_quantity.dart  ← RN003
│       │   ├── get_cart_summary.dart         ← RN004
│       │   ├── apply_coupon.dart            ← RN005, RN006
│       │   └── validate_stock.dart           ← RN002 (useCase separado)
│       ├── repositories/
│       │   └── cart_repository.dart          ← interface
│       └── core/
│           └── failures.dart                 ← Errores del dominio
```

Detalle de failures:

```
// failures.dart
sealed class Failure {}

final class CartLimitExceeded extends Failure {}
final class ProductOutOfStock extends Failure {}
final class InvalidQuantity extends Failure {}
final class ExpiredCoupon extends Failure {}
final class DiscountLimitExceeded extends Failure {}
final class CartNotFound extends Failure {}
final class UnknownFailure extends Failure {}
```

Detalle de entidad Cart:

```
Cart
├── id: String
├── items: List<CartItem>
├── subtotal: double      → calculado: suma de (precioUnitario * cantidad)
├── discount: double      → 0 si no hay cupón
├── tax: double           → 16% de (subtotal - discount)
├── total: double         → subtotal - discount + tax
└── couponCode: String?   → cupón aplicado (si hay)
```

✓ Estructura DATA

```
lib/
└─ cart/
   └─ data/
      ├── datasources/
      │   ├── cart_remote_data_source.dart ← Supabase
      │   └── cart_local_data_source.dart ← Cache local
      ├── models/
      │   ├── cart_model.dart ← fromJson/toJson
      │   ├── product_model.dart
      │   ├── cart_item_model.dart
      │   └── coupon_model.dart
      └── repositories/
          └── cart_repository_impl.dart ← Implementa CartRepository
```

Ejemplo de modelo:

```
// cart_model.dart (solo estructura, sin implements)
class CartModel {
  final String id;
  final List<CartItemModel> items;
  final String? couponCode;

  CartModel({
    required this.id,
    required this.items,
    this.couponCode,
  });

  factory CartModel.fromJson(Map<String, dynamic> json) =>
    CartModel(
      id: json['id'] as String,
      items: (json['items'] as List)
        .map((e) => CartItemModel.fromJson(e as Map<String, dynamic>))
        .toList(),
      couponCode: json['coupon_code'] as String?,
    );

  Map<String, dynamic> toJson() => {
    'id': id,
    'items': items.map((e) => e.toJson()).toList(),
    'coupon_code': couponCode,
  };
}
```

✓ Estructura PRESENTATION

```
lib/
├── cart/
│   └── presentation/
│       ├── cubit/
│       │   ├── cart_cubit.dart
│       │   └── cart_state.dart
│       ├── pages/
│       │   └── cart_page.dart
│       └── widgets/
│           ├── cart_item_tile.dart
│           ├── cart_summary_card.dart
│           ├── coupon_input.dart
│           └── empty_cart_placeholder.dart
```

Estados del Cubit:

```
CartState
├── CartInitial      → No hay datos aún
├── CartLoading      → Cargando desde la fuente
├── CartLoaded       → Datos disponibles
│   ├── cart: Cart
│   └── isCheckedOut: bool
├── CartError        → Error
│   └── message: String
├── CartActionLoading → Acción en curso (agregar, quitar, etc.)
│   ├── cart: Cart      → Estado anterior (para no perder UI)
│   └── loadingItemId: String?
```

✓ Matriz de trazabilidad

Conecta operación → UseCase → regla → contrato → test (teoría en [05f-criterios-aceptacion-trazabilidad.md](#)):

Operación	UseCase	Regla	Contrato	Test
Agregar producto	AddProductToCart	RN002, RN003	cartRepository.addProduct()	Unit + integration
Quitar producto	RemoveProductFromCart	—	cartRepository.removeProduct()	Unit
Actualizar cantidad	UpdateProductQuantity	RN003	cartRepository.updateQuantity()	Unit
Ver resumen	GetCartSummary	RN004	cartRepository.getCart()	Unit + widget
Aplicar cupón	ApplyCoupon	RN005, RN006	cartRepository.applyCoupon()	Unit + integration
Validar stock	ValidateStock	RN002	cartRepository.addProduct()	Unit
Acceso al carrito	—	RS001	cartRepository.getCart() (RLS)	Integration

Operación	UseCase	Regla	Contrato	Test
Lectura paginada	—	RT001	<code>cartRemoteDataSource</code>	—

Nota: RT001 y RS001 no generan UseCase ni archivo en DOMAIN. Se implementan en DATA (datasource + RLS/migración) — ver [05e-diseno-supabase.md](#).

Verificación: Lista de chequeo

Al terminar, verifica:

- ☐ ¿Cada entidad de FADER tiene su archivo en `domain/entities/` ?
- ☐ ¿Cada operación atómica tiene su UseCase en `domain/usecases/` ?
- ☐ ¿Las interfaces de repositorio están en `domain/repositories/` ?
- ☐ ¿Los errores del dominio están tipados en `domain/core/failures.dart` ?
- ☐ ¿Los modelos tienen `fromJson` / `toJson` en `data/models/` ?
- ☐ ¿Las fuentes de datos están separadas (remota/local) en `data/datasources/` ?
- ☐ ¿La implementación del repo está en `data/repositories/` ?
- ☐ ¿Los estados del Cubit reflejan todas las situaciones de UI?
- ☐ ¿Cada operación, regla y contrato está en la matriz de trazabilidad?
- ☐ ¿Las reglas RT/RS están en DATA (datasource/RLS) y no en el dominio?
- ☐ ¿No hay código de DOMAIN dependiendo de DATA o PRESENTATION?
- ☐ ¿No hay código de DATA o PRESENTATION en DOMAIN?

Siguiendo paso

Ahora que tienes la estructura de capas definida, ve a [03-contratos-primero.md](#) para aprender a diseñar los contratos (interfaces) antes de implementar.

Tiempo: 30-40 minutos

Material: Papel y lápiz + tu hoja FADER de la práctica anterior

Contract-First Design: Primero los Contratos

Aprende a diseñar las interfaces (contratos) entre capas antes de implementar cualquier funcionalidad. El arte de definir el "qué" antes del "cómo".

¿Qué es Contract-First Design?

Es la práctica de **definir las interfaces públicas** de cada capa **antes** de escribir cualquier implementación. Así como un contrato legal define qué debe hacer cada parte sin decir cómo lo hará.

× ENFOQUE TRADICIONAL:
"Ya sé cómo voy a implementar el carrito, empiezo por el DataSource y subo"
→ Terminas con código acoplado y difícil de testear

✓ CONTRACT-FIRST:
"Primero defino cómo se comunican las capas, luego implemento cada una"
→ Terminas con código desacoplado y testeable

Los 3 Contratos Clave

En Clean Architecture hay 3 contratos principales que debes diseñar primero:

1. Contrato de Repositorio (Domain → Data)

Define **qué datos necesita el dominio**, sin importar de dónde vengan.

```
// 🚫 CONTRATO: Lo que el DOMINIO necesita
abstract class CartRepository {
    Future<Either<Failure, Cart>> getCart(String userId);
    Future<Either<Failure, Cart>> addProduct({
        required String userId,
        required Product product,
        required int quantity,
    });
    Future<Either<Failure, Cart>> removeProduct({
        required String userId,
        required String productId,
    });
    Future<Either<Failure, Cart>> updateQuantity({
        required String userId,
        required String productId,
        required int quantity,
    });
    Future<Either<Failure, Cart>> applyCoupon({
        required String userId,
        required String couponCode,
    });
}
```

Alternativa con UserSession: Inyectando `UserSession` en `CartRepositoryImpl`, el contrato puede omitir `userId` — el repository lo obtiene internamente: `getCart()` en vez de `getCart(String userId)`. Esto simplifica el contrato cuando el `userId` pertenece siempre al usuario autenticado. La contraparte: los tests deben asegurar que `UserSession` retorne el `userId` correcto.

Preguntas para diseñar este contrato:

Pregunta	Decisión de diseño
¿El método retorna el <code>Cart</code> completo o solo el cambio?	Retornar el Cart completo mantiene la UI consistente
¿Parámetros individuales o un objeto <code>ValueObject</code> ?	Para 3+ parámetros, usa named parameters
¿Manejo de errores con excepciones o <code>Either</code> ?	<code>Either</code> es más explícito (como se usa en fpdart)
¿El <code>userId</code> se pasa siempre o se obtiene del contexto?	Depende: si el <code>userId</code> lo necesita el contrato (Domain), inyecta <code>UserSession</code> en el <code>RepositoryImpl</code> y el contrato no lo recibe. Si el <code>userId</code> lo necesita el negocio (UseCase), pásalo como parámetro. En general, el contrato del Repository no debe recibir <code>userId</code> porque es un detalle de implementación — el <code>RepositoryImpl</code> lo obtiene de <code>UserSession</code> .

2. Contrato de DataSource (Data → API/DB)

Define **qué operaciones de bajo nivel** necesita el repositorio.

```
// 🚩 CONTRATO: Lo que DATA necesita del mundo exterior
abstract class CartRemoteDataSource {
    Future<CartModel> fetchCart(String userId);
    Future<CartModel> addItem(String userId, Map<String, dynamic> itemData);
    Future<CartModel> removeItem(String userId, String productId);
    Future<CartModel> updateItemQuantity(
        String userId, String productId, int quantity);
    Future<CartModel> applyCoupon(String userId, String couponCode);
}

abstract class CartLocalDataSource {
    Future<CartModel> getCacheCart(String userId);
    Future<void> cacheCart(String userId, CartModel cart);
    Future<void> clearCart(String userId);
}
```

3. Contrato del UseCase (Presentation → Domain)

Define **qué operaciones de negocio** están disponibles para la UI.

```
// 🚩 CONTRATO: Lo que PRESENTATION necesita del DOMINIO
abstract class AddProductToCartUseCase {
    Future<Either<Failure, Cart>> call({
        required String userId,
        required Product product,
        required int quantity,
    });
}

abstract class GetCartSummaryUseCase {
    Future<Either<Failure, Cart>> call(String userId);
}

abstract class ApplyCouponUseCase {
    Future<Either<Failure, Cart>> call({
        required String userId,
        required String couponCode,
    });
}
```

El Proceso Contract-First en 4 pasos

Paso 1: Identifica los puntos de comunicación

Dibuja las flechas de comunicación entre capas:

```
PRESENTATION  —llama—>  DOMAIN (UseCase)
DOMAIN        —llama—>  DOMAIN (Repository interface)
DATA (Repo)   —llama—>  DATA (DataSource interface)
DATA (DS)     —llama—>  Sistema externo (API, DB)
```

Cada flecha es un **contrato** que debes diseñar.

Paso 2: Diseña la interface ideal

Sin pensar en la implementación, responde:

- **¿Qué método expone?** → Cada operación atómica del FADER
- **¿Qué parámetros recibe?** → Lo mínimo indispensable
- **¿Qué retorna?** → El tipo de dato o un Result/Either
- **¿Qué errores puede producir?** → Los failures del dominio

Paso 3: Valida el contrato

Revisa que el contrato sea:

- **Independiente de tecnología:** No menciones HTTP, Supabase, SQL, etc.
- **Completo:** Cubre todas las operaciones de la descomposición FADER
- **Consistente:** Sigue el mismo patrón en todos los métodos
- **Testeable:** Puedes crear un Fake/Mock fácilmente

Paso 4: Congela el contrato

Una vez definido, el contrato NO cambia por decisiones de implementación.

× "Necesito agregar un parámetro 'cacheKey' al contrato porque en la implementación me conviene tenerlo"

✓ "Necesito que el contrato reciba 'userId' para identificar el carrito. Cómo se cachea internamente es problema de la implementación."

Architecture Decision Records (ADR)

Los ADR son documentos cortos que registran las decisiones de arquitectura importantes. Úsalos para congelar tus contratos.

Formato de un ADR:

ADR-001: Contrato del Repositorio de Carrito

Contexto

Necesitamos definir cómo el dominio accede a los datos del carrito.

Decisión

Usaremos una interfaz abstracta `CartRepository` en `domain/repositories/`.

Cada método retorna `Future<Either<Failure, Cart>>`.

El carrito completo se retorna siempre, no solo el cambio.

Consecuencias

Positivas:

- El dominio no depende de la fuente de datos
- Fácil de testear con fakes
- Podemos cambiar de Supabase a Firebase sin tocar el dominio

Negativas:

- Retornar el carrito completo puede ser ineficiente en carritos muy grandes
- Más boilerplate inicial

Alternativas consideradas

1. Retornar solo el item cambiado → Descartado: inconsistencia de estado
2. Usar excepciones en vez de Either → Descartado: menos explícito
3. No usar interface → Descartado: viola Clean Architecture

¿Cuándo escribir un ADR?

- Cuando defines un contrato nuevo
 - Cuando cambias un contrato existente
 - Cuando eliges entre alternativas de diseño
 - Cuando hay trade-offs involucrados
-

Ejemplo: ADR del Carrito

```
# ADR-002: Modelo de Estados del Cubit

## Contexto
El Cubit del carrito necesita representar los estados de la UI.

## Decisión
Usaremos un sealed class con 5 estados:
- CartInitial
- CartLoading
- CartLoaded (contiene cart + isCheckedOut)
- CartError (contiene message)
- CartActionLoading (contiene cart anterior + loadingItemId)

## Consecuencias
Positivas:
- Cada estado posible está tipado
- La UI no puede renderizar un estado inválido
- Fácil de testear cada estado por separado

Negativas:
- Más clases que un solo estado genérico con flags

## Alternativas consideradas
1. Un solo estado con `isLoading`, `error`, etc. → Descartado: estados inválidos posibles
2. Freezed → Descartado: preferimos explícito sobre generado
```

Plantilla de Contrato

Usa esta plantilla para diseñar cualquier contrato:

```
/// [DESCRIPCIÓN DEL CONTRATO]
///
/// Define cómo [CAPA_ORIGEN] se comunica con [CAPA_DESTINO].
///
/// Responsabilidades:
/// - [RESPONSABILIDAD_1]
/// - [RESPONSABILIDAD_2]
///
/// No responsabilidades:
/// - [NO_RESPONSABILIDAD_1]
///
abstract class [NombreContrato] {
    /// [DESCRIPCIÓN DEL MÉTODO]
    ///
    /// [QUÉ HACE]
    /// [QUÉ RETORNA EN ÉXITO]
    /// [QUÉ RETORNA EN ERROR]
    Future<Either<Failure, [TipoExitoso]>> [metodo]({
        required [param1Type] [param1Name],
        required [param2Type] [param2Name],
    });
}
```

Errores comunes

Error	Síntoma	Solución
Contrato atado a tecnología	<code>fetchCartFromSupabase()</code>	Renómbralo a <code>fetchCart()</code>
Contrato con demasiados métodos	15 métodos en un repositorio	Divide en repositorios más pequeños
Contrato que cambia por la implementación	Agregas params que solo usa un DataSource	El contrato es del dominio, no del DataSource
Reglas técnicas/seguridad en el contrato de dominio	<code>fetchCartPaginado(page, limit)</code> en el Repository	La paginación es RT: va en el DataSource, no en el dominio
No documentar decisiones	"¿Por qué retornamos el carrito completo?"	Escribe un ADR
Contrato sin manejo de errores	Retorna <code>Cart?</code> en vez de <code>Either</code>	Define failures explícitos
Contrato sin trazabilidad	Operaciones sin UseCase o UseCase sin test	Regístralo en la matriz de trazabilidad

Siguiente paso

Ve a la [práctica de contratos](#) y diseña los contratos del Carrito de Compras usando lo aprendido aquí.

Tiempo estimado de lectura: 20 minutos

Tiempo estimado de práctica: 30 minutos

Práctica: Contratos del Carrito de Compras

Diseña los contratos (interfaces) del Carrito de Compras antes de implementar cualquier cosa.

Instrucciones

1. Toma tu estructura de capas de la práctica anterior
2. En papel, diseña **las interfaces** de cada capa
3. No escribas implementaciones. Solo contratos
4. Al final, escribe un ADR para la decisión más importante
5. Compara con la solución sugerida

Enunciado

Usando la descomposición FADER y la estructura de capas que definiste, diseña:

1. El contrato de `CartRepository` (Domain)
2. El contrato de `CartRemoteDataSource` y `CartLocalDataSource` (Data)
3. Los contratos de `AddProductToCartUseCase` y `ApplyCouponUseCase` (Domain)
4. El contrato del `CartState` (Presentation)
5. Un ADR para justificar por qué el UseCase retorna `Either<Failure, Cart>` en vez de lanzar excepciones

Paso a Paso

Paso 1: Contrato del Repositorio

Diseña la interfaz de `CartRepository`. Considera:

- ¿Qué métodos necesita?
- ¿Qué parámetros recibe cada método?
- ¿Qué retorna?
- ¿Qué failures puede producir?

Recuerda: esto es un contrato, no una implementación. **No menciones HTTP, SQL, ni ninguna tecnología.**

Paso 2: Contratos de DataSources

Diseña `CartRemoteDataSource` y `CartLocalDataSource`. Considera:

- El DataSource remoto habla con una API (REST/GraphQL)
- El DataSource local habla con SQLite, Hive o SharedPreferences
- ¿Usan los mismos tipos de datos?
- ¿Cómo separas responsabilidades entre remoto y local?

Paso 3: Contratos de UseCases

Diseña al menos:

- `AddProductToCartUseCase`: ¿Qué validaciones incluye?
- `ApplyCouponUseCase`: ¿Qué reglas de negocio aplica?

Usa el patrón de `call()` para que el Cubit pueda invocarlos directamente.

Paso 4: Contrato del Estado

Diseña los estados del Cubit:

- ¿Qué estados puede tener el carrito?
- ¿Qué datos lleva cada estado?
- ¿Cómo evitas estados inválidos?

Paso 5: Escribe un ADR

Redacta un ADR para la decisión más relevante de tus contratos.

Solución Sugerida

✓ CartRepository

```
// domain/repositories/cart_repository.dart

/// Contrato para la gestión de datos del carrito.
///
/// Define las operaciones que el dominio necesita para persistir
/// y recuperar datos del carrito, independientemente de la fuente.
///
/// Responsabilidades:
/// - Persistir items del carrito
/// - Aplicar cupones de descuento
/// - Recuperar el estado actual del carrito
///
/// No responsabilidades:
/// - Validar reglas de negocio (eso es responsabilidad del UseCase)
/// - Conocer la fuente de datos (API, BD local, etc.)
///
abstract class CartRepository {
  /// Obtiene el carrito de un usuario.
  Future<Either<Failure, Cart>> getCart(String userId);

  /// Agrega un producto al carrito.
  Future<Either<Failure, Cart>> addProduct({
    required String userId,
    required Product product,
    required int quantity,
  });

  /// Elimina un producto del carrito.
  Future<Either<Failure, Cart>> removeProduct({
    required String userId,
    required String productId,
  });

  /// Actualiza la cantidad de un producto en el carrito.
  Future<Either<Failure, Cart>> updateQuantity({
    required String userId,
    required String productId,
    required int quantity,
  });

  /// Aplica un cupón de descuento al carrito.
  Future<Either<Failure, Cart>> applyCoupon({
    required String userId,
    required String couponCode,
  });
}
```

✓ CartRemoteDataSource

```
// data/datasources/cart_remote_data_source.dart

/// Fuente de datos remota para el carrito.
///
/// Se comunica con Supabase (REST API) para operaciones CRUD.
/// Los modelos retornados ya vienen serializados de la API.
///
abstract class CartRemoteDataSource {
  Future<CartModel> fetchCart(String userId);
  Future<CartModel> addItem(String userId, Map<String, dynamic> itemData);
  Future<CartModel> removeItem(String userId, String productId);
  Future<CartModel> updateItemQuantity(
    String userId, String productId, int quantity);
  Future<CartModel> applyCoupon(String userId, String couponCode);
}
```

✓ CartLocalDataSource

```
// data/datasources/cart_local_data_source.dart

/// Fuente de datos local (caché) para el carrito.
///
/// Almacena el carrito en SQLite/Hive para acceso offline.
///
abstract class CartLocalDataSource {
  Future<CartModel?> getCachedCart(String userId);
  Future<void> cacheCart(String userId, CartModel cart);
  Future<void> clearCache(String userId);
}
```

✓ UseCases

```
// domain/usecases/add_product_to_cart.dart

/// Agrega un producto al carrito.
///
/// Valida:
/// - Que el producto tenga stock disponible (RN002)
/// - Que la cantidad sea mayor a 0 (RN003)
/// - Que el carrito no exceda 50 items (RN001)
///
class AddProductToCart {
  final CartRepository repository;

  const AddProductToCart(this.repository);

  Future<Either<Failure, Cart>> call({
    required String userId,
    required Product product,
    required int quantity,
  }) {
    // Las validaciones se hacen aquí, en el dominio
    if (quantity <= 0) return Left(InvalidQuantity());
    if (product.stock <= 0) return Left(ProductOutOfStock());

    return repository.addProduct(
      userId: userId,
      product: product,
      quantity: quantity,
    );
  }
}
```

```
// domain/usecases/apply_coupon.dart

/// Aplica un cupón de descuento al carrito.
///
/// Valida:
/// - Que el cupón no esté expirado (RN005)
/// - Que el descuento no exceda el 50% (RN006)
///
class ApplyCoupon {
  final CartRepository repository;

  const ApplyCoupon(this.repository);

  Future<Either<Failure, Cart>> call({
    required String userId,
    required String couponCode,
  }) {
    return repository.applyCoupon(
      userId: userId,
      couponCode: couponCode,
    );
  }
}
```

✓ CartState

```
// presentation/cubit/cart_state.dart

sealed class CartState {}

final class CartInitial extends CartState {}

final class CartLoading extends CartState {}

final class CartLoaded extends CartState {
  final Cart cart;
  final bool isCheckingOut;

  const CartLoaded({
    required this.cart,
    this.isCheckingOut = false,
  });
}

final class CartError extends CartState {
  final String message;

  const CartError(this.message);
}

final class CartActionLoading extends CartState {
  final Cart cart;
  final String? loadingItemId;

  const CartActionLoading({
    required this.cart,
    this.loadingItemId,
  });
}
```

✓ ADR-001

ADR-001: Uso de Either en Contract-First

Contexto

Necesitamos definir cómo los contratos del dominio manejan errores.

Decisión

Todos los métodos de repositorio y usecases retornarán
`Future<Either<Failure, T>>` en vez de lanzar excepciones.

Consecuencias

Positivas:

- El tipo de retorno es explícito: el llamante sabe que puede fallar
- Los failures están tipados: se puede reaccionar distinto a cada error
- No hay excepciones inesperadas en tiempo de ejecución
- La UI puede mapear cada Failure a un mensaje distinto

Negativas:

- Más código que con excepciones
- Curva de aprendizaje para desarrolladores nuevos en Either

Alternativas consideradas

1. Lanzar excepciones → Descartado: no son explícitas en el tipo de retorno
2. Retornar null en error → Descartado: perdemos información del error
3. Callbacks onSuccess/onError → Descartado: menos composable

Verificación: Lista de chequeo

- ☐ ¿Cada contrato tiene un nombre descriptivo?
- ☐ ¿Los contratos de dominio no mencionan tecnologías específicas?
- ☐ ¿Cada método retorna un tipo explícito (Either)?
- ☐ ¿Los failures están tipados y son específicos?
- ☐ ¿Los parámetros son los mínimos necesarios?
- ☐ ¿El DataSource remoto usa modelos en vez de entidades?
- ☐ ¿Las reglas técnicas/seguridad (RT/RS) no se colaron en el contrato de dominio?
- ☐ ¿El ADR documenta la decisión más importante?
- ☐ ¿Los estados del Cubit cubren todos los escenarios?
- ☐ ¿Cada contrato está registrado en la matriz de trazabilidad?

Siguiendo paso

Ahora que tienes los contratos definidos, ve a [04-flujo-datos.md](#) para aprender a diseñar el flujo de datos entre las capas.

Tiempo: 30 minutos

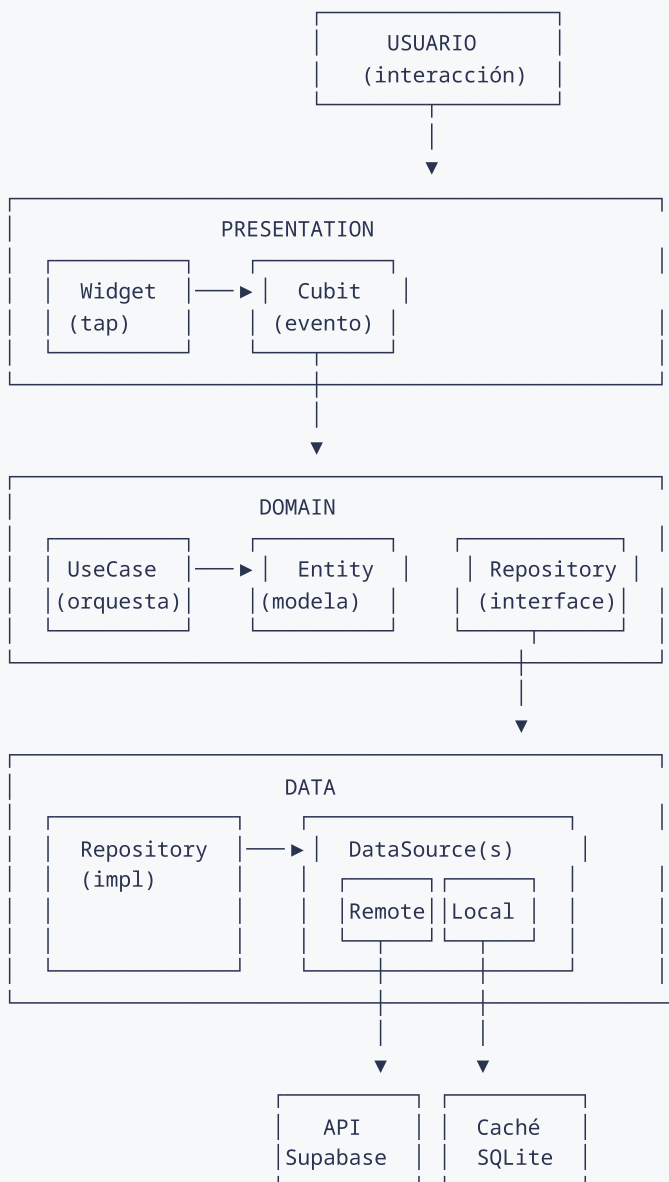
Material: Papel y lápiz + estructura de capas de la práctica anterior

Flujo de Datos entre Capas

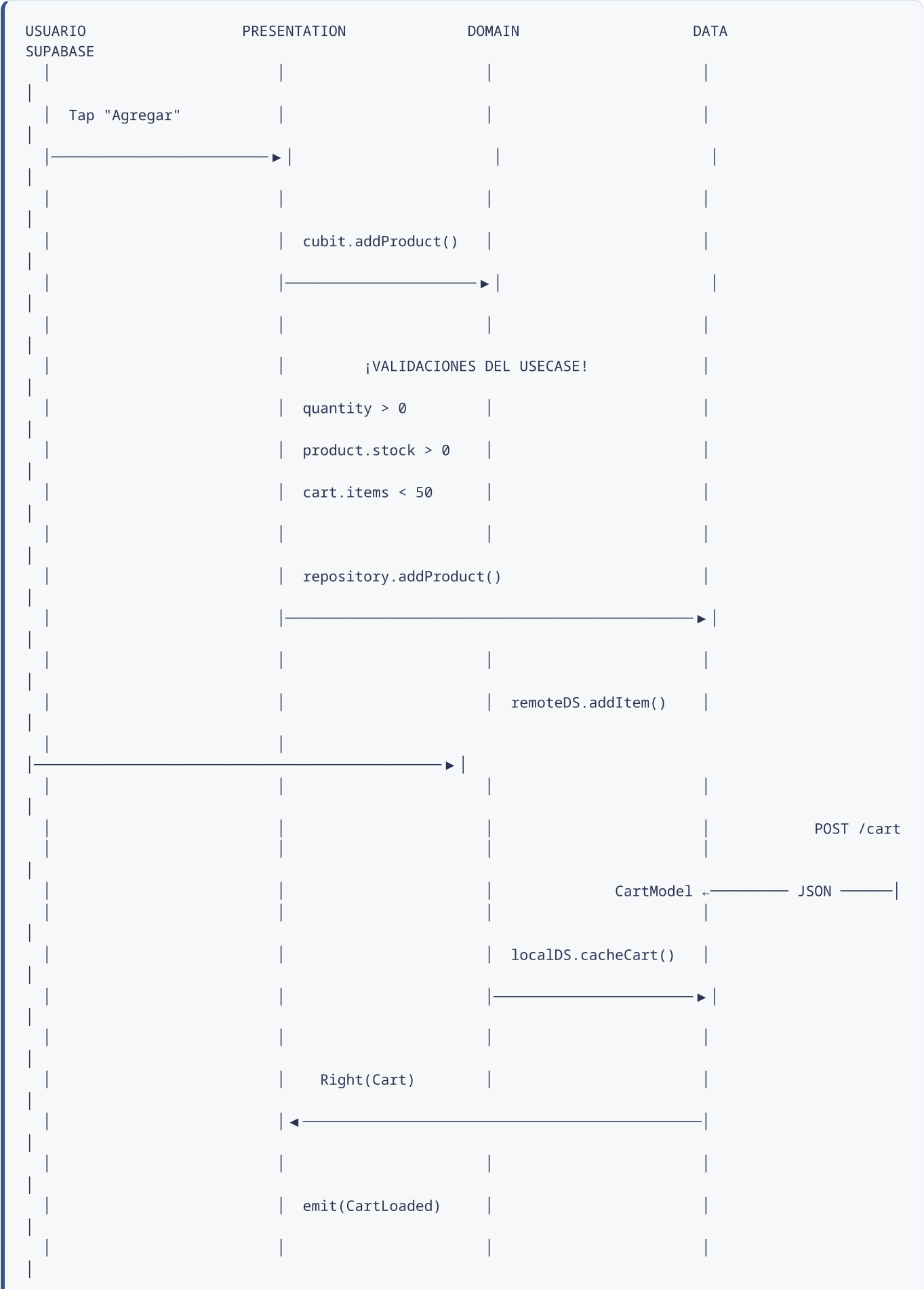
Diseña cómo viajan los datos a través de las capas de Clean Architecture. De la UI a la base de datos y de vuelta.

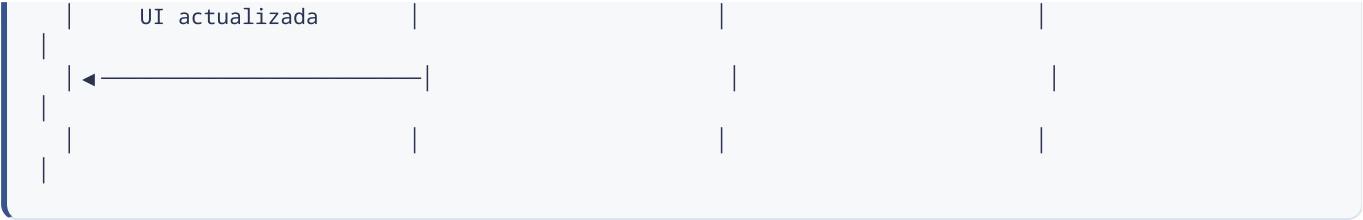
El Viaje de los Datos

Cada operación en Clean Architecture sigue un flujo en forma de **U**:



Secuencia Completa: Agregar Producto al Carrito



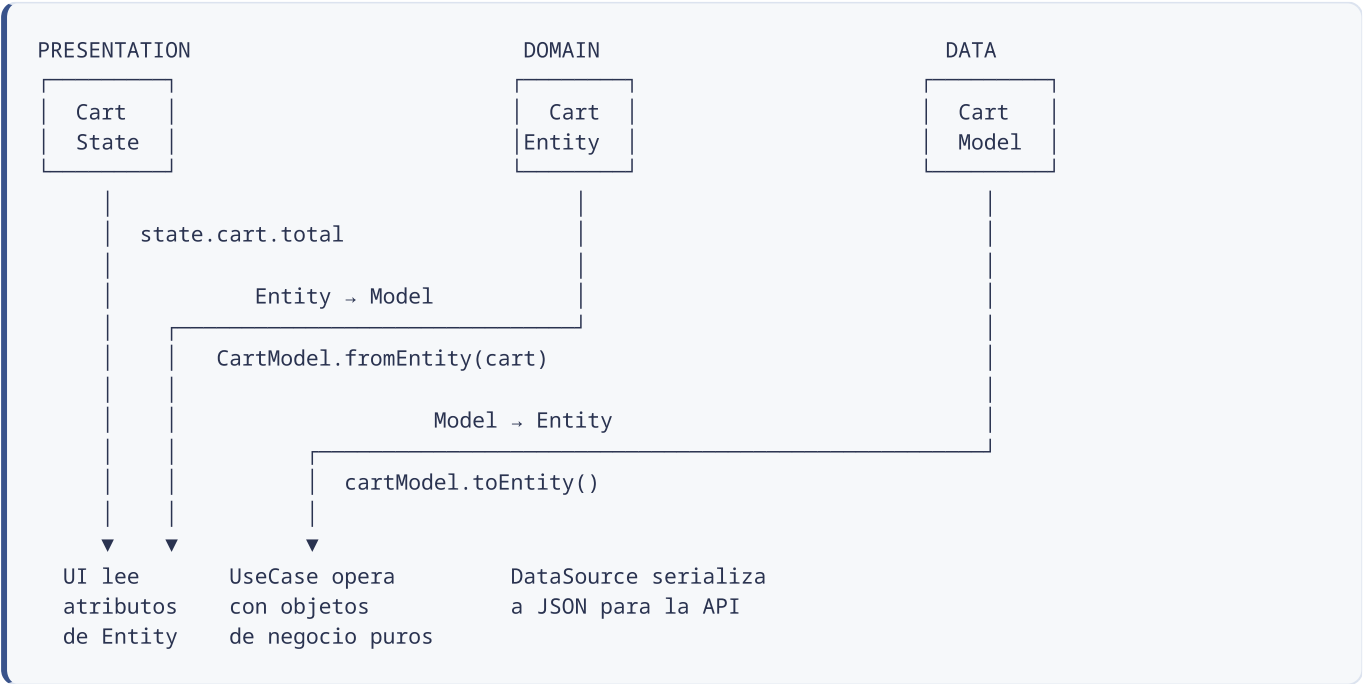


Mapecto de Tipos entre Capas

Cada capa usa sus propios tipos. Nunca pasa un `Model` a la UI ni una `Entity` al DataSource.

CAPA	TIPO DE ENTRADA	TIPO DE SALIDA
Widget	UserAction	Event (a Cubit)
Cubit	Event	Call UseCase
UseCase	Input params	Either<Failure, Entity>
Repository (interface)	Domain params	Either<Failure, Entity>
RepositoryImpl	Domain params	Either<Failure, Entity>
DataSource	Model params	Model
API/DB	JSON/SQL	JSON/Row

Regla de transformaci3n:



Ejemplo: Flujo de ApplyCoupon

1. Widget detecta acción

```
// El usuario escribe un código de cupón y presiona "Aplicar"
onPressed: () => cubit.applyCoupon(codeController.text)
```

2. Cubit orquesta

```
void applyCoupon(String code) async {
  emit(CartActionLoading(cart: state.cart));

  final result = await _applyCouponUseCase(
    userId: userId,
    couponCode: code,
  );

  result.fold(
    (failure) => emit(CartError(_mapFailureToMessage(failure))),
    (cart) => emit(CartLoaded(cart: cart)),
  );
}
```

3. UseCase valida reglas

```
Future<Either<Failure, Cart>> call({
  required String userId,
  required String couponCode,
}) async {
  // RN005: Cupón vigente
  if (couponCode.isEmpty) return Left(InvalidCoupon());

  // Delega la validación de expiración al repositorio
  return repository.applyCoupon(
    userId: userId,
    couponCode: couponCode,
  );
}
```

4. RepositoryImpl orquesta fuentes

```
Future<Either<Failure, Cart>> applyCoupon({
    required String userId,
    required String couponCode,
}) async {
    try {
        // 1. Llama al remoto
        final cartModel = await remoteDataSource.applyCoupon(userId, couponCode);

        // 2. Actualiza caché local
        await localDataSource.cacheCart(userId, cartModel);

        // 3. Convierte a entidad y retorna
        return Right(cartModel.toEntity());
    } on CartException catch (e) {
        return Left(CouponError(e.message));
    }
}
```

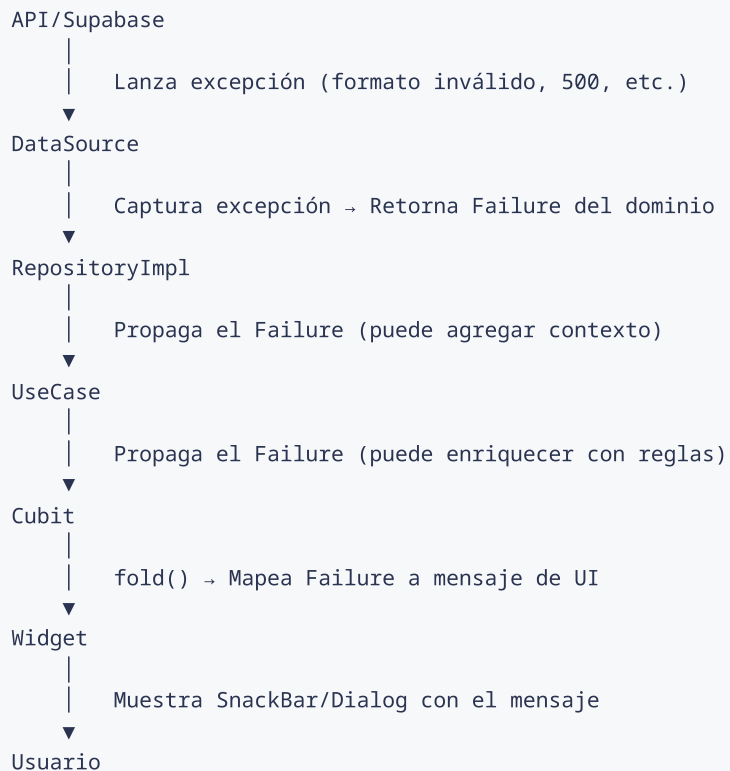
5. DataSource llama a la API

```
Future<CartModel> applyCoupon(String userId, String couponCode) async {
    final response = await supabase
        .from('carts')
        .update({'coupon_code': couponCode})
        .eq('user_id', userId)
        .select()
        .single();

    return CartModel.fromJson(response);
}
```

Flujo de Errores

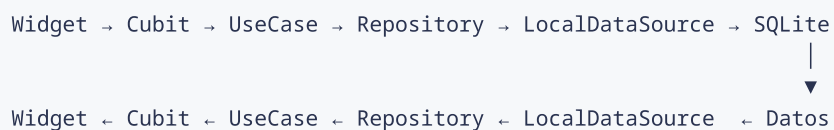
Los errores también fluyen de abajo arriba:



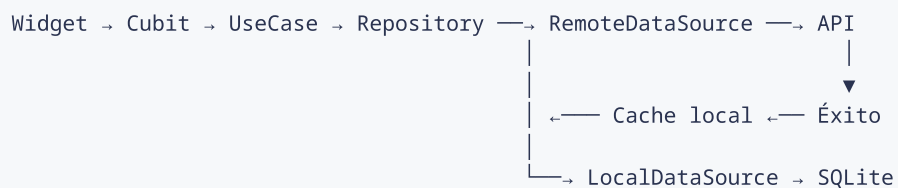
Regla: Nunca dejes que una excepción técnica (HTTP 500, timeout, SQL error) llegue a la UI. Cada capa traduce el error al nivel de abstracción correspondiente.

Estrategias de Flujo

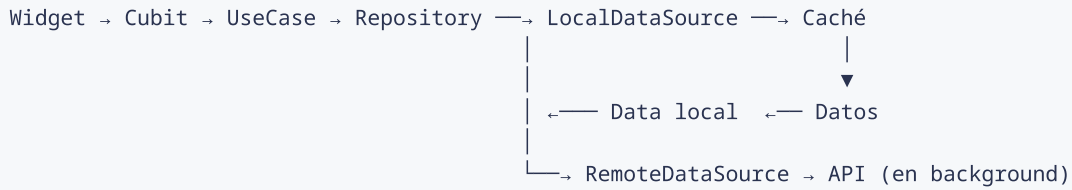
Flujo Síncrono (lectura de caché)



Flujo con Red (escritura remota + caché)

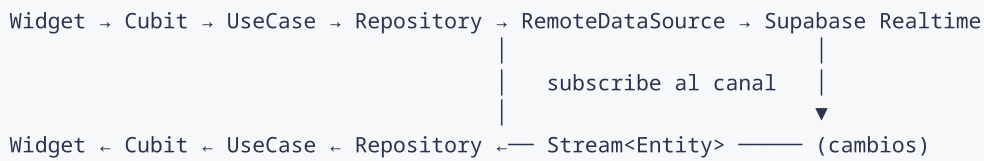


Flujo Offline-First



Flujo Realtime (Supabase Streams)

Cuando otros usuarios pueden cambiar los datos mientras la pantalla está abierta, el flujo clásico "pido → recibo" no basta. El DataSource expone un **Stream** y el Cubit escucha.

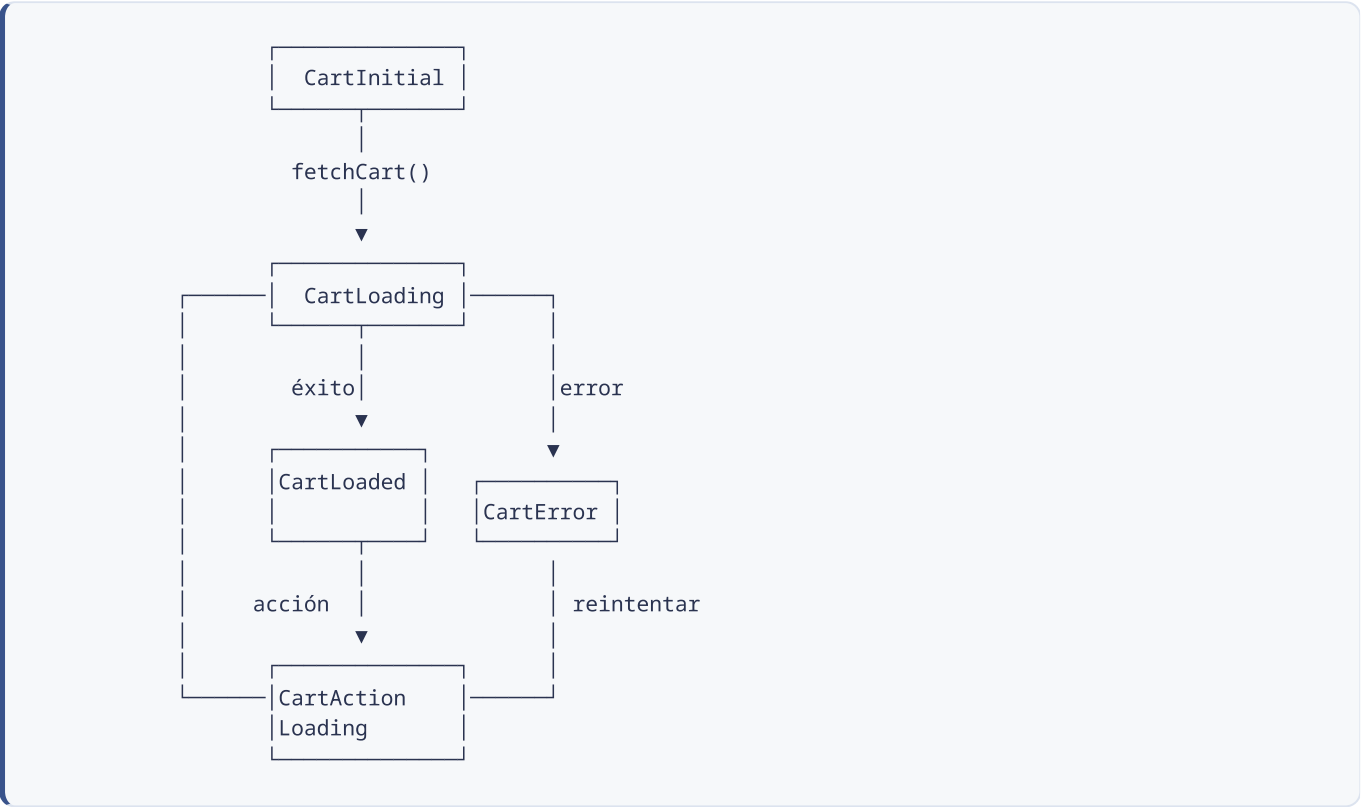


En el contrato, la lectura en vivo es un Stream además de las operaciones CRUD:

```
abstract class TicketsDataSource {
    Future<List<TicketModel>> fetchTickets(String raffleId);
    Stream<TicketModel> watchTickets(String raffleId); // realtime
}
```

¿Cuándo usarlo? Diseñarlo en la sección [Supabase por feature](#). Regla práctica: si la pantalla se abre una vez y no cambia, no necesitas realtime.

Diagrama de Estados del Carrito



Siguiente paso

Ve a la [práctica de flujo de datos](#) y diagrama el flujo completo del Carrito de Compras.

Tiempo estimado de lectura: 25 minutos

Tiempo estimado de práctica: 30 minutos

Práctica: Flujo de Datos del Carrito de Compras

Diagrama el flujo completo del Carrito de Compras a través de todas las capas de Clean Architecture.

Instrucciones

1. Toma tus contratos de la práctica anterior
2. En papel, dibuja el flujo de datos para cada operación del carrito
3. Incluye: tipos de datos, transformaciones entre capas, manejo de errores
4. Al final, compara con las soluciones sugeridas

Enunciado

Para cada una de estas operaciones del Carrito, dibuja el flujo completo desde la UI hasta la API y de vuelta:

1. **Agregar producto** (`addProduct`)
2. **Quitar producto** (`removeProduct`)
3. **Aplicar cupón** (`applyCoupon`)
4. **Ver resumen** (`getCartSummary`)

Para cada flujo, incluye:

- El tipo de dato que viaja en cada paso
- Dónde se transforman los datos (Model → Entity, etc.)
- Dónde se manejan los errores
- Los estados del Cubit que se emiten

Paso a Paso



Paso 1: Flujo Agregar Producto

Dibuja la secuencia desde que el usuario presiona "Agregar" hasta que ve el resultado.

Incluye:

1. Widget → Cubit (evento y estado inicial)
2. Cubit → UseCase (llamada y parámetros)
3. UseCase → Repository (validaciones y delegación)
4. Repository → DataSources (remoto y local)
5. DataSource → API (serialización)
6. API → DataSource → Repository → UseCase → Cubit (respuesta y transformaciones)
7. Cubit → Widget (nuevo estado)

Marca en el diagrama:

-  Dónde se validan las reglas RN002 (stock) y RN003 (cantidad)
-  Dónde se manejan los errores
-  Dónde se transforman los tipos

Paso 2: Flujo Quitar Producto

Similar al paso 1 pero para `removeProduct`. Nota las diferencias:

- ¿Qué validaciones aplican? (o no aplican)
- ¿El flujo es más simple? ¿Por qué?
- ¿Qué pasa si el producto no existe en el carrito?

Paso 3: Flujo Aplicar Cupón

Dibuja el flujo de `applyCoupon`. Incluye:

1. ¿Dónde se valida que el cupón no esté expirado? (RN005)
2. ¿Dónde se valida el descuento máximo? (RN006)
3. ¿El cupón se valida localmente o contra la API?
4. ¿Qué pasa si el cupón es inválido?

Paso 4: Flujo Ver Resumen

Para `getCartSummary`:

- ¿Este flujo usa remoto, local o ambos?
 - ¿Cómo sería una estrategia offline-first para este caso?
 - ¿Qué estados del Cubit aplican?
-

Solución Sugerida

✓ Flujo Agregar Producto (detallado)

CAPA	TIPO	QUÉ PASA
USUARIO	Tap en "Agregar"	Presiona botón en ProductCard
WIDGET	Product, int qty	onTap: cubit.addProduct(product, 1)
CUBIT	CartActionLoading (estado con cart actual)	emit(CartActionLoading(cart: current))
	AddProductToCart.call(userId: u, product: p, quantity: 1)	Llama al UseCase con parámetros
USECASE	VALIDA REGLAS: Si alguna falla: → Left(InvalidQuantity ProductOutOfStock CartLimitExceeded) Si todo ok: → repository.addProduct(...)	RN002: product.stock > 0 RN003: quantity > 0 RN001: cart.items.length < 50
REPOSITORY (INT)	CartRepository.addProduct(userId, product, qty)	Interface (no implementa nada)
REPOSITORY (IMPL)	CartEntity	1. remoteDS.addItem(userId, itemData) 2. localDS.cacheCart(userId, model) 3. model.toEntity() → Cart 4. Right(cart)
DATASOURCE (REMOTE)	CartModel	remoteDS.addItem() → POST a Supabase CartModel.fromJson(response)
DATASOURCE (LOCAL)	void	localDS.cacheCart(userId, model) Guarda en SQLite/Hive
API SUPABASE	JSON	POST /carts/{userId}/items Retorna CartModel como JSON
————— VUELTA —————		
DATASOURCE	CartModel	Modelo deserializado
REPOSITORY	Cart (Entity)	cartModel.toEntity()
USECASE	Right(Cart)	Propaga el Cart al Cubit
CUBIT	CartLoaded(cart: updated)	fold(): éxito → CartLoaded error → CartError(mensaje)
WIDGET	Rebuild con CartLoaded.cart	Muestra item en lista, actualiza totales

USUARIO	Ve el producto en su carrito	UI actualizada
---------	---------------------------------	----------------

✓ Flujo Quitar Producto (simplificado)

USUARIO	→ Tap "Eliminar" en item del carrito
WIDGET	→ <code>cubit.removeProduct(productId)</code>
CUBIT	→ <code>emit(CartActionLoading(cart: current, loadingItemId: id))</code>
USECASE	→ <code>RemoveProductFromCart.call(userId, productId)</code> → Sin reglas extra (cualquier producto se puede quitar)
REPOSITORY	→ <code>repository.removeProduct(userId, productId)</code>
REPO IMPL	→ <code>remoteDS.removeItem(userId, productId)</code> → <code>localDS.cacheCart(userId, updatedModel)</code> → <code>model.toEntity()</code>
CUBIT	→ <code>fold()</code> → éxito: <code>CartLoaded(cart: cartWithoutItem)</code> → error: <code>CartError("No se pudo eliminar el producto")</code>
WIDGET	→ Rebuild sin el item

Diferencia clave: Al quitar un producto no hay validaciones de negocio. El flujo es más directo.

✓ Flujo Aplicar Cupón

USUARIO	→ Escribe código de cupón → Tap "Aplicar"
WIDGET	→ <code>cubit.applyCoupon(code)</code>
CUBIT	→ <code>emit(CartActionLoading(cart: current))</code>
USECASE	→ <code>ApplyCoupon.call(userId, code)</code> → Valida: <code>couponCode</code> no vacío → <code>repository.applyCoupon(userId, code)</code>
REPO IMPL	→ <code>remoteDS.applyCoupon(userId, code)</code> ← La API valida el cupón → La API retorna error si expiró (RN005) → La API retorna error si descuento > 50% (RN006) → <code>localDS.cacheCart(userId, model)</code> → <code>model.toEntity()</code>
CUBIT	→ <code>fold()</code> → éxito: <code>CartLoaded(cart: cartWithDiscount)</code> → error: <code>CartError(mensaje: "Cupón expirado" "Cupón inválido")</code>
WIDGET	→ Muestra descuento aplicado y total actualizado

Nota importante: Las reglas RN005 y RN006 se validan del lado del servidor (API). El UseCase podría también tener una validación local del formato del código como precaución.

✓ Flujo Ver Resumen (Offline-First)

```
USUARIO    → Navega a la pantalla del carrito
WIDGET     → CartPage.initState() → cubit.fetchCart()
CUBIT      → emit(CartLoading)
USECASE    → GetCartSummary.call(userId)
REPO_IMPL  → ESTRATEGIA OFFLINE-FIRST:
              1. localDS.getCachedCart(userId) ← Responde inmediato
              2. En paralelo: remoteDS.fetchCart(userId)
              3. Si remote ok → localDS.cacheCart() → retorna remoto
              4. Si remote fail → retorna lo que tenía del caché
CUBIT      → fold()
              → éxito: CartLoaded(cart)
              → error: CartError("No se pudo cargar el carrito")
WIDGET     → Muestra lista de items, totales, input de cupón
```

Verificación: Lista de chequeo

- ☐ ¿Cada flujo muestra el tipo de dato en cada paso?
- ☐ ¿Las transformaciones Model ↔ Entity están marcadas?
- ☐ ¿El manejo de errores está cubierto en cada capa?
- ☐ ¿Los estados del Cubit se emiten en los momentos correctos?
- ☐ ¿Las validaciones de negocio están en el UseCase (no en otra capa)?
- ☐ ¿El DataSource remoto usa Models, no Entities?
- ☐ ¿El flujo offline-first está considerado donde aplica?
- ☐ ¿Se evaluó si la pantalla necesita realtime (Streams)? (ver [05e-diseno-supabase.md](#))
- ☐ ¿Los diagramas son claros para alguien que no conoce el código?

Siguiendo paso

Ahora que dominas el flujo de datos, enfrenta el [caso completo](#): diseña desde cero un Sistema de Reservas aplicando todo lo aprendido.

Tiempo: 30 minutos

Material: Papel y lápiz + contratos de la práctica anterior

Diseño Supabase por Feature

Después de mapear y definir contratos, diseña la sección tecnológica específica de Supabase: tablas, operaciones, RLS, atomicidad y realtime. Aquí viven las reglas técnicas y de seguridad.

Alcance: esto no es "diseñar un backend". Define **lo que la app necesita del servidor**. Con Supabase (BaaS) la app lo implementa (tablas, RLS, RPC). Con una REST API (Python/otro) la app solo especifica el **contrato** que consume su `RemoteDataSource` con Dio — el backend la diseña e implementa.

¿Por qué una sección Supabase?

El diseño no termina en los contratos. Como tu stack es estable (Flutter + Supabase), cada feature puede documentar exactamente qué necesita de la base de datos **antes** de escribir la primera consulta.

Esto evita que aparezca esto en un Cubit o un widget:

```
// × Diseño de Supabase improvisado dentro de la UI
supabase.from('tickets').select().eq('raffle_id', id);
```

Y fuerza a decidir antes, en papel:

- ¿Qué tablas toca la feature?
- ¿Qué consultas y escrituras hace?
- ¿Quién puede leer/escribir cada fila (RLS)?
- ¿La operación necesita ser atómica (transacción o RPC)?
- ¿Necesita datos en tiempo real?

¿Y si tu backend es una REST API (Python/otro)?

El diseño no cambia: la feature sigue necesitando lo mismo del servidor. Solo cambia **quién implementa**.

Con Supabase (BaaS)	Con una REST API
La app diseña e implementa tablas, RLS y RPC	La app especifica el contrato (verbo + path, DTO, códigos de error, garantías) y el backend lo implementa
RS → policies de RLS	RS → se exige al backend autorización en el servidor (roles/middleware)
Atomicidad → función RPC	Atomicidad → se exige al backend una operación atómica e idempotente en 1 request

Con Supabase (BaaS)	Con una REST API
Realtime → canales de Supabase	Realtime → se exige WebSocket/SSE con el mismo contrato <code>Stream<Entity></code>

El `RemoteDataSource` es la costura: con Supabase usa `SupabaseClient`, con una REST API usa Dio — el repositorio no sabe ni le importa cuál (ver [05b-data-layer](#)).

Reglas técnicas y de seguridad

En `01-descomposicion-feature.md` clasificas las reglas en tres tipos. Esta sección recoge las que NO pertenecen al dominio:

```
Reglas de negocio (RN)    → van al UseCase          → dominio puro
Reglas técnicas (RT)      → van al contrato con el backend → esta sección
Reglas de seguridad (RS) → van a la autorización del servidor (Supabase: RLS · REST:
middleware/roles) → esta sección
```

Reglas técnicas:

- La consulta debe estar paginada.
- La búsqueda debe tener debounce.
- La operación debe ejecutarse en una transacción.
- Se debe manejar pérdida de conexión.

Reglas de seguridad:

- RLS debe impedir acceder a rifas ajenas.
- El organizador solo puede modificar sus tickets.
- Solo se puede leer el propio perfil.

No confundas: las reglas técnicas y de seguridad no van en el dominio. Se documentan aquí, durante el diseño, para que el `DataSource` y la migración SQL las implementen (o el contrato que se entrega al backend REST).

La sección Supabase por feature

```
## Supabase

### Tablas
- tickets
- raffles
- buyers

### Operaciones
- select      (listar compradores de una rifa)
- update      (aprobar/liberar tickets)
- RPC         (aprobar seleccionados + liberar el resto)
- realtime    (opcional: refrescar lista)

### RLS
- Política de lectura:  el organizador lee solo sus tickets
- Política de escritura: el organizador actualiza solo sus tickets

### Atomicidad
- ¿Necesita transacción?      Sí (aprobar + liberar debe ser 1 operación)
- ¿Necesita función RPC?      Sí, ver `approve_and_release_tickets`
- ¿Cuándo?                    Cuando hay 2+ escrituras que deben ser atómicas

### Realtime
- ¿Lo necesita?      No (en esta versión)
- Canal:
- Evento:
```

La decisión clave: transacción vs RPC

Cuando una operación necesita escribir en **2+ lugares** (o validar y escribir al mismo tiempo), el DataSource no puede hacer dos `update` separados sin riesgo de quedarse a medias.

Necesidad	Solución	Cuándo
Transacción	<code>rpc()</code> con una función que hace BEGIN/COMMIT	La validación depende de leer el estado actual (ej: no dobles reservas)
RPC simple	Función PostgreSQL	La operación es un procedimiento que combina varias escrituras
Dos updates	Sin función	Las escrituras son independientes y toleran fallar por separado

Test de atomicidad técnico: si una regla dice *"los seleccionados se aprueban Y los demás se liberan"*, y falla a mitad, ¿el sistema queda en un estado inválido? Si sí → necesita RPC/transacción.

Ejemplo completo: Buyers

Regla del negocio: "Cuando el organizador confirma, los tickets seleccionados se aprueban y los demás se liberan."

Esto son 2 escrituras dependientes → no puede ser un `update` simple.

```
## Supabase (feature Buyers)

### Tablas
- raffles
- tickets

### Operaciones
- [R] select      listar tickets de una rifa
- [U] update      liberar tickets (estado = liberado)
- [RPC] approve_and_release_tickets(
    p_raffle_id, p_winner_ids[]
) → aprueba los seleccionados Y libera el resto en una transacción

### RLS
- Lectura:  tickets.where(raffle.organizer_id == auth.uid())
- Escritura: solo via RPC, que verifica organizer_id antes de escribir

### Atomicidad
- Transacción: Sí
- RPC: approve_and_release_tickets
- Alternativa descartada: dos updates separados (estado inválido si falla el segundo)

### Realtime
- ¿Lo necesita?  No
- Canal:
- Evento:
```

Realtime: cuándo y cómo

No toda feature necesita tiempo real. Antes de añadir un canal, pregúntate:

Pregunta	Si responde "sí"
¿Otro usuario puede cambiar los datos mientras yo los veo?	Necesita realtime
¿La pantalla se abre una vez y no cambia?	No lo necesita
¿Puedo refrescar con pull-to-refresh?	Probablemente no

```
### Realtime
- ¿Lo necesita?  Sí
- Canal:         tickets_raffle_{raffleId}
- Evento:        UPDATE (aprobación/liberación de tickets)
- Filtro RLS:    el canal debe respetar las mismas políticas
```

Contrato del DataSource con realtime: el Repository expone un `Stream<Entity>` además de las operaciones CRUD:

```
abstract class TicketsDataSource {
    Future<List<TicketModel>> fetchTickets(String raffleId);
    Stream<TicketModel> watchTickets(String raffleId); // realtime
    Future<void> approveAndRelease(String raffleId, List<String> winnerIds);
}
```

Buenas prácticas

- **RLS siempre.** Si una tabla es accesible sin RLS, la seguridad depende del cliente — y el cliente no se debe confiar.
- **Los RPC verifican permisos dentro de la función,** no asumen que el llamador es legítimo.
- **Los índices se diseñan con la consulta:** filtros por `raffle_id`, búsquedas por nombre con `pg_trgm`.
- **Una operación atómica = un contrato.** Si el contrato del repositorio dice `approveAndRelease(...)`, el DataSource lo implementa con un RPC — no con dos llamadas.
- **El dominio sigue sin saber de Supabase.** Todo lo de este archivo se implementa en DATA y se refleja en las migraciones.

Errores comunes

Error	Síntoma	Solución
Consultas de Supabase en la UI	<code>supabase.from(...)</code> en un widget	Diseña el DataSource en esta sección
RLS ausente	Cualquiera lee todo	Toda tabla accesible desde el cliente tiene política
Operación atómica con 2 updates	Estado inválido si falla el segundo	RPC en transacción
Realtime innecesario	Canales abiertos que nadie usa	Responde las preguntas de la tabla
Reglas técnicas dentro del UseCase	Paginación/debounce en dominio	Documenta como RT, implementa en DATA

Siguiente paso

Ya definiste qué necesita la feature de la base de datos. Ahora escribe los [criterios de aceptación y la matriz de trazabilidad](#) para que cada operación quede verificable y ninguna regla quede sin implementar.

Para profundizar en RLS, RPC y migraciones, consulta el módulo [03-SUPABASE](#).

Tiempo estimado: 10-20 minutos por feature

Criterios de Aceptación y Matriz de Trazabilidad

Convierte el diseño en pruebas verificables. Los criterios de aceptación dicen *qué significa que la feature funcione*; la matriz de trazabilidad garantiza que *nada se quedó sin implementar ni sin probar*.

Parte 1: Criterios de Aceptación

FADER explica bien el problema, pero una regla escrita como texto puede interpretarse distinto. Los criterios de aceptación lo convierten en **escenarios verificables**.

Formato BDD

```
CA01:  
Dado que [contexto inicial],  
cuando [acción del actor],  
entonces [resultado esperado].
```

Ejemplo: Buyers

```
CA01:  
Dado que soy organizador,  
cuando abro Buyers,  
entonces veo los últimos 30 compradores.
```

```
CA02:  
Dado que un ticket está aprobado,  
cuando intento liberarlo,  
entonces la operación es rechazada.
```

```
CA03:  
Dado que selecciono algunos tickets,  
cuando confirmo la aprobación,  
entonces los seleccionados se aprueban  
y los demás se liberan.
```

Cómo escribirlos

Regla	Ejemplo
Un criterio = un escenario	No mezcles dos "entonces" independientes
Sé concreto y medible	"últimos 30" en vez de "los recientes"

Regla	Ejemplo
Cubre el borde	Estado vacío, límite, formato inválido
Cubre el error	Qué pasa cuando la operación se rechaza
Referencia la regla que valida	CA02 ↔ RN005 "no re-aprobar"

¿Cuántos criterios por feature?

Tamaño de feature	Criterios típicos
Hotfix (1-2 ops)	1-3
Pequeña (3-6 ops)	3-6
Mediana (7-15 ops)	8-15
Grande (16-30 ops)	15-25

Regla práctica: cada operación atómica de FADER debería tener al menos un criterio de happy path y uno de error.

De criterio a test

CA03 (aprobar y liberar) →
Test unitario del UseCase ApproveTickets
Test de integración del RPC approve_and_release_tickets
Widget test: botón de confirmar solo se habilita con selección

Parte 2: Matriz de Trazabilidad

La matriz conecta el diseño de punta a punta:

Operación → UseCase → Regla → Contrato → Test

Ejemplo: Buyers

Operación	UseCase	Regla	Contrato	Test
Aprobar tickets	ApproveTickets	RN004, RN005, RN008	approveTickets()	Unit + integration
Liberar tickets	ReleaseTickets	RN004, RN006	releaseTickets()	Unit + integration
Listar compradores	GetBuyers	RS001, RT001	getBuyers()	Unit + widget

Qué verifica la matriz

- 1. **Ninguna operación quedó sin implementación.** Toda fila de FADER tiene su UseCase.
- 2. **Ninguna regla quedó sin prueba.** Toda regla aparece en alguna fila con test.
- 3. **Ningún contrato quedó huérfano.** Cada contrato se usa desde al menos un UseCase.
- 4. **Los criterios de aceptación tienen respaldo.** CA03 se cumple por la fila "Aprobar tickets".

Lectura cruzada

```
[ ] Cada operación de FADER está en la matriz
[ ] Cada regla RN/RT/RS está en la matriz
[ ] Cada contrato definido en 03-contratos-primero.md está en la matriz
[ ] Cada criterio de aceptación (CA) tiene una fila que lo respalda
[ ] Cada fila tiene un tipo de test asignado
```

Plantillas

Plantilla de criterios de aceptación

```
## Criterios de aceptación

CA01:
Dado que [contexto],
cuando [acción],
entonces [resultado].

CA02:
Dado que [contexto],
cuando [acción],
entonces [resultado].
```

Plantilla de matriz de trazabilidad

```
| Operación | UseCase | Regla | Contrato | Test |
|-----|-----|-----|-----|-----|
| [op de FADER] | [UseCase] | [RN/RT/RS] | [método] | Unit/integration |
```

Errores comunes

Error	Síntoma	Solución
Criterios vagos	"debe funcionar bien"	Escribe resultado medible

Error	Síntoma	Solución
Un CA con 3 "entonces"	Escenario imposible de testear	Divide en CAs separados
Matriz incompleta	Reglas sin UseCase o UseCase sin test	Corre la lectura cruzada
Criterios sin regla	El test no sabe qué validar	Vincula CA ↔ RN
Matriz solo de un lado	Operaciones implementadas, reglas sin probar	Llena las 5 columnas

Siguiendo paso

Con criterios y trazabilidad, tu diseño ya tiene la forma de [la plantilla por feature](#) lista para reutilizar. Cuando llegues a los tests, el módulo [05-TESTING](#) te muestra cómo escribirlos a partir de estos criterios.

Tiempo estimado: 15-20 minutos por feature

Caso Completo: Sistema de Reservas

Aplica todo lo aprendido (FADER + Mapeo + Contratos + Flujo + Supabase + Criterios) para diseñar un Sistema de Reservas desde cero, en papel y lápiz.

Objetivo

Sin abrir el editor, sin escribir una sola línea de código, vas a diseñar la arquitectura completa de un Sistema de Reservas usando la metodología FADER y Clean Architecture.

Al terminar, tendrás:

- ☒ Feature descompuesta (Alcance, Formular, Actorizar, Descomponer, Entidades, Reglas)
- ☒ Mapeo completo a capas de Clean Architecture
- ☒ Contratos (interfaces) de cada capa
- ☒ Diagramas de flujo de datos
- ☒ Diseño de Supabase (tablas, RPC, RLS, realtime)
- ☒ Criterios de aceptación y matriz de trazabilidad
- ☒ ADRs documentando decisiones clave

Enunciado del Sistema de Reservas

Somos un equipo que desarrolla una app para **clínicas veterinarias**. El equipo de producto nos pide:

Los dueños de mascotas deben poder agendar citas veterinarias seleccionando un doctor, una fecha disponible y el tipo de consulta. El sistema debe validar disponibilidad, evitar dobles reservas, y enviar un recordatorio 24 horas antes. Los doctores deben poder ver su agenda del día y marcar citas como completadas o canceladas. Los administradores de la clínica deben poder configurar horarios laborales y bloquear días no laborables.

Tu Misión

Completa cada sección en orden, en papel y lápiz. La siguiente guía te lleva paso a paso.

Sección 0: Alcance

Antes de descomponer, define los límites del sistema (teoría en [00-alcance-feature.md](#)):

- 1. **Incluye:** ¿qué cubre el sistema de reservas?
- 2. **No incluye:** ¿qué NO cubre? (ej: facturación, recetas, pagos)
- 3. **Dependencias:** ¿qué necesita que exista antes?
- 4. **Suposiciones:** ¿qué das por hecho?
- 5. **Preguntas abiertas:** ¿qué no sabes todavía?

Sección 1: FADER

Paso 1: Formular

Define al menos 3 enunciados "Como [actor], quiero [acción] para [valor]".

Pregúntate:

- ¿Qué necesidad real resuelve agendar una cita?
- ¿Qué pasa si el dueño no puede agendar desde la app? (llama por teléfono, va sin cita)
- ¿Cuál es el valor de los recordatorios?

Paso 2: Actorizar

Identifica todos los actores:

Actor	Tipo	¿Qué puede hacer?
Dueño de mascota	Primario	
Doctor	Secundario	
Administrador	Secundario	
Sistema de notificaciones	Interno	
Calendario (Google/Apple)	Externo	
?	?	

Completa la tabla con los permisos de cada actor.

Paso 3: Descomponer

Enumera todas las operaciones atómicas. Clasifícalas como CRUD, Validación, Cálculo, o Transición.

Ejemplo para empezar:

- Crear cita

- [R] Ver agenda del doctor
- [R] Ver citas del dueño
- [U] Cancelar cita
- [U] Marcar como completada
- ...
- ¿Qué más?

Identifica dependencias:

- ¿Qué debe pasar antes de crear una cita?
- ¿Qué debe pasar después de crearla?

Paso 4: Entidades

Define las entidades de negocio. Para cada una, lista sus atributos esenciales.

Posibles entidades:

- Cita (Appointment)
- Doctor (Veterinarian)
- Mascota (Pet)
- Dueño (Owner)
- HorarioLaboral (WorkSchedule)
- TipoConsulta (ConsultationType)
- Recordatorio (Reminder)

Pregúntate:

- ¿Doctor y Dueño son la misma entidad que Usuario?
- ¿Mascota pertenece al dueño o a la clínica?
- ¿Un HorarioLaboral pertenece a un doctor o a la clínica?

Paso 5: Reglas

Enuncia al menos 8 reglas de negocio con formato RN001, RN002... y añade reglas técnicas (RT) y de seguridad (RS) cuando aplique (teoría en [01-descomposicion-feature.md](#)).

Áreas a cubrir:

- Disponibilidad (no dobles reservas)
 - Antelación mínima para agendar
 - Antelación mínima para cancelar
 - Duración de consultas
 - Límite de citas por día
 - Recordatorios
 - Cancelaciones vs No-show
 - Reagendamiento
-

Sección 2: Mapeo a Capas

Paso 1: Estructura DOMAIN

Dibuja el árbol de `domain/`:

```
domain/
├── entities/
│   ├── cita.dart
│   ├── doctor.dart
│   ├── mascota.dart
│   ├── dueño.dart
│   ├── horario_laboral.dart
│   └── tipo_consulta.dart
├── usecases/
│   ├── agendar_cita.dart           - ¿Qué reglas valida?
│   ├── cancelar_cita.dart         - ¿Qué reglas valida?
│   ├── obtener_agenda_doctor.dart
│   ├── obtener_citas_dueno.dart
│   ├── marcar_completada.dart
│   └── configurar_horario.dart
├── repositories/
│   ├── cita_repository.dart        - interface
│   └── doctor_repository.dart      - interface
└── core/
    └── failures.dart              - Errores del dominio
```

Pregúntate:

- ¿Todos los UseCases son necesarios o algunos se pueden combinar?
- ¿Necesitamos un repository para `Mascota` o va dentro de `CitaRepository`?
- ¿Dónde va la lógica de recordatorios?

Paso 2: Estructura DATA

Dibuja el árbol de `data/`:

```
data/
├── datasources/
│   ├── cita_remote_data_source.dart
│   └── cita_local_data_source.dart
├── models/
│   ├── cita_model.dart
│   ├── doctor_model.dart
│   ├── mascota_model.dart
│   └── horario_model.dart
└── repositories/
    ├── cita_repository_impl.dart
    └── doctor_repository_impl.dart
```

Pregúntate:

- ¿El DataSource local es solo caché o la fuente primaria?
- ¿Necesitamos un modelo separado para cada entidad?

- ¿Cómo manejamos los horarios recurrentes (lunes a viernes 9-18)?

Paso 3: Estructura PRESENTATION

Dibuja el árbol de `presentation/`:

```
presentation/  
├── cubit/  
│   ├── agenda_cubit.dart  
│   ├── agenda_state.dart  
│   ├── agendar_cita_cubit.dart  
│   └── agendar_cita_state.dart  
├── pages/  
│   ├── agenda_page.dart  
│   ├── agendar_cita_page.dart  
│   └── detalle_cita_page.dart  
└── widgets/  
    ├── slot_horario_widget.dart  
    ├── tarjeta_cita_widget.dart  
    └── calendario_widget.dart
```

Pregúntate:

- ¿Un solo Cubit o varios? (agenda del doctor vs agendar nueva cita)
- ¿Qué estados puede tener cada Cubit?
- ¿El calendario widget es un componente reutilizable?

Sección 3: Contratos

Paso 1: Contrato CitaRepository

Diseña la interfaz `CitaRepository`:

```
abstract class CitaRepository {  
  // ¿Qué métodos necesita el dominio?  
  // ¿Qué parámetros recibe cada uno?  
  // ¿Qué retorna?  
  // ¿Qué failures puede producir?  
}
```

Operaciones a considerar:

- Crear cita
- Cancelar cita
- Reagendar cita
- Obtener citas por rango de fechas
- Obtener citas de un doctor
- Obtener citas de un dueño
- Marcar como completada

- Marcar como no-show

Paso 2: Contrato DoctorRepository

```
abstract class DoctorRepository {  
    // ¿Cómo se obtienen los doctores disponibles?  
    // ¿Cómo se obtienen los horarios?  
}
```

Paso 3: Contrato de DataSource Remoto

```
abstract class CitaRemoteDataSource {  
    // ¿Qué operaciones de bajo nivel?  
    // ¿Usa modelos o mapas?  
}
```

Paso 4: Estados de UI

Diseña los estados para `AgendaCubit` y `AgendarCitaCubit`:

```
AgendaState:  
- ¿Qué estados? (loading, loaded, error, etc.)  
- ¿Qué datos lleva cada estado?  
  
AgendarCitaState:  
- ¿Qué estados? (initial, loading, success, error, etc.)  
- ¿Cómo manejas la selección de fecha y doctor?
```

Paso 5: ADR

Escribe al menos un ADR documentando una decisión clave, como:

- ¿Por qué separamos CitaRepository de DoctorRepository? (o por qué no)
- ¿Cómo manejamos la validación de disponibilidad (local vs API)?
- ¿Dónde se genera el recordatorio (dominio vs sistema externo)?

Sección 4: Flujo de Datos

Paso 1: Flujo Agendar Cita

Dibuja la secuencia completa desde que el dueño selecciona un horario hasta que la cita queda registrada.

Incluye:

- Widget → Cubit → UseCase → Repository → DataSources → API
- Validaciones en cada paso

- Transformación de tipos
- Manejo de errores
- Actualización de estados del Cubit

Pregúntate:

- ¿Cuándo se valida que el horario sigue disponible?
- ¿Qué pasa si otro dueño reservó el mismo slot mientras tanto?
- ¿Cómo se actualiza la UI del doctor en tiempo real?



Paso 2: Flujo Cancelar Cita

Dibuja el flujo de cancelación.

Pregúntate:

- ¿Hay una regla de "solo se puede cancelar con 2 horas de antelación"?
- ¿Dónde se valida esa regla?
- ¿Qué pasa con el recordatorio programado?
- ¿Se notifica al doctor?



Paso 3: Flujo Ver Agenda del Doctor

Dibuja el flujo de carga de la agenda.

Pregúntate:

- ¿Es offline-first o siempre online?
- ¿Cómo manejas el caché de horarios?
- ¿Qué pasa si la API falla?

Sección 5: Diseño Supabase

Ahora aterriza el diseño en Supabase (teoría en [05e-diseno-supabase.md](#)):

¿Tu backend es una REST API (Python/otro)? Aquí no implementas el servidor: solo especificas el **contrato** (endpoints, DTOs, códigos de error, garantías de atomicidad y autorización) que consume tu `RemoteDataSource` con Dio; el backend la diseña e implementa.

1. **Tablas:** define las tablas, columnas y tipos (snake_case).
2. **Operaciones:** para cada UseCase, ¿qué operación lo implementa (select/insert/update/rpc)?
3. **RLS:** define políticas que hagan cumplir RS001.
4. **Atomicidad:** identifica operaciones que deben ser atómicas (ej: agendar un slot disponible → RPC en transacción).
5. **Realtime:** ¿qué tablas se sincronizan en vivo?

Pregúntate:

- ¿La validación de slot disponible (RN001) vive en un RPC atómico o en el cliente?
 - ¿Cómo evitas que el dueño A vea citas del dueño B (RS001)?
 - ¿El recordatorio (RN006) lo dispara Postgres (cron) o un servicio externo?
-

Sección 6: Criterios de Aceptación y Trazabilidad

Cierra el diseño definiendo criterios (teoría en [05f-criterios-aceptacion-trazabilidad.md](#)):

1. **Criterios de aceptación:** escribe al menos 5 en formato BDD (Dado/Cuando/Entonces).
2. **Matriz de trazabilidad:** para los UseCases críticos, cruza UseCase → Regla → Contrato → Fuente de verdad → Test.

Pregúntate:

- ¿El criterio "cancelar con 2h de antelación" es verificable?
 - ¿Toda regla RN tiene al menos un UseCase y un test que la cubra?
 - ¿Los criterios distinguen el frontend (validación optimista) del servidor (fuente de verdad)?
-

Solución Sugerida

 Resuelve cada sección en papel primero. La solución sugerida es para comparar después.

FEATURE: Sistema de Reservas (Clínica Veterinaria)

[F]ormular:

- F1: Como dueño de mascota, quiero agendar citas veterinarias online para asegurar atención en el horario que me conviene.
- F2: Como doctor, quiero ver mi agenda del día para organizar mi trabajo.
- F3: Como administrador, quiero configurar horarios laborales para gestionar la disponibilidad de la clínica.

[A]ctorizar:

1. Dueño de mascota (primario)
 - Agendar, cancelar, reagendar citas
 - Ver historial de citas de su mascota
2. Doctor (secundario)
 - Ver agenda del día
 - Marcar citas como completada/cancelada/no-show
3. Administrador (secundario)
 - Configurar horarios laborales
 - Bloquear días no laborables
 - Gestionar doctores
4. Sistema de Notificaciones (interno)
 - Recordatorio 24h antes
 - Notificar cancelación al doctor
5. Google/Apple Calendar (externo, opcional)
 - Exportar cita al calendario del dueño

[D]escomponer:

Dueño:

- [C] Agendar cita
- [R] Ver citas próximas
- [R] Ver historial de citas
- [U] Cancelar cita
- [U] Reagendar cita

Doctor:

- [R] Ver agenda del día
- [U] Marcar cita como completada
- [U] Marcar cita como cancelada
- [U] Marcar cita como no-show

Admin:

- [C] Configurar horario laboral
- [C] Bloquear día no laborable
- [CUD] Gestionar doctores

Sistema:

- [Validación] Validar disponibilidad (no doble reserva)
- [Validación] Validar antelación mínima (4h)
- [Cálculo] Generar slots disponibles
- [Transición] Enviar recordatorio (24h antes)
- [Transición] Notificar cancelación

[E]ntidades:

Cita: id, mascota, doctor, fechaHora, duracion, tipoConsulta, estado, motivo, notas
Doctor: id, nombre, especialidad, email, telefono

Mascota: id, nombre, especie, raza, edad, dueño
Dueño: id, nombre, email, telefono, mascotas
HorarioLaboral: id, doctor, diaSemana, horaInicio,
horaFin, activo
BloqueoDia: id, fecha, motivo
TipoConsulta: id, nombre, duracionMinutos, precio
Recordatorio: id, cita, tipo, enviadoEn, estado

[R]eglas:

RN001: No se puede agendar una cita en un slot ocupado
RN002: Antelación mínima de 4 horas para agendar
RN003: Máximo 8 citas por doctor por día
RN004: Cancelación solo con 2h de antelación
RN005: No-show se marca si pasan 15min de la hora
RN006: Recordatorio automático 24h antes
RN007: Duración de consulta según tipoConsulta
RN008: Un dueño no puede tener 2 citas el mismo día
para la misma mascota (máximo 1 consulta/día)
RT001: Slots en huso horario de la clínica
RS001: Solo el dueño ve citas de su propia mascota

✓ Estructura de Capas

```
lib/
├── appointments/
│   ├── domain/
│   │   ├── entities/
│   │   │   ├── appointment.dart
│   │   │   ├── veterinarian.dart
│   │   │   ├── pet.dart
│   │   │   ├── owner.dart
│   │   │   ├── work_schedule.dart
│   │   │   ├── blocked_day.dart
│   │   │   └── consultation_type.dart
│   │   └── usecases/
│   │       ├── create_appointment.dart
│   │       ├── cancel_appointment.dart
│   │       ├── reschedule_appointment.dart
│   │       ├── get_doctor_agenda.dart
│   │       ├── get_owner_appointments.dart
│   │       ├── mark_completed.dart
│   │       ├── mark_no_show.dart
│   │       └── configure_schedule.dart
│   │   └── repositories/
│   │       ├── appointment_repository.dart
│   │       └── veterinarian_repository.dart
│   ├── core/
│   │   └── failures.dart
│   ├── data/
│   │   ├── datasources/
│   │   │   ├── appointment_remote_datasource.dart
│   │   │   └── appointment_local_datasource.dart
│   │   ├── models/
│   │   │   ├── appointment_model.dart
│   │   │   ├── veterinarian_model.dart
│   │   │   └── pet_model.dart
│   │   └── repositories/
│   │       ├── appointment_repository_impl.dart
│   │       └── veterinarian_repository_impl.dart
│   └── presentation/
│       ├── cubit/
│       │   ├── agenda_cubit.dart
│       │   ├── agenda_state.dart
│       │   ├── create_appointment_cubit.dart
│       │   └── create_appointment_state.dart
│       ├── pages/
│       │   ├── agenda_page.dart
│       │   ├── create_appointment_page.dart
│       │   └── appointment_detail_page.dart
│       └── widgets/
│           ├── time_slot_widget.dart
│           ├── appointment_card.dart
│           └── doctor_selector.dart
```

✓ Contrato Clave (CitaRepository)

```
abstract class AppointmentRepository {
    Future<Either<Failure, Appointment>> create(CreateAppointmentParams params);
    Future<Either<Failure, Appointment>> cancel(String appointmentId);
    Future<Either<Failure, Appointment>> reschedule(
        String appointmentId, DateTime newDateTime);
    Future<Either<Failure, Appointment>> markCompleted(String appointmentId);
    Future<Either<Failure, Appointment>> markNoShow(String appointmentId);
    Future<Either<Failure, List<Appointment>>> getDoctorAgenda(
        String doctorId, DateTime date);
    Future<Either<Failure, List<Appointment>>> getOwnerAppointments(
        String ownerId, {AppointmentStatus? status});
    Future<Either<Failure, List<DateTime>>> getAvailableSlots(
        String doctorId, DateTime date, String consultationTypeId);
}
```

✓ ADR Recomendado

ADR-003: Validación de disponibilidad del lado del servidor

Contexto

Cuando dos dueños intentan agendar el mismo slot simultáneamente, necesitamos evitar dobles reservas (RN001).

Decisión

La validación de disponibilidad se hará en el backend (Supabase) usando una transacción atómica. El frontend validará de forma optimista, pero la validación definitiva es del servidor.

Consecuencias

Positivas:

- Garantiza que no haya dobles reservas (race condition)
- El frontend es rápido (validación optimista)

Negativas:

- Mayor latencia en caso de conflicto
- El frontend debe manejar el caso "slot ya ocupado"

Alternativas consideradas

1. Bloqueo optimista con versión → Descartado: complejidad innecesaria
2. Validación solo frontend → Descartado: race condition
3. Cola de reservas → Descartado: sobreingeniería para este caso

✓ Diseño Supabase

```
appointments
├── id          uuid PK
├── pet_id      uuid FK → pets
├── doctor_id   uuid FK → doctors
├── date_time   timestampz
├── duration_min int
├── status      enum (scheduled, completed, cancelled, no_show)
├── created_at  timestampz

-- RS001: cada dueño solo ve citas de sus mascotas
create policy "owner sees own pets appointments"
on appointments for select
using (pet_id in (select id from pets where owner_id = auth.uid()));

-- RN001: slot atómico (evita doble reserva)
-- rpc agendar_cita(p_pet_id, p_doctor_id, p_date_time, p_duration_min)
-- → BEGIN; INSERT con constraint único (doctor_id, date_time); COMMIT;
-- → si hay conflicto, retorna error "slot_ocupado"
```

✓ Criterios de Aceptación (ejemplos)

Escenario: Agendar cita en slot libre
Dado un doctor con un slot libre el 2026-08-10 10:00
Y la reserva se hace con más de 4h de antelación (RN002)
Cuando el dueño agenda ese slot
Entonces la cita queda con estado "scheduled"
Y otro dueño ya no ve ese slot disponible

Escenario: Intentar agendar el mismo slot dos veces
Dado un slot recién reservado por el dueño A
Cuando el dueño B intenta agendar el mismo slot
Entonces el sistema rechaza con "slot_ocupado" (RN001)

✓ Matriz de Trazabilidad

UseCase	Regla(s)	Contrato	Fuente de verdad	Test
CreateAppointment	RN001, RN002	AppointmentRepository.create	RPC atómico	unit + widget
CancelAppointment	RN004	AppointmentRepository.cancel	API + RLS	unit
GetDoctorAgenda	RN007, RT001	AppointmentRepository.getDoctorAgenda	API	unit
MarkNoShow	RN005	AppointmentRepository.markNoShow	RPC + cron	unit + integration
Ver agenda (dueño)	RS001	AppointmentRepository.getOwnerAppointments	RLS	integration (RLS)

Entregable Final

Al completar este caso, deberías tener en papel:

1. ☒ Una hoja FADER completa (Alcance, Formular, Actorizar, Descomponer, Entidades, Reglas)
2. ☒ El árbol de carpetas de las 3 capas (domain, data, presentation)
3. ☒ Los contratos de AppointmentRepository y VeterinarianRepository
4. ☒ Los estados de los Cubits
5. ☒ Al menos 1 ADR documentando una decisión clave
6. ☒ Diagrama de flujo de "Agendar Cita" (de UI a BD y vuelta)
7. ☒ El diseño de Supabase (tablas, RPC atómico, RLS)
8. ☒ Criterios de aceptación y matriz de trazabilidad

Con esto, estás listo para abrir el editor y empezar a codificar.



Siguiente paso

Con el diseño completo en papel, abre tu editor y continúa con [01-CLEAN-ARCHITECTURE](#) para implementar todo lo diseñado.

Tiempo estimado: 2-3 horas

Material: Papel, lápiz. Muchas hojas.

Caso Práctico: Plataforma de Cursos Online (E-Learning)

Aplica FADER + Mapeo + Contratos + Flujo + Supabase + Criterios para diseñar una plataforma de cursos online desde cero.

Enunciado

Somos el equipo técnico de una startup de tecnología educativa. El equipo de producto nos pide:

Los estudiantes deben poder explorar un catálogo de cursos, inscribirse a un curso, ver el contenido (lecciones en video y texto), y marcar lecciones como completadas para trackear su progreso. Los instructores deben poder crear y editar cursos con múltiples lecciones, y ver reportes de avance de sus estudiantes. Los administradores deben poder gestionar usuarios (estudiantes e instructores) y aprobar cursos antes de publicarlos.

Instrucciones

1. Trabaja en papel y lápiz. No abras el editor de código.
2. Sigue cada sección en orden.
3. Al final, compara con la solución sugerida.

Sección 0: Alcance

Antes de descomponer, define los límites del sistema (teoría en [00-alcance-feature.md](#)):

1. **Incluye:** ¿qué cubre la plataforma?
2. **No incluye:** ¿qué NO cubre? (ej: foros, certificados, gamificación)
3. **Dependencias:** ¿qué necesita que exista antes?
4. **Suposiciones:** ¿qué das por hecho?
5. **Preguntas abiertas:** ¿qué no sabes todavía?

Sección 1: FADER

Paso 1: Formular

Escribe al menos 3 enunciados "Como [actor], quiero [acción] para [valor]".

Pregúntate:

- ¿Un curso es un solo video o una serie de lecciones?
- ¿El progreso se calcula sobre lecciones completadas o tiempo visto?
- ¿Qué diferencia hay entre "inscribirse" y "comprar" un curso?
- ¿Los cursos tienen fecha de inicio o son auto-gestionados?

Paso 2: Actorizar

Identifica todos los actores y sus permisos:

Actor	Tipo	¿Qué puede hacer?	¿Qué NO puede hacer?
Estudiante	Primario		
Instructor	Secundario		
Admin	Secundario		
Sistema de pagos	Externo		
?	?		

Pregúntate:

- ¿Un instructor también puede ser estudiante de otros cursos?
- ¿El admin puede editar contenido de cursos o solo aprobarlos?
- ¿Hay actores invitados (usuarios sin cuenta que ven contenido gratuito)?

Paso 3: Descomponer

Enumera todas las operaciones atómicas clasificadas en CRUD, Validación, Cálculo, Transición.

Considera:

- Catálogo de cursos (búsqueda, filtros, categorías)
- Inscripción (gratuita vs paga)
- Consumo de contenido (video, texto)
- Progreso (qué lecciones completó, porcentaje del curso)
- Creación de cursos (estructura: curso → módulos → lecciones)
- Aprobación de cursos (borrador → pendiente → publicado)
- Reportes de avance

Identifica dependencias:

- ¿Qué debe existir antes de inscribirse?
- ¿Qué debe pasar antes de marcar una lección como completada?
- ¿Qué condiciones habilitan la publicación de un curso?

Paso 4: Entidades

Define las entidades de negocio con sus atributos esenciales.

Posibles entidades:

- `Curso` (Course)
- `Lección` (Lesson)
- `Módulo` (Module)
- `Estudiante` (Student)
- `Instructor` (Instructor)
- `Inscripción` (Enrollment)
- `ProgresoLección` (LessonProgress)
- `Categoría` (Category)

Pregúntate:

- ¿`Curso` contiene `Lección` directamente o hay `Módulo` como nivel intermedio?
- ¿`Progreso` es una entidad separada o un atributo de `Inscripción`?
- ¿`Instructor` es un tipo de `Usuario` o una entidad distinta?
- ¿El precio del curso está en `Curso` o en una entidad separada de pricing?

Paso 5: Reglas

Enuncia al menos 8 reglas de negocio con formato RN001, RN002... y añade reglas técnicas (RT) y de seguridad (RS) cuando aplique (teoría en [01-descomposicion-feature.md](#)).

Áreas a cubrir:

- Inscripción: ¿puede inscribirse a un curso ya completado?
- Contenido: ¿qué pasa si un curso no tiene lecciones publicadas?
- Progreso: ¿cómo se calcula el porcentaje de avance?
- Publicación: ¿qué condiciones debe cumplir un curso para ser publicado?
- Acceso: ¿un estudiante ve todo el contenido al inscribirse o se libera por módulos?
- Eliminación: ¿se puede eliminar un curso con estudiantes inscritos?

Sección 2: Mapeo a Capas

Paso 1: Estructura DOMAIN

Dibuja el árbol de `domain/`:

```

domain/
├── entities/
│   ├── course.dart
│   ├── module.dart
│   ├── lesson.dart
│   ├── enrollment.dart
│   ├── lesson_progress.dart
│   ├── user.dart
│   └── category.dart
├── usecases/
│   ├── browse_courses.dart
│   ├── enroll_course.dart
│   ├── watch_lesson.dart
│   ├── complete_lesson.dart
│   ├── get_progress.dart
│   ├── create_course.dart
│   ├── publish_course.dart
│   └── approve_course.dart
├── repositories/
│   ├── course_repository.dart
│   ├── enrollment_repository.dart
│   └── user_repository.dart
└── core/
    └── failures.dart

```

Pregúntate:

- ¿ `complete_lesson` y `watch_lesson` son el mismo UseCase?
- ¿ `publish_course` es un UseCase o parte de `update_course`?
- ¿Necesitas un `enrollment_repository` separado o va dentro de `course_repository`?

Paso 2: Estructura DATA

Dibuja el árbol de `data/`:

```

data/
├── datasources/
│   ├── course_remote_data_source.dart
│   ├── course_local_data_source.dart
│   ├── enrollment_remote_data_source.dart
│   └── progress_data_source.dart
├── models/
│   ├── course_model.dart
│   ├── lesson_model.dart
│   ├── enrollment_model.dart
│   └── user_model.dart
└── repositories/
    ├── course_repository_impl.dart
    ├── enrollment_repository_impl.dart
    └── user_repository_impl.dart

```

Paso 3: Estructura PRESENTATION

Dibuja el árbol de `presentation/`:

```

presentation/
├── cubit/
│   ├── catalog_cubit.dart
│   ├── catalog_state.dart
│   ├── course_detail_cubit.dart
│   ├── player_cubit.dart
│   ├── student_progress_cubit.dart
│   └── instructor_courses_cubit.dart
├── pages/
│   ├── catalog_page.dart
│   ├── course_detail_page.dart
│   ├── lesson_player_page.dart
│   └── instructor_dashboard_page.dart
└── widgets/
    ├── course_card.dart
    ├── lesson_list.dart
    ├── progress_bar.dart
    └── video_player_widget.dart

```

Pregúntate:

- ¿Cuántos Cubits necesitas realmente? ¿Puedes combinar algunos?
- ¿El player de video es un widget reutilizable o una página completa?
- ¿El dashboard del instructor usa los mismos UseCases que la vista del estudiante?

Sección 3: Contratos

Paso 1: Contrato CourseRepository

```

abstract class CourseRepository {
  // ¿Qué métodos necesita el dominio de cursos?
  // ¿Cómo se buscan cursos? ¿Por categoría, por instructor, por texto libre?
  // ¿Cómo se obtiene el detalle de un curso con sus lecciones?
  // ¿Qué failures puede producir?
}

```

Paso 2: Contrato EnrollmentRepository

```

abstract class EnrollmentRepository {
  // Inscribir estudiante
  // Obtener cursos de un estudiante
  // Obtener estudiantes de un curso
  // Validar si ya está inscrito
}

```

Paso 3: Contrato de DataSource Remoto

```
abstract class CourseRemoteDataSource {  
    // Operaciones CRUD vs API  
    // Paginación del catálogo  
    // Búsqueda y filtros  
}
```

Paso 4: Estados de UI

Diseña los estados para al menos `CatalogCubit` y `PlayerCubit`.

Pregúntate:

- El catálogo tiene búsqueda y filtros — ¿cómo modelas eso en el estado?
- El player de video tiene estados: cargando, reproduciendo, pausado, error — ¿cómo los representas?
- ¿El progreso del estudiante se actualiza en tiempo real o al recargar?

Paso 5: ADR

Escribe al menos un ADR. Ejemplos de decisiones a documentar:

- ¿El progreso se calcula por lecciones completadas o por tiempo visto?
- ¿Los cursos se almacenan como un árbol (curso → módulos → lecciones) en un solo documento o como colecciones separadas?
- ¿La búsqueda de cursos se hace en el backend o se cachea localmente?

Sección 4: Flujo de Datos

Paso 1: Flujo Inscribirse a Curso

Dibuja la secuencia completa desde que el estudiante hace clic en "Inscribirse" hasta que ve el curso en "Mis Cursos".

Incluye:

- Validaciones (¿ya está inscrito? ¿el curso está publicado? ¿hay costo?)
- Transformaciones entre capas
- Actualización del estado del Cubit
- Manejo de errores (pago rechazado, curso lleno, etc.)

Paso 2: Flujo Completar Lección y Actualizar Progreso

Dibuja la secuencia desde que el estudiante termina un video hasta que ve su porcentaje de avance actualizado.

Pregúntate:

- ¿El progreso se calcula en el frontend o en el backend?
- ¿Qué pasa si el estudiante recarga la página sin internet?
- ¿El progreso se guarda automáticamente o requiere acción manual?

Paso 3: Flujo Publicar Curso (Instructor → Admin → Público)

Dibuja la máquina de estados de un curso: Borrador → En revisión → Publicado → Archivado.

Incluye:

- ¿Quién puede hacer cada transición?
- ¿Qué validaciones se ejecutan al solicitar publicación?
- ¿Qué pasa cuando se publica? (notificaciones, acceso a estudiantes, etc.)

Sección 5: Diseño Supabase

Aterriza el diseño en Supabase (teoría en [05e-diseno-supabase.md](#)):

¿Tu backend es una REST API (Python/otro)? Aquí no implementas el servidor: solo especificas el **contrato** (endpoints, DTOs, códigos de error, garantías de atomicidad y autorización) que consume tu `RemoteDataSource` con Dio; el backend la diseña e implementa.

1. **Tablas:** cursos, módulos, lecciones, inscripciones, progreso.
2. **Operaciones:** ¿qué UseCase usa select/insert/update/rpc?
3. **RLS:** policies que cumplan RS001 (progreso propio) y RS002 (solo instructor edita).
4. **Atomicidad:** ¿la inscripción a curso pago es atómica (RPC)?
5. **Realtime:** ¿el progreso se sincroniza en vivo?

Pregúntate:

- ¿El cálculo de progreso (RN004) es una función de BD o se calcula en el cliente?
- ¿Cómo evitas que un estudiante lea lecciones de un curso no inscrito (RN002)?
- ¿La transición "publicado" (RN008) la hace solo el admin: RLS o RPC con claim?

Sección 6: Criterios de Aceptación y Trazabilidad

Cierra el diseño (teoría en [05f-criterios-aceptacion-trazabilidad.md](#)):

1. **Criterios:** escribe al menos 5 en formato BDD.
2. **Matriz:** cruza UseCase → Regla → Contrato → Fuente de verdad → Test.

Pregúntate:

- ¿El criterio "marcar lección completada" considera la inscripción activa?

- ¿RN006 (instructor no se inscribe a su propio curso) tiene test?
 - ¿La matriz cubre RS001 y RS002 aunque no pasen por un UseCase?
-

Solución Sugerida

 Resuelve cada sección en papel primero. La solución sugerida es para comparar después.

FEATURE: Plataforma de Cursos Online (E-Learning)

[F]ormular:

- F1: Como estudiante, quiero explorar cursos e inscribirme para aprender a mi ritmo.
- F2: Como instructor, quiero crear cursos con lecciones para compartir mi conocimiento.
- F3: Como administrador, quiero aprobar cursos antes de publicarlos para garantizar calidad.

[A]ctorizar:

1. Estudiante (primario)
 - Explorar catálogo, inscribirse, ver lecciones, marcar completadas, ver progreso
2. Instructor (secundario)
 - CRUD cursos y lecciones, ver reportes de estudiantes
3. Admin (secundario)
 - Gestionar usuarios, aprobar/rechazar cursos
4. Sistema de Pagos (externo)
 - Cobrar inscripción en cursos pagos
5. Sistema de Videos (externo)
 - Alojar y servir videos (Vimeo, YouTube, etc.)

[D]escomponer:

Estudiante:

- [R] Explorar catálogo (búsqueda, filtros, paginación)
- [R] Ver detalle del curso
- [C] Inscribirse a curso (gratis o pago)
- [R] Ver lecciones del curso
- [U] Marcar lección como completada
- [R] Ver progreso general
- [R] Ver listado "Mis Cursos"

Instructor:

- [C] Crear curso
- [U] Editar curso (contenido, precio, imagen)
- [C] Agregar módulos y lecciones
- [U] Reordenar lecciones
- [U] Solicitar publicación
- [R] Ver reporte de estudiantes por curso

Admin:

- [R] Ver cursos pendientes de aprobación
- [U] Aprobar curso
- [U] Rechazar curso (con motivo)
- [CRUD] Gestionar usuarios

Sistema:

- [Validación] Validar contenido mínimo antes de publicar
- [Validación] Verificar que el estudiante no esté inscrito
- [Cálculo] Calcular progreso (% lecciones completadas)
- [Transición] Notificar al instructor al inscribirse un estudiante

[E]ntidades:

Curso: id, titulo, descripcion, precio, imagenUrl, estado (borrador, revisión, publicado, archivado), categoriaId, instructorId, fechaCreacion

Modulo: id, cursoId, titulo, orden

Leccion: id, moduloId, titulo, tipo (video/texto),
 contenidoUrl, duracionMinutos, orden

Inscripcion: id, estudianteId, cursoId, fecha,
 estado (activa, completada, cancelada),
 pagoId (nullable)

ProgresoLeccion: id, estudianteId, leccionId,
 completada, fechaCompletado

Usuario: id, nombre, email, rol (estudiante, instructor,
 admin), fotoUrl

Categoria: id, nombre, slug, icono

[R]eglas:

RN001: No se puede inscribir a un curso ya inscrito

RN002: Solo se pueden ver lecciones de cursos inscritos

RN003: Un curso debe tener al menos 1 lección para
 solicitar publicación

RN004: El progreso = (lecciones completadas / total
 lecciones) * 100

RN005: Al completar todas las lecciones, la inscripción
 pasa a estado "completada"

RN006: Un instructor no puede inscribirse a su propio curso

RN007: Los cursos pagos requieren confirmación de pago
 antes de activar la inscripción

RN008: Solo el admin puede cambiar el estado a "publicado"

RN009: No se puede eliminar un curso con estudiantes
 inscritos (solo archivarlo)

RN010: Las lecciones de tipo "video" requieren contenidoUrl
 no vacío

RT001: La búsqueda del catálogo se delega al backend
 (full-text search), no se filtra en el cliente

RS001: Un estudiante solo lee su propio progreso

RS002: Solo el instructor del curso puede editar su
 contenido

✔ Contrato CourseRepository

```
abstract class CourseRepository {
    Future<Either<Failure, List<Course>>> getCatalog({
        required int page,
        int? categoryId,
        String? searchQuery,
    });

    Future<Either<Failure, Course>> getCourseDetail(String courseId);

    Future<Either<Failure, Course>> createCourse(CreateCourseParams params);

    Future<Either<Failure, Course>> updateCourse(
        String courseId, UpdateCourseParams params);

    Future<Either<Failure, Course>> requestPublication(String courseId);

    Future<Either<Failure, Course>> approveCourse(String courseId);

    Future<Either<Failure, Course>> rejectCourse(
        String courseId, String reason);

    Future<Either<Failure, List<Course>>> getCoursesByInstructor(
        String instructorId);

    Future<Either<Failure, List<Course>>> getCoursesByStudent(
        String studentId);

    Future<Either<Failure, Module>> addModule(String courseId, Module module);

    Future<Either<Failure, Lesson>> addLesson(
        String moduleId, Lesson lesson);

    Future<Either<Failure, void>> reorderLessons(
        String moduleId, List<String> lessonIds);
}
```

✓ Contrato EnrollmentRepository

```
abstract class EnrollmentRepository {
    Future<Either<Failure, Enrollment>> enroll(
        String studentId, String courseId, String? paymentId);

    Future<Either<Failure, List<Enrollment>>> getStudentEnrollments(
        String studentId);

    Future<Either<Failure, bool>> isEnrolled(
        String studentId, String courseId);

    Future<Either<Failure, List<Student>>> getCourseStudents(
        String courseId);

    Future<Either<Failure, void>> markLessonCompleted(
        String studentId, String lessonId);

    Future<Either<Failure, double>> getProgress(
        String studentId, String courseId);

    Future<Either<Failure, List<LessonProgress>>> getLessonProgress(
        String studentId, String courseId);
}
```

✓ Estados del PlayerCubit

```
sealed class PlayerState {}

final class PlayerLoading extends PlayerState {
    final String lessonId;
}

final class PlayerReady extends PlayerState {
    final Lesson lesson;
    final bool isCompleted;
    final double courseProgress;
}

final class PlayerPlaying extends PlayerState {
    final Lesson lesson;
    final Duration position;
    final double courseProgress;
}

final class PlayerPaused extends PlayerState {
    final Lesson lesson;
    final Duration position;
    final double courseProgress;
}

final class PlayerError extends PlayerState {
    final String message;
    final String lessonId;
}

final class PlayerCompleted extends PlayerState {
    final Lesson lesson;
    final double courseProgress;
}
```

✓ ADR Sugerido

```
# ADR-004: Cálculo de Progreso del Estudiante

## Contexto
Necesitamos mostrar al estudiante su avance en cada curso.

## Decisión
El progreso se calculará como:
   $(lecciones\_completadas / total\_lecciones\_del\_curso) \times 100$ 

El cálculo se hará en el backend (Supabase) y se cacheará localmente.
Cada vez que se completa una lección, se recalcula y persiste.

## Consecuencias
Positivas:
- Simple y predecible para el estudiante
- Fácil de testear
- El frontend solo lee, no calcula

Negativas:
- No considera tiempo de visualización de video
- Una lección de 1 minuto cuenta igual que una de 30 minutos

## Alternativas consideradas
1. Progreso por minutos vistos / duración total:
   Descartado: más complejo, requiere tracking de posición de video,
   inconsistente si el estudiante adelanta el video.
2. Progreso calculado en frontend:
   Descartado: inconsistente entre dispositivos, se pierde al cerrar sesión.
```

✓ Flujo Completar Lección

```
ESTUDIANTE    → Termina de ver video o hace clic en "Completar"
WIDGET        → onCompleted() → playerCubit.markCompleted(lessonId)
CUBIT         → emit(PlayerCompleted(lesson, progress))
USECASE       → CompleteLesson.call(studentId, lessonId)
REPO IMPL     → 1. progressDS.markCompleted(studentId, lessonId)
               2. progressDS.getProgress(studentId, courseId)
               3. enrollmentDS.enrollment.completed =
                  (progress == 100.0) ? "completada" : "activa"
CUBIT         → fold()
               → éxito: emit(PlayerReady(lesson, isCompleted: true,
                  courseProgress: newProgress))
               → error: emite error pero mantiene lección completada
                  localmente (optimista)

ESTUDIANTE    → Ve su progreso actualizado en la barra de avance
```

✓ Diseño Supabase

```
enrollments
├── id          uuid PK
├── student_id  uuid FK → profiles
├── course_id   uuid FK → courses
├── status      enum (active, completed, cancelled)
└── payment_id  uuid (nullable)

lesson_progress
├── id          uuid PK
├── student_id  uuid FK → profiles
├── lesson_id   uuid FK → lessons
├── completed   bool
├── completed_at timestampz
└── unique (student_id, lesson_id)

-- RS001: cada estudiante solo lee su propio progreso
create policy "student reads own progress"
  on lesson_progress for select
  using (student_id = auth.uid());

-- RN002: solo se leen lecciones de cursos con inscripción activa
--   → RPC get_lesson(student_id, lesson_id) que valida la inscripción
--   → RLS: lessons visible solo vía courses publicadas
-- RN004: progreso = RPC calcular_progreso(student_id, course_id)
-- RN007: inscripción paga = RPC inscribirse que verifica payment_id
```

✓ Criterios de Aceptación (ejemplos)

Escenario: Completar la última lección
Dado un estudiante inscrito en un curso con 5 lecciones
Y 4 lecciones completadas
Cuando completa la lección 5
Entonces su progreso es 100% (RN004)
Y su inscripción pasa a "completada" (RN005)

Escenario: Ver lecciones sin estar inscrito
Dado un curso publicado
Cuando un estudiante no inscrito intenta abrir una lección
Entonces el sistema rechaza el acceso (RN002, RLS)

✓ Matriz de Trazabilidad

UseCase	Regla(s)	Contrato	Fuente de verdad	Test
EnrollCourse	RN001, RN007	EnrollmentRepository.enroll	RPC atómico	unit + integration
CompleteLesson	RN004, RN005	EnrollmentRepository.markLessonCompleted	RPC + trigger	unit
BrowseCourses	RT001	CourseRepository.getCatalog	API + full-text	unit

UseCase	Regla(s)	Contrato	Fuente de verdad	Test
ApproveCourse	RN003, RN008	CourseRepository.approveCourse	RPC + claim	unit + widget
Ver progreso	RS001	EnrollmentRepository.getLessonProgress	RLS	integration (RLS)
Ver lecciones	RN002, RS002	CourseRepository.getCourseDetail	RPC + RLS	integration (RLS)

Siguiendo paso

Continúa con el [Sistema de Facturación](#) para practicar con máquinas de estado complejas.

Tiempo estimado: 2-3 horas

Material: Papel y lápiz

Caso Práctico: Sistema de Facturación

Aplica FADER + Mapeo + Contratos + Flujo + Supabase + Criterios para diseñar un sistema de facturación con máquina de estados compleja.

Enunciado

Somos el equipo de ingeniería de una empresa SaaS. El equipo de producto nos pide:

Los clientes deben poder generar facturas (borrador), enviarlas a sus clientes, registrar pagos, y hacer seguimiento de facturas vencidas. El sistema debe soportar múltiples monedas, impuestos configurables por país, descuentos, facturas recurrentes, y notas de crédito. Los administradores deben poder emitir facturas desde el panel de control y generar reportes de ingresos.

Instrucciones

1. Trabaja en papel y lápiz. No abras el editor de código.
2. Sigue cada sección en orden.
3. Al final, compara con la solución sugerida.

Sección 0: Alcance

Antes de descomponer, define los límites del sistema (teoría en [00-alcance-feature.md](#)):

1. **Incluye:** ¿qué cubre el sistema de facturación?
2. **No incluye:** ¿qué NO cubre? (ej: conciliación bancaria, contabilidad completa)
3. **Dependencias:** ¿qué necesita que exista antes?
4. **Suposiciones:** ¿qué das por hecho?
5. **Preguntas abiertas:** ¿qué no sabes todavía?

Sección 1: FADER

Paso 1: Formular

Escribe al menos 3 enunciados "Como [actor], quiero [acción] para [valor]".

Pregúntate:

- ¿Quién crea la factura? ¿El emisor (vendedor) o el receptor (cliente)?
- ¿Se envía la factura al cliente o el cliente la descarga?
- ¿Una factura recurrente se genera automáticamente o manualmente?
- ¿Qué diferencia hay entre una nota de crédito y una factura de anulación?

Paso 2: Actorizar

Identifica todos los actores y sus permisos:

Actor	Tipo	¿Qué puede hacer?
Emisor (vendedor)	Primario	
Cliente (receptor)	Secundario	
Admin	Secundario	
Pasarela de Pagos	Externo	
Sistema de Impuestos	Externo	
?	?	

Pregúntate:

- ¿El cliente puede ver el estado de pago de sus facturas?
- ¿El admin puede emitir facturas en nombre del emisor?
- ¿El sistema de impuestos es externo (API de AFIP/SII) o interno?

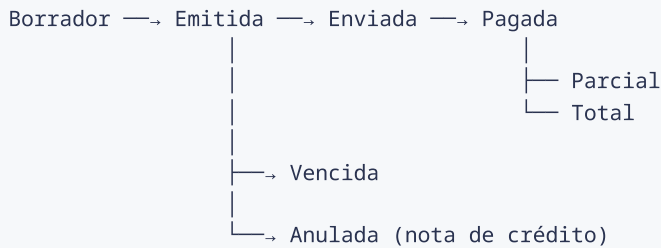
Paso 3: Descomponer

Enumera todas las operaciones atómicas.

Considera operaciones como:

- Crear factura (borrador)
- Emitir factura (validar y generar número fiscal si aplica)
- Enviar factura al cliente
- Registrar pago (total o parcial)
- Generar nota de crédito
- Programar factura recurrente
- Reporte de ingresos mensuales

Máquina de estados de una factura:



Identifica dependencias:

- ¿Qué se necesita para emitir una factura? (items, impuestos, cliente válido)
- ¿Qué pasa si se paga una factura ya vencida?
- ¿Se puede anular una factura pagada?

Paso 4: Entidades

Define las entidades de negocio:

Posibles entidades:

- Factura (Invoice)
- ItemFactura (InvoiceLine)
- Cliente (Customer)
- Emisor (Issuer)
- Pago (Payment)
- NotaCredito (CreditNote)
- Impuesto (Tax)
- PlanRecurrente (RecurringPlan)

Pregúntate:

- ItemFactura es una entidad o un value object?
- Pago puede ser parcial o total — ¿cómo se relaciona con Factura?
- ¿Cliente es el destinatario de la factura o el que paga?
- ¿Impuesto es un porcentaje fijo o varía por producto?

Paso 5: Reglas

Enuncia al menos 10 reglas de negocio con formato RN001, RN002... y añade reglas técnicas (RT) y de seguridad (RS) cuando aplique (teoría en [01-descomposicion-feature.md](#)).

Áreas a cubrir:

- Números de factura (secuenciales por emisor)
- Fechas (emisión no puede ser futura, vencimiento vs emisión)
- Impuestos (IVA 21%, RG 10%, exento)
- Pagos (parciales, totales, sobrantes)

- Notas de crédito (solo referencian una factura existente)
- Recurrentes (periodicidad, tope de emisiones)
- Anulación (no anular facturas con más de X días)

Sección 2: Mapeo a Capas



Paso 1: Estructura DOMAIN

Dibuja el árbol de `domain/` :

```
domain/
├── entities/
│   ├── invoice.dart
│   ├── invoice_line.dart
│   ├── customer.dart
│   ├── payment.dart
│   ├── credit_note.dart
│   ├── tax.dart
│   └── recurring_plan.dart
├── usecases/
│   ├── create_invoice.dart
│   ├── issue_invoice.dart
│   ├── send_invoice.dart
│   ├── register_payment.dart
│   ├── cancel_invoice.dart
│   ├── generate_credit_note.dart
│   ├── create_recurring_plan.dart
│   ├── generate_recurring_invoices.dart
│   └── get_revenue_report.dart
├── repositories/
│   ├── invoice_repository.dart
│   ├── customer_repository.dart
│   └── payment_repository.dart
├── services/
│   └── tax_calculator.dart (interface)
└── core/
    └── failures.dart
```

Pregúntate:

- ¿ `tax_calculator` es un servicio o parte de Invoice?
- ¿ `issue_invoice` y `create_invoice` son UseCases separados o fases del mismo?
- ¿ `generate_credit_note` crea una nueva factura o modifica la existente?



Paso 2: Estructura DATA

Dibuja el árbol de `data/` .

Considera:

- Fuente remota (API REST)
- Fuente local (caché para consulta offline)

- Modelos con fromJson/toJson
- Implementaciones de repositorios

Pregúntate:

- ¿El catálogo de impuestos se cachea o siempre se consulta al servidor?
- ¿Las facturas emitidas se almacenan localmente?
- ¿El reporte de ingresos se genera en backend o frontend?

Paso 3: Estructura PRESENTATION

Dibuja el árbol de `presentation/`.

Considera:

- Formulario de creación de factura (multipaso: seleccionar cliente → agregar items → impuestos → resumen)
- Lista de facturas con filtros (estado, fecha, cliente)
- Detalle de factura con timeline de estados
- Dashboard de ingresos

Sección 3: Contratos

Paso 1: Contrato InvoiceRepository

```
abstract class InvoiceRepository {  
    // CRUD de facturas  
    // Transiciones de estado (emitir, enviar, anular)  
    // Búsqueda con filtros (estado, fecha, cliente)  
    // ¿Qué failures produce cada operación?  
}
```

Paso 2: Contrato PaymentRepository

```
abstract class PaymentRepository {  
    // Registrar pago  
    // Obtener pagos de una factura  
    // Conciliar pagos  
    // Reembolsar  
}
```

Paso 3: Contrato TaxCalculator

```
abstract class TaxCalculator {  
    // Calcular impuestos para un conjunto de items  
    // Depende del país del cliente y tipo de producto  
    double calculateTax(ProductType type, double amount, String countryCode);  
}
```

Paso 4: Estados de UI

Diseña los estados para `CreateInvoiceCubit` (formulario multipaso).

Pregúntate:

- ¿Cómo modelas un formulario que tiene pasos? (seleccionar cliente → items → impuestos → revisar)
- ¿Cada paso tiene su propio estado o todo en un solo estado?
- ¿Cómo manejas validación por paso?

Paso 5: ADR

Escribe al menos un ADR. Ejemplos:

- ¿La generación de facturas recurrentes se hace con cron job o con Cloud Scheduler?
- ¿El número de factura se genera en frontend o backend?
- ¿Los pagos parciales se aplican al total o a items específicos?
- ¿El cálculo de impuestos es interno o llama a una API externa?

Sección 4: Flujo de Datos

Paso 1: Flujo Emitir Factura

Dibuja la secuencia desde que el usuario llena el formulario hasta que la factura tiene número fiscal.

Incluye:

- Validaciones en cada paso del formulario
- Transición de estado: Borrador → Emitida
- Asignación de número de factura secuencial
- Cálculo de impuestos
- Manejo de errores (cliente inválido, items sin stock, etc.)

Paso 2: Flujo Registrar Pago

Dibuja el flujo de registro de pago, incluyendo pagos parciales.

Pregúntate:

- ¿Cómo afecta un pago parcial al saldo pendiente?
- ¿Qué pasa si el pago supera el total? (sobrante)
- ¿La factura cambia a "pagada" solo cuando saldo = 0?

Paso 3: Flujo Generar Facturas Recurrentes

Dibuja el flujo automático de generación de facturas recurrentes.

Pregúntate:

- ¿Quién dispara el proceso? (cron, Cloud Function, webhook)
- ¿Qué facturas se generan hoy según los planes activos?
- ¿Qué pasa si falla la generación de una factura?
- ¿Cómo se notifica al cliente?

Sección 5: Diseño Supabase

Aterriza el diseño en Supabase (teoría en [05e-diseno-supabase.md](#)):

¿Tu backend es una REST API (Python/otro)? Aquí no implementas el servidor: solo especificas el **contrato** (endpoints, DTOs, códigos de error, garantías de atomicidad y autorización) que consume tu `RemoteDataSource` con Dio; el backend la diseña e implementa.

1. **Tablas:** facturas, items, pagos, notas de crédito, planes recurrentes.
2. **Operaciones:** ¿qué UseCase usa select/insert/update/rpc?
3. **RLS:** políticas que cumplan RS001 (emisor ve solo sus facturas).
4. **Atomicidad:** emisión con número secuencial (RT001) y registro de pago.
5. **Realtime:** ¿el timeline de estados se sincroniza en vivo?

Pregúntate:

- ¿El contador `proximoNumeroFactura` vive en la BD (transacción) o en el cliente?
- ¿Marcar "vencida" (RN007) es un cron de Postgres o un trigger?
- ¿Los pagos parciales (RN008) se validan con un RPC atómico?

Sección 6: Criterios de Aceptación y Trazabilidad

Cierra el diseño (teoría en [05f-criterios-aceptacion-trazabilidad.md](#)):

1. **Criterios:** escribe al menos 5 en formato BDD (prioriza la máquina de estados).
2. **Matriz:** cruza UseCase → Regla → Contrato → Fuente de verdad → Test.

Pregúntate:

- ¿El criterio "emitir sin items" cubre RN002 y falla con mensaje claro?
 - ¿RN009 (sobrepago → nota de crédito) tiene un criterio y un test?
 - ¿RS002 (número no editable) se verifica en la matriz aunque no pase por UseCase?
-

Solución Sugerida

 Resuelve cada sección en papel primero. La solución sugerida es para comparar después.

FEATURE: Sistema de Facturación

[F]ormular:

- F1: Como emisor, quiero crear y emitir facturas para cobrar a mis clientes.
- F2: Como emisor, quiero registrar pagos para hacer seguimiento de cuentas corrientes.
- F3: Como administrador, quiero generar reportes de ingresos para analizar la salud financiera.

[A]ctorizar:

1. Emisor (primario) – crea, emite, envía, anula facturas
2. Cliente (secundario) – recibe facturas, ve estado
3. Admin (secundario) – reportes, gestionar plantillas
4. API de Impuestos (externa) – validar cálculos fiscales
5. Pasarela de Pago (externa) – procesar cobros
6. Cron/Cloud Scheduler (sistema) – generar recurrentes

[D]escomponer:

Emisor:

- [C] Crear factura borrador
- [U] Editar factura (solo en borrador)
- [Transición] Emitir factura (borrador → emitida)
- [Transición] Enviar factura (emitida → enviada)
- [Transición] Anular factura (→ anulada + nota crédito)
- [C] Registrar pago
- [C] Programar factura recurrente
- [R] Ver reporte de ingresos

Admin:

- [R] Ver dashboard financiero
- [CRUD] Gestionar plantillas de factura

Sistema:

- [Transición] Generar facturas recurrentes (cron)
- [Validación] Validar factura antes de emitir
- [Validación] Marcar vencidas automáticamente
- [Cálculo] Calcular impuestos según país y tipo producto

[E]ntidades:

Factura: id, numeroFactura, emisorId, clienteId,
fechaEmision, fechaVencimiento,
estado (borrador, emitida, enviada, pagada,
vencida, anulada),
moneda, subtotal, descuento, impuesto, total,
notas, plantillaId

ItemFactura: id, facturaId, descripcion, cantidad,
precioUnitario, porcentajeImpuesto, total

Cliente: id, nombre, email, direccion, pais,
rut/cuit, condicionFiscal

Emisor: id, nombre, rut, direccion, pais,
proximoNumeroFactura

Pago: id, facturaId, monto, fecha, metodo,
referenciaExterna, estado

NotaCredito: id, facturaOriginalId, motivo, monto,
fecha

PlanRecurrente: id, emisorId, clienteId, periodicidad,
siguienteEmision, items, monto

Impuesto: id, pais, tipoProducto, porcentaje,

codigoFiscal

[R]eglas:

RN001: El número de factura es secuencial por emisor

RN002: No se puede emitir una factura sin items

RN003: fechaVencimiento > fechaEmision

RN004: Solo facturas en borrador se pueden editar

RN005: Solo facturas emitidas se pueden enviar

RN006: Solo facturas emitidas/enviadas se pueden pagar

RN007: Una factura se marca vencida si fechaVencimiento < hoy y estado es emitida o enviada

RN008: Pagos parciales: estado = "pagada" solo si suma pagos >= total

RN009: Si suma pagos > total, generar nota de crédito por el excedente

RN010: Nota de crédito solo referencia una factura emitida

RN011: Al anular, el monto queda como crédito a favor del cliente

RN012: Las recurrentes se generan el día siguiente al de su fecha de siguienteEmision

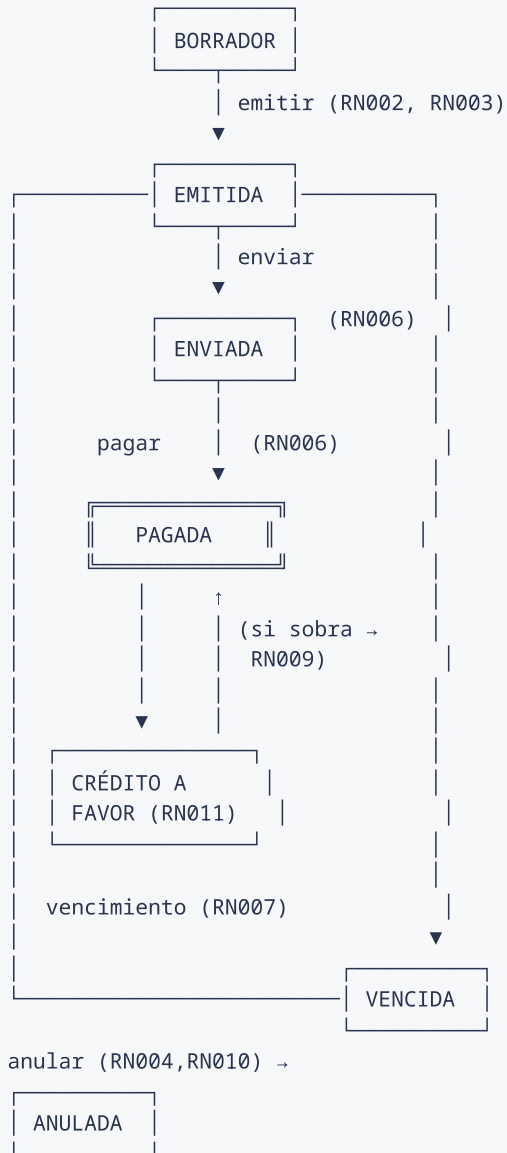
RN013: Impuesto IVA 21% para productos nacionales, 0% para exportaciones

RT001: Número secuencial FFF-000001 se genera en el backend (transacción atómica sobre contador)

RS001: Un emisor solo ve sus propias facturas

RS002: El número de factura no es editable por el frontend (columna protegida por RLS)

✓ Máquina de Estados de Factura



✓ Contrato InvoiceRepository

```
abstract class InvoiceRepository {  
    // CRUD  
    Future<Either<Failure, Invoice>> createDraft(CreateInvoiceParams params);  
    Future<Either<Failure, Invoice>> updateDraft(  
        String invoiceId, UpdateInvoiceParams params);  
    Future<Either<Failure, Invoice>> getInvoice(String invoiceId);  
    Future<Either<Failure, List<Invoice>>> getInvoices({  
        required String issuerId,  
        InvoiceStatus? status,  
        DateTime? fromDate,  
        DateTime? toDate,  
        String? customerId,  
        int page = 1,  
    });  
  
    // Transiciones de estado  
    Future<Either<Failure, Invoice>> issue(String invoiceId);  
    Future<Either<Failure, Invoice>> send(String invoiceId);  
    Future<Either<Failure, Invoice>> cancel(String invoiceId, String reason);  
  
    // Notas de crédito  
    Future<Either<Failure, CreditNote>> generateCreditNote({  
        required String invoiceId,  
        required double amount,  
        required String reason,  
    });  
  
    // Recurrentes  
    Future<Either<Failure, RecurringPlan>> createRecurringPlan(  
        CreateRecurringPlanParams params);  
    Future<Either<Failure, List<Invoice>>> generateRecurringInvoices();  
}
```

✓ ADR Sugerido

ADR-005: Generación del Número de Factura

Contexto

Cada factura emitida necesita un número único y secuencial por emisor.
El formato requerido es FFF-000001 (prefijo + número de 6 dígitos).

Decisión

El número de factura se genera EXCLUSIVAMENTE en el backend (Supabase), usando una transacción atómica que incrementa un contador en la tabla `emisores.proximoNumeroFactura`. Nunca se genera en el frontend.

Consecuencias

Positivas:

- Garantiza secuencialidad sin huecos ni duplicados
- El frontend no puede manipular la numeración
- Fácil de auditar

Negativas:

- La factura debe estar en borrador y enviarse al backend para obtener número
- Si falla la conexión al emitir, la factura queda sin número

Alternativas consideradas

1. UUID como número de factura:
Descartado: requisito legal exige numeración secuencial.
2. Generar en frontend con optimistic lock:
Descartado: riesgo de colisión y manipulación.
3. Generar al crear borrador vs al emitir:
Elegimos al emitir para evitar huecos por borradores descartados.

✓ Flujo Emitir Factura (resumido)

```
USUARIO    → Completa formulario (cliente, items, impuestos)
CUBIT      → Validaciones por paso
              Paso 1: cliente válido
              Paso 2: items con precio > 0
              Paso 3: impuestos calculados
              Paso 4: revisar y confirmar

USUARIO    → Tap "Emitir Factura"
CUBIT      → emit(InvoiceSubmitting)
USECASE    → IssueInvoice.call(invoiceId)
              → Valida RN002 (items no vacío)
              → Valida RN003 (fechas válidas)
              → InvoiceRepository.issue(invoiceId)
REPO IMPL  → 1. remoteDS.getNextInvoiceNumber(issuerId)
              2. remoteDS.updateInvoiceStatus(id, "emitida", number)
              3. Atómico: incrementar contador
              4. remoteDS.fetchInvoice(id)
              5. model.toEntity()
USECASE    → Right(invoice)
CUBIT      → emit(InvoiceIssued(invoice))
WIDGET     → Muestra factura emitida con número asignado

ERRORES:
- items vacío → InvoiceHasNoItems
- fecha inválida → InvalidInvoiceDate
- cliente sin datos fiscales → MissingTaxInfo
```

✓ Diseño Supabase

```
invoices
├── id                uuid PK
├── number            text (asignado al emitir, RT001)
├── issuer_id         uuid FK → profiles
├── customer_id       uuid FK → customers
├── status            enum (draft, issued, sent, paid, overdue, voided)
├── due_date          date
├── total             numeric
├── currency          text
└── constraint chk_dates check (due_date > issued_at)

issuers
├── id                uuid PK
└── next_invoice_number int -- contador atómico (RT001)

payments
├── id                uuid PK
├── invoice_id         uuid FK → invoices
├── amount            numeric
└── status            enum (pending, confirmed)

-- RS001: un emisor solo ve sus propias facturas
create policy "issuer sees own invoices"
  on invoices for select
  using (issuer_id = auth.uid());

-- RN001/RT001: emitir = RPC emitir_factura(p_invoice_id)
--   → BEGIN; UPDATE issuers SET next_invoice_number = n + 1 RETURNING n;
--   → UPDATE invoices SET number = ..., status = 'issued';
--   → COMMIT; (falla como unidad → sin huecos ni duplicados)
```

✓ Criterios de Aceptación (ejemplos)

Escenario: Emitir factura sin items
Dado un borrador sin items
Cuando el emisor intenta emitirlo
Entonces el sistema rechaza con InvoiceHasNoItems (RN002)

Escenario: Pago parcial que completa el total
Dado una factura emitida por \$100 con un pago de \$60
Cuando se registra un pago de \$40
Entonces el saldo pendiente es \$0
Y la factura pasa a "pagada" (RN008)

Escenario: Sobre pago
Dado una factura emitida por \$100
Cuando se registra un pago de \$120
Entonces la factura pasa a "pagada"
Y se genera una nota de crédito por \$20 (RN009)

✓ Matriz de Trazabilidad

UseCase	Regla(s)	Contrato	Fuente de verdad	Test
IssueInvoice	RN001, RN002, RN003	InvoiceRepository.issue	RPC atómico (RT001)	unit + integration
RegisterPayment	RN006, RN008, RN009	PaymentRepository (vía Invoice)	RPC atómico	unit + integration
CancelInvoice	RN004, RN010, RN011	InvoiceRepository.cancel	API + trigger	unit + widget
GenerateRecurring	RN012	InvoiceRepository.generateRecurringInvoices	cron + RPC	integration
Ver facturas propias	RS001, RS002	InvoiceRepository.getInvoices	RLS	integration (RLS)

Siguiendo paso

Continúa con la [App de Delivery](#) para practicar con flujos en tiempo real y geolocalización.

Tiempo estimado: 2-3 horas

Material: Papel y lápiz

Caso Práctico: App de Delivery

Aplica FADER + Mapeo + Contratos + Flujo + Supabase + Criterios para diseñar una app de delivery con múltiples actores, geolocalización y tiempo real.

Enunciado

Somos el equipo de ingeniería de una startup de delivery de comida. El equipo de producto nos pide:

Los clientes deben poder ver restaurantes cercanos, hacer pedidos, rastrear a su repartidor en tiempo real, y pagar online o en efectivo. Los restaurantes deben poder gestionar su menú, aceptar o rechazar pedidos, y marcar pedidos como listos. Los repartidores deben poder ver pedidos disponibles cercanos, aceptar entregas, y actualizar el estado del pedido (en camino, entregado). Los administradores deben poder gestionar usuarios, comisiones, y ver reportes de operaciones.

Instrucciones

1. Trabaja en papel y lápiz. No abras el editor de código.
2. Sigue cada sección en orden.
3. Al final, compara con la solución sugerida.

Sección 0: Alcance

Antes de descomponer, define los límites del sistema (teoría en [00-alcance-feature.md](#)):

1. **Incluye:** ¿qué cubre la app de delivery?
2. **No incluye:** ¿qué NO cubre? (ej: delivery de supermercado, programa de fidelidad)
3. **Dependencias:** ¿qué necesita que exista antes?
4. **Suposiciones:** ¿qué das por hecho?
5. **Preguntas abiertas:** ¿qué no sabes todavía?

Sección 1: FADER

Paso 1: Formular

Escribe al menos 4 enunciados "Como [actor], quiero [acción] para [valor]".

Pregúntate:

- ¿El delivery es solo de comida o de cualquier producto?
- ¿El cliente paga al restaurante, a la plataforma, o al repartidor?
- ¿Los precios varían por zona o por distancia?
- ¿El repartidor puede aceptar múltiples pedidos a la vez?

Paso 2: Actorizar

Identifica todos los actores y sus permisos:

Actor	Tipo	¿Qué puede hacer?
Cliente	Primario	
Restaurante	Secundario	
Repartidor	Secundario	
Admin	Terciario	
Pasarela de Pagos	Externo	
API de Mapas	Externo	
?	?	

Pregúntate:

- ¿El restaurante puede ver la ubicación del repartidor asignado?
- ¿El cliente puede ver la ubicación del restaurante Y del repartidor?
- ¿El repartidor puede rechazar un pedido después de aceptarlo?
- ¿Hay un modo "invitado" o siempre requiere registro?

Paso 3: Descomponer

Enumera todas las operaciones atómicas.

Considera operaciones como:

Cliente:

- Ver restaurantes cercanos (geolocalización)
- Ver menú de un restaurante
- Agregar items al pedido
- Hacer pedido (con pago)

- Rastrear pedido en tiempo real
- Calificar restaurante y repartidor

Restaurante:

- Gestionar menú (items, precios, disponibilidad)
- Recibir pedido entrante
- Aceptar/rechazar pedido
- Marcar pedido como "en preparación"
- Marcar pedido como "listo para recoger"

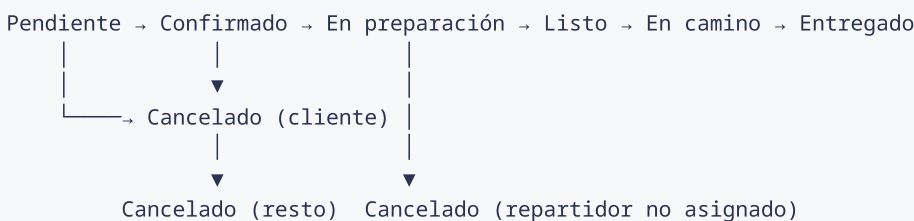
Repartidor:

- Ver pedidos disponibles cercanos
- Aceptar pedido (asignación)
- Marcar "en camino al restaurante"
- Marcar "recogido"
- Marcar "en camino al cliente"
- Marcar "entregado"

Sistema:

- Asignar repartidor automáticamente
- Calcular tarifa de envío
- Estimar tiempo de entrega
- Actualizar ubicación en tiempo real (WebSockets/streams)

Máquina de estados de un pedido:



Identifica dependencias:

- ¿Qué pasa si no hay repartidores disponibles?
- ¿El pago se cobra al hacer el pedido o al entregarlo?
- ¿El cliente puede cancelar después de que el restaurante aceptó?



Paso 4: Entidades

Define las entidades de negocio:

Posibles entidades:

- **Usuario** (User) — con roles: cliente, restaurante, repartidor, admin
- **Restaurante** (Restaurant)

- `ItemMenu` (MenuItem)
- `Pedido` (Order)
- `ItemPedido` (OrderItem)
- `Repartidor` (DeliveryPerson)
- `Ubicacion` (Location) — value object
- `Calificacion` (Rating)
- `Pago` (Payment)
- `Comision` (Commission)

Pregúntate:

- `Repartidor` y `Restaurante` son subtipos de `Usuario` o entidades separadas?
- `Ubicacion` es un value object (inmutable, sin identidad propia)?
- `Pedido` contiene la dirección de entrega o es un valor calculado?
- ¿`ItemMenu` tiene precio variable por zona?

Paso 5: Reglas

Enuncia al menos 10 reglas de negocio con formato RN001, RN002... y añade reglas técnicas (RT) y de seguridad (RS) cuando aplique (teoría en [01-descomposicion-feature.md](#)).

Áreas a cubrir:

- Asignación de repartidores (distancia máxima, capacidad)
- Tiempo máximo de espera para aceptar pedido
- Cancelaciones (quién puede cancelar y en qué estados)
- Pagos (contra entrega vs online)
- Tarifas de envío (distancia, hora del día, demanda)
- Calificaciones (solo pedidos entregados, período de 7 días)
- Comisiones de la plataforma
- Disponibilidad de items (no pedir items agotados)

Sección 2: Mapeo a Capas

Paso 1: Estructura DOMAIN

Dibuja el árbol de `domain/`:

```

domain/
├── entities/
│   ├── user.dart
│   ├── restaurant.dart
│   ├── menu_item.dart
│   ├── order.dart
│   ├── order_item.dart
│   ├── delivery_person.dart
│   ├── rating.dart
│   └── payment.dart
├── value_objects/
│   ├── location.dart
│   └── money.dart
├── usecases/
│   ├── browse_restaurants.dart
│   ├── place_order.dart
│   ├── accept_order.dart
│   ├── assign_delivery.dart
│   ├── track_order.dart
│   ├── cancel_order.dart
│   ├── update_location.dart
│   ├── complete_delivery.dart
│   └── rate_order.dart
├── repositories/
│   ├── restaurant_repository.dart
│   ├── order_repository.dart
│   ├── delivery_repository.dart
│   └── payment_repository.dart
├── services/
│   ├── location_service.dart (interface)
│   └── payment_service.dart (interface)
└── core/
    └── failures.dart

```

Pregúntate:

- ¿ `Location` es value object porque dos ubicaciones con mismas coordenadas son iguales?
- ¿ `track_order` es un UseCase o es parte del Cubit vía Stream?
- ¿ `browse_restaurants` necesita geolocalización o recibe lat/lng como parámetro?



Paso 2: Estructura DATA

Dibuja el árbol de `data/` :

```

data/
├── datasources/
│   ├── restaurant_remote_datasource.dart
│   ├── order_remote_datasource.dart
│   ├── order_realtime_datasource.dart ← WebSockets/Streams
│   └── location_datasource.dart      ← GPS del dispositivo
├── models/
│   ├── restaurant_model.dart
│   ├── order_model.dart
│   ├── user_model.dart
│   └── location_model.dart
└── repositories/
    ├── restaurant_repository_impl.dart
    ├── order_repository_impl.dart
    ├── delivery_repository_impl.dart
    └── payment_repository_impl.dart

```

Pregúntate:

- La actualización en tiempo real de ubicación, ¿va por WebSocket o polling?
- ¿La ubicación GPS es un DataSource o un Service?
- ¿Necesitas un DataSource especial para streams o con el remoto alcanza?

Paso 3: Estructura PRESENTATION

Dibuja el árbol de `presentation/`:

```

presentation/
├── cubit/
│   ├── restaurant_list_cubit.dart
│   ├── menu_cubit.dart
│   ├── cart_cubit.dart
│   ├── checkout_cubit.dart
│   ├── order_tracking_cubit.dart
│   ├── delivery_cubit.dart
│   └── ratings_cubit.dart
├── pages/
│   ├── home_page.dart
│   ├── restaurant_detail_page.dart
│   ├── cart_page.dart
│   ├── order_tracking_page.dart
│   ├── delivery_home_page.dart
│   └── restaurant_orders_page.dart
└── widgets/
    ├── restaurant_card.dart
    ├── menu_item_tile.dart
    ├── order_status_timeline.dart
    ├── delivery_map.dart
    └── rating_widget.dart

```

Pregúntate:

- ¿Cuántos Cubits necesitas realmente?
- El mapa de rastreo, ¿es un widget o una página?
- ¿El flujo del repartidor es una app separada o la misma app con otro rol?

Sección 3: Contratos

Paso 1: Contrato OrderRepository

```
abstract class OrderRepository {  
    // CRUD de pedidos  
    // Transiciones de estado  
    // Consultas con filtros  
    // Streams para actualizaciones en tiempo real  
}
```

Paso 2: Contrato DeliveryRepository

```
abstract class DeliveryRepository {  
    // Asignar repartidor  
    // Obtener pedidos disponibles cercanos  
    // Actualizar ubicación del repartidor  
    // Stream de ubicación del repartidor  
}
```

Paso 3: Contrato LocationService

```
abstract class LocationService {  
    // Obtener ubicación actual del dispositivo  
    // Stream de ubicación en tiempo real  
    // Calcular distancia entre dos puntos  
}
```

Paso 4: Contrato para Realtime

El tracking en tiempo real es un caso especial. Necesitas contratos que retornen Streams:

```
abstract class OrderRealtimeDataSource {  
    // Stream que emite cambios del pedido en tiempo real  
    Stream<OrderModel> watchOrder(String orderId);  
  
    // Stream que emite ubicación del repartidor  
    Stream<LocationModel> watchDeliveryLocation(String deliveryPersonId);  
  
    // Emitir nueva ubicación del repartidor  
    Future<void> updateLocation(String deliveryPersonId, LocationModel location);  
}
```

Pregúntate:

- ¿El Stream se cierra cuando el pedido se entrega?
- ¿Cómo manejas la reconexión si se pierde el WebSocket?
- ¿El UseCase retorna un Stream o el Cubit se suscribe directamente?

Paso 5: ADR

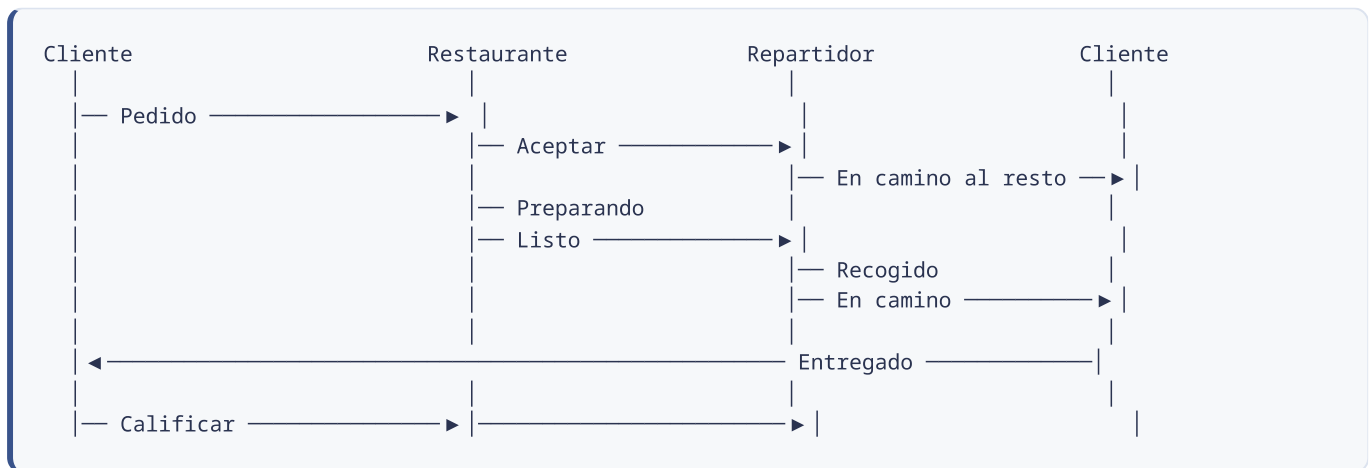
Escribe al menos un ADR. Ejemplos:

- ¿La asignación de repartidores es automática (algoritmo) o manual (repartidor elige)?
- ¿La tarifa de envío se calcula por distancia fija o por demanda (surge pricing)?
- ¿El tracking en tiempo real usa WebSockets (Supabase Realtime) o polling?

Sección 4: Flujo de Datos

Paso 1: Flujo Completar Pedido (Cliente → Restaurante → Repartidor → Cliente)

Dibuja la secuencia completa del ciclo de vida de un pedido:



Incluye:

- Para cada flecha, identifica qué capa procesa la acción
- Dónde se emiten los eventos de cambio de estado
- Dónde se actualiza la UI en tiempo real
- Manejo de errores (repartidor no acepta, restaurante rechaza, pago falla)

Paso 2: Flujo de Asignación de Repartidor

Dibuja el flujo cuando un pedido está listo y se necesita asignar un repartidor.

Pregúntate:

- ¿El sistema asigna automáticamente al repartidor más cercano?
- ¿O los repartidores ven una lista de pedidos disponibles y aceptan?
- ¿Qué pasa si ningún repartidor acepta en X minutos?
- ¿Cómo se actualiza la UI del cliente mientras se asigna un repartidor?

Paso 3: Flujo de Tracking en Tiempo Real

Dibuja el flujo de datos desde que el repartidor se mueve hasta que el cliente ve su ubicación en el mapa.

Considera:

- GPS del repartidor → DataSource → Supabase Realtime → Stream en Cubit → Widget de mapa
 - Frecuencia de actualización de ubicación
 - Manejo de pérdida de conexión
 - Actualización de tiempo estimado de llegada
-

Sección 5: Diseño Supabase

Aterriza el diseño en Supabase (teoría en [05e-diseno-supabase.md](#)):

¿Tu backend es una REST API (Python/otro)? Aquí no implementas el servidor: solo especificas el **contrato** (endpoints, DTOs, códigos de error, garantías de atomicidad y autorización) que consume tu `RemoteDataSource` con Dio; el backend la diseña e implementa.

1. **Tablas:** usuarios (roles), restaurantes, menú, pedidos, pagos, calificaciones, ubicaciones.
2. **Operaciones:** ¿qué UseCase usa select/insert/update/rpc?
3. **RLS:** políticas para RS001 (cliente ve solo su repartidor) y RS002 (repartidor actualiza su ubicación).
4. **Atomicidad:** asignación de repartidor (RN001, RN006) y cobro online (RN007).
5. **Realtime:** tabla `delivery_locations` con Broadcast y tabla `orders` con cambios.

Pregúntate:

- ¿La tarifa (RN005) se calcula en Postgres con PostGIS (RT002) o en el cliente?
 - ¿Cómo evitas que dos repartidores acepten el mismo pedido a la vez (race condition)?
 - ¿Las reglas RN003/RN004 (timeout del restaurante) son un cron de Postgres?
-

Sección 6: Criterios de Aceptación y Trazabilidad

Cierra el diseño (teoría en [05f-criterios-aceptacion-trazabilidad.md](#)):

1. **Criterios:** escribe al menos 5 en formato BDD (prioriza la máquina de estados y el tracking).
2. **Matriz:** cruza UseCase → Regla → Contrato → Fuente de verdad → Test.

Pregúntate:

- ¿El criterio "dos repartidores aceptan el mismo pedido" tiene caso de prueba de race condition?
 - ¿RN009 (calificar dentro de 7 días) se puede testear?
 - ¿RS001 y RS002 están en la matriz aunque sean solo RLS?
-

Solución Sugerida

 Resuelve cada sección en papel primero. La solución sugerida es para comparar después.

FEATURE: App de Delivery

[F]ormular:

- F1: Como cliente, quiero pedir comida de restaurantes cercanos para recibirla en mi casa.
- F2: Como restaurante, quiero gestionar pedidos entrantes para prepararlos a tiempo.
- F3: Como repartidor, quiero ver pedidos disponibles cercanos para elegir los que me convengan.
- F4: Como cliente, quiero rastrear a mi repartidor en tiempo real para saber cuándo llega mi pedido.

[A]ctorizar:

1. Cliente (primario)
 - Ver restaurantes, menú, hacer pedido, rastrear, calificar, historial
2. Restaurante (secundario)
 - Gestionar menú, aceptar/rechazar pedidos, marcar estados, ver historial
3. Repartidor (secundario)
 - Ver pedidos disponibles, aceptar, actualizar estados, compartir ubicación
4. Admin (terciario)
 - Gestionar usuarios, comisiones, reportes
5. Pasarela de Pagos (externa)
6. API de Mapas (externa) – Google Maps, Mapbox

[D]escomponer:

Cliente:

- [R] Ver restaurantes cercanos (geolocalización)
- [R] Ver menú del restaurante
- [C] Agregar item al carrito
- [C] Hacer pedido (checkout)
- [R] Rastrear pedido (stream en tiempo real)
- [U] Cancelar pedido (solo en estados iniciales)
- [C] Calificar (restaurante + repartidor)

Restaurante:

- [CRUD] Gestionar items del menú
- [R] Ver pedidos entrantes (stream en tiempo real)
- [U] Aceptar pedido
- [U] Rechazar pedido (con motivo)
- [U] Marcar "en preparación"
- [U] Marcar "listo"

Repartidor:

- [R] Ver pedidos disponibles cercanos
- [U] Aceptar entrega
- [U] Marcar "en camino al restaurante"
- [U] Marcar "recogido"
- [U] Marcar "en camino al cliente"
- [U] Marcar "entregado"
- [C] Compartir ubicación (stream de GPS)

Sistema:

- [Cálculo] Calcular tarifa de envío (distancia)
- [Cálculo] Estimar tiempo de entrega
- [Validación] Validar cobertura (¿el restaurante entrega en esa zona?)

- [Transición] Re-asignar si repartidor no avanza
- [Transición] Notificar cambios de estado

[E]ntidades:

Usuario: id, nombre, email, telefono, rol,
ubicacionActual, activo

Restaurante: id, nombre, direccion, ubicacion,
categoria, rating, tiempoEstimado,
activo, fotoUrl

ItemMenu: id, restauranteId, nombre, descripcion,
precio, categoria, disponible, fotoUrl

Pedido: id, clienteId, restauranteId, repartidorId?,
items, estado, subtotal, tarifaEnvio, total,
direccionEntrega, ubicacionEntrega,
metodoPago, tiempoEstimado,
fechaCreacion, fechaEntregado

ItemPedido: productoId, nombre, cantidad, precioUnitario

EstadoPedido (enum): pendiente, confirmado,
enPreparacion, listo, enCaminoAlResto, recogido,
enCaminoAlCliente, entregado, cancelado

Pago: id, pedidoId, monto, metodo, estado,
referenciaExterna

Calificacion: id, pedidoId, usuarioId,
puntuacion, comentario, fecha

Ubicacion (value object): latitud, longitud

[R]eglas:

RN001: El repartidor no puede tener más de 1 pedido activo a la vez

RN002: El cliente puede cancelar solo si estado es pendiente o confirmado

RN003: El restaurante tiene 3 minutos para aceptar o rechazar un pedido entrante

RN004: Si el restaurante no responde en 3 min, el pedido se cancela automáticamente

RN005: La tarifa de envío es \$1.5/km desde el restaurante al cliente

RN006: El repartidor solo puede ser asignado si está a menos de 2km del restaurante

RN007: El pago se procesa al hacer el pedido si es online; al entregar si es efectivo

RN008: Solo se puede calificar si el pedido fue entregado

RN009: La calificación debe hacerse dentro de los 7 días posteriores a la entrega

RN010: Si el repartidor no se mueve por 5 minutos, notificar al admin para re-asignación

RN011: No se puede pedir un item no disponible

RN012: El restaurante solo entrega en un radio máximo de 5km

RT001: La ubicación del repartidor se emite cada 3s vía Supabase Realtime (Broadcast), no polling

RT002: Tarifa de envío = RPC en Supabase que usa PostGIS (calculada en servidor)

RS001: El cliente solo ve la ubicación de su propio repartidor asignado

RS002: El repartidor solo actualiza su propia ubicación (RLS por auth.uid())

✓ Contrato OrderRepository

```
abstract class OrderRepository {  
    // Comandos  
    Future<Either<Failure, Order>> placeOrder(PlaceOrderParams params);  
    Future<Either<Failure, Order>> cancelOrder(  
        String orderId, String reason);  
    Future<Either<Failure, Order>> acceptOrder(String orderId);  
    Future<Either<Failure, Order>> rejectOrder(  
        String orderId, String reason);  
  
    // Transiciones de estado  
    Future<Either<Failure, Order>> markPreparing(String orderId);  
    Future<Either<Failure, Order>> markReady(String orderId);  
    Future<Either<Failure, Order>> markPickedUp(String orderId);  
    Future<Either<Failure, Order>> markDelivered(String orderId);  
  
    // Consultas  
    Future<Either<Failure, Order>> getOrder(String orderId);  
    Future<Either<Failure, List<Order>>> getOrdersByStatus(  
        String userId, OrderStatus status);  
  
    // Streams en tiempo real  
    Stream<Either<Failure, Order>> watchOrder(String orderId);  
    Stream<Either<Failure, List<Order>>> watchRestaurantOrders(  
        String restaurantId);  
}
```

✓ Contrato DeliveryRepository

```
abstract class DeliveryRepository {  
    Future<Either<Failure, List<Order>>> getAvailableOrders(  
        Location nearLocation, double radiusKm);  
  
    Future<Either<Failure, Order>> acceptDelivery(  
        String orderId, String deliveryPersonId);  
  
    Future<Either<Failure, void>> updateLocation(  
        String deliveryPersonId, Location location);  
  
    Stream<Location> watchDeliveryLocation(String deliveryPersonId);  
  
    Future<Either<Failure, double>> calculateDeliveryFee(  
        Location restaurantLocation, Location customerLocation);  
}
```

✓ ADR Sugerido

ADR-006: Tracking en Tiempo Real con Supabase Realtime

Contexto

El cliente debe ver la ubicación del repartidor en tiempo real y los cambios de estado del pedido sin recargar la página.

Decisión

Usaremos Supabase Realtime (WebSockets) para:

1. Suscribirse a cambios en la tabla `orders` (cambios de estado)
2. Suscribirse a cambios en la tabla `delivery\_locations` (ubicación) usando la capacidad de Broadcast de Supabase Realtime
3. El repartidor emite su ubicación cada 3 segundos desde el GPS
4. El cliente recibe actualizaciones vía Stream en el Cubit

Consecuencias

Positivas:

- Sin servidor propio de WebSockets
- Escalable (Supabase maneja la conexión)
- Integración directa con Flutter (supabase_flutter)

Negativas:

- Costo de suscripción Realtime en producción
- Latencia de 1-3 segundos (suficiente para tracking)
- Dependencia de conectividad a internet

Alternativas consideradas

1. Polling HTTP cada 5 segundos:
Descartado: mayor latencia, más requests, peor experiencia.
2. Firebase Realtime Database:
Descartado: ya usamos Supabase, no queremos dos backends.
3. WebSocket propio con server-sent events:
Descartado: sobreingeniería, Supabase Realtime ya resuelve esto.

✓ Flujo de Asignación de Repartidor (diagrama)

PEDIDO CONFIRMADO (restaurante aceptó, comida en preparación)

SISTEMA: Buscar repartidores
disponibles en un radio de 2km
del restaurante (RN006)

¿Hay repartidores? —Sí—► Notificar a repartidores
cercanos con el pedido

REPARTIDOR
Ve pedido en
lista disponibles

¿Acepta?

Sí —► Asignado
No —► Sigue
buscando

No hay repis —► Esperar 30s y reintentar

3 intentos sin éxito —►
Cancelar pedido con motivo
"No hay repartidores disponibles"

Aparece repi —► Asignar

✓ Diseño Supabase

```
orders
├── id                uuid PK
├── client_id         uuid FK → profiles
├── restaurant_id     uuid FK → restaurants
├── delivery_person_id uuid (nullable)
├── status            enum (pendiente, confirmado, enPreparacion,
                        listo, enCaminoAlResto, recogido,
                        enCaminoAlCliente, entregado, cancelado)
├── total             numeric
└── delivery_fee      numeric

delivery_locations
├── delivery_person_id uuid PK
├── lat                float8
├── lng                float8
└── updated_at        timestampz

-- RS001: el cliente solo lee la ubicación de SU repartidor
create policy "client reads own courier location"
  on delivery_locations for select
  using (delivery_person_id in (
    select o.delivery_person_id from orders o
    where o.client_id = auth.uid() and o.status not in ('entregado','cancelado')
  ));

-- RS002: el repartidor solo escribe su propia ubicación
create policy "courier updates own location"
  on delivery_locations for update
  using (delivery_person_id = auth.uid());

-- RN005/RT002: tarifa = RPC calcular_tarifa(restaurant_id, client_lat, lng)
--   con PostGIS (ST_Distance sobre la ubicación del restaurante)
-- RN001/RN006: asignar repartidor = RPC asigna_repartidor(p_order_id, p_courier_id)
--   → BEGIN; verifica courier activo sin pedido activo y a < 2km;
--   → UPDATE orders SET delivery_person_id ...; COMMIT;
```

✓ Criterios de Aceptación (ejemplos)

Escenario: Dos repartidores aceptan el mismo pedido

Dado un pedido listo sin repartidor asignado
Cuando dos repartidores llaman asigna_repartidor al mismo tiempo
Entonces solo uno obtiene la asignación (RN001, RN006, RPC atómico)
Y el otro recibe "pedido_ya_asignado"

Escenario: Rastrear repartidor asignado

Dado un pedido con repartidor asignado
Cuando el cliente abre el tracking
Entonces recibe la ubicación de su repartidor vía Realtime (RT001)
Y no puede ver la ubicación de repartidores de otros pedidos (RS001)

Escenario: Calificar después de 7 días

Dado un pedido entregado hace 8 días
Cuando el cliente intenta calificarlo
Entonces el sistema rechaza la calificación (RN009)

✓ Matriz de Trazabilidad

UseCase	Regla(s)	Contrato	Fuente de verdad	Test
PlaceOrder	RN007, RN011, RN012	OrderRepository.placeOrder	RPC + API pagos	unit + integration
AcceptOrder	RN003, RN004	OrderRepository.acceptOrder	cron + RPC	unit + integration
AssignDelivery	RN001, RN006	DeliveryRepository.acceptDelivery	RPC atómico	unit + race condition
TrackOrder	RT001, RS001	OrderRepository.watchOrder / Realtime	Supabase Realtime	widget + integration
UpdateLocation	RS002	DeliveryRepository.updateLocation	RLS + Broadcast	integration (RLS)
RateOrder	RN008, RN009	OrderRepository.rateOrder	API + trigger	unit
CalcularTarifa	RN005, RT002	DeliveryRepository.calculateDeliveryFee	RPC + PostGIS	unit

Siguiente paso

Con estos casos completados, tienes un repertorio sólido de patrones de diseño. Vuelve a aplicar FADER a cualquier feature que enfrentes en tu trabajo real.

Tiempo estimado: 2-3 horas

Material: Papel y lápiz

15: Estimación de Complejidad de Features

Antes de escribir código, estima cuánto tiempo te va a llevar. Esto te ayuda a planificar y a no prometer cosas imposibles.

Framework FADER

⚠ **Ojo:** este FADER (Frontend/Api/Data/Estados/Requisitos) NO es el FADER de diseño (Formular/Actorizar/Descomponer/Entidades/Reglas). El de diseño te dice **qué construir**; este de estimación te dice **cuánto cuesta cada tipo de trabajo**. Usa el primero para descomponer (01) y este para estimar cada pieza.

Una feature se descompone en 4 componentes:

Componente	Descripción	Ejemplo
Frontend	UI, widgets, animaciones	Pantalla de perfil
Api/Backend	Endpoints, Supabase functions	CRUD de usuarios
Data	Modelos, repositories, datasources	User model + repository
Estados	Cubit/BLoC, states, lógica	AuthCubit
Requisitos	Auth, permisos, validaciones	Login required

Pasos para estimar

Estima sobre el diseño, no sobre la intuición. Antes de estimar, tu feature debe tener: Alcance definido (00), FADER completo (01) y la [matriz de trazabilidad](#). Si la feature no está descompuesta, cualquier estimación es un número adivinado.

1. Descompón la feature

Feature: Sistema de notificaciones

F: UI de lista de notificaciones + badge

A: Supabase Realtime + Edge Function para enviar

D: Notification model + repository

E: NotificationsCubit (unread count)

R: Push notifications permission

2. Estima cada componente

Componente	Complejidad	Tiempo estimado
Frontend (UI)	Media	3-4h
API/Backend	Alta	4-6h
Data	Baja	1-2h
Estados	Media	2-3h
Requisitos	Media	1-2h

3. Suma con factor de buffer

Tiempo base: 11-17 horas

Buffer (30%): +3.5-5h

Total estimado: 15-22 horas

Niveles de complejidad

Feature simple (1-3h)

- Solo UI
- Sin llamadas a backend
- Sin estados complejos
- Ejemplo: Pantalla de settings

Feature media (4-8h)

- UI + 1-2 llamadas a backend
- 1 Cubit/BLoC
- Validación básica
- Ejemplo: Formulario de registro

Feature compleja (8-16h)

- UI + backend + realtime
- Múltiples estados
- Auth + permisos
- Ejemplo: Chat en tiempo real

Feature épica (16-40h)

- Todo lo anterior + más
- Múltiples pantallas
- Integraciones externas
- Ejemplo: Sistema de pagos

Errores de estimación comunes

Error	Cómo evitarlo
Subestimar backend	Siempre suma 30% extra
Olvidar testing	Agrega 20% para tests
No contar deployment	Agrega 1-2h para deploy
Ignorar edge cases	Agrega tiempo para errores
No hacer buffer	Siempre 30% buffer

Template de estimación

```
## Feature: [Nombre]

### Descomposición
- F: [Frontend] → [tiempo]
- A: [API] → [tiempo]
- D: [Data] → [tiempo]
- E: [Estados] → [tiempo]
- R: [Requisitos] → [tiempo]

### Total
- Base: [X] horas
- Buffer (30%): +[X] horas
- Estimado: [X] horas

### Riesgos
- [Riesgo 1]
- [Riesgo 2]

### Dependencias
- [Dependencia 1]
- [Dependencia 2]
```

La plantilla completa por feature (con alcance, FADER, reglas, contratos, Supabase, criterios y esta estimación) está en [PLANTILLA-DISEÑO-FEATURE.md](#).

Volver al índice: [README.md](#)

Plantilla de Diseño de Feature

Copia esta plantilla para cada feature nueva. Completa cada sección en orden, en papel o aquí. Cada sección enlaza a su teoría en este módulo.

Feature: [nombre] **Proyecto:** [nombre] **Fecha:** [YYYY-MM-DD] **Complejidad estimada:** [Simple / Intermedia / Compleja] → ver [15-estimacion-complejidad.md](#)

0. Alcance

Teoría: [00-alcance-feature.md](#)

- **Incluye:** [qué cubre la feature]
- **No incluye:** [qué NO cubre — límites explícitos]
- **Dependencias:** [qué debe existir antes: features, tablas, servicios]
- **Suposiciones:** [qué das por hecho]
- **Preguntas abiertas:** [qué no sabes todavía]

1. FADER

Teoría: [01-descomposicion-feature.md](#)

Formular

- ☐ Como [actor], quiero [acción] para [valor].
- ☐ Como [actor], quiero [acción] para [valor].

Actorizar

Actor	Tipo	¿Qué puede hacer?	¿Qué NO puede hacer?
[actor]	Primario		

Descomponer

- [C/R/U/D] [operación] — [actor]
- [Validación] [operación]
- [Cálculo] [operación]
- [Transición] [operación]

Entidades

- [Entidad]: [atributos esenciales]

Reglas

- RN001: [regla de negocio → UseCase/dominio]
- RN002: [regla de negocio]
- RT001: [regla técnica → DATA/datasource] (paginación, realtime, caching...)
- RS001: [regla de seguridad → autorización del servidor (RLS en Supabase / middleware en REST)]

2. Mapeo a Capas

Teoría: [02-mapeo-capas.md](#)

Responsabilidad	Capa propietaria	Pieza	Regla(s) que defiende
[validar X]	DOMAIN	[UseCase]	RN00x
[persistir]	DATA	[DataSource]	RT00x
[autorizar]	DATA	[RLS]	RS00x
[mostrar]	PRESENTATION	[Widget]	— (nunca reglas)

Estructura de carpetas

```
domain/...
data/...
presentation/...
```

3. Contratos

Teoría: [03-contratos-primerio.md](#)

```
abstract class XRepository {  
    // Métodos con params tipados y retorno Either<Failure, T>  
}
```

Estados de UI

```
sealed class XState { /* loading, loaded, error... */ }
```

4. Flujo de Datos

Teoría: [04-flujo-datos.md](#)

WIDGET → CUBIT → USECASE → REPO → DATA SOURCE → API/BD → fold() → emit()

- Validaciones en cada paso:
- Transformaciones de tipos:
- Errores manejados:
- Realtime (Streams):

5. Contrato con el Backend (Supabase / REST API)

Teoría: [05e-diseno-supabase.md](#)

Esta sección define lo que la app necesita del servidor. Con Supabase la implementas tú (tablas, RLS, RPC). Con una REST API solo especificas el **contrato** (endpoint, DTO, errores, garantías); el backend lo implementa.

- Backend:
- Garantías del servidor:

Tablas (Supabase) / Contrato de endpoints (REST API)

[nombre_tabla]	[VERBO /path] → [DTO request → DTO response]
└─ columna tipo	Errores: [códigos]
└─ constraint ...	

Operaciones por UseCase

UseCase	Operación (Supabase)	Endpoint (REST API)	Atómico?
[UseCase]	[select/insert/update/rpc]	[GET/POST/PATCH/DELETE]	[sí/no]

RLS / Autorización del servidor

- RS00x → policy RLS (Supabase) / rol+permiso en el servidor (REST): [descripción]

Realtime

- [tabla/feature] → [cambios/Broadcast]

6. Criterios de Aceptación y Trazabilidad

Teoría: 05f-criterios-aceptacion-trazabilidad.md

Criterios (BDD)

Escenario: [título]
Dado [contexto]
Cuando [acción]
Entonces [resultado]

Matriz de trazabilidad

UseCase	Regla(s)	Contrato	Fuente de verdad	Test
[UseCase]	RN00x	[método]	[API / RPC / RLS / cron]	[unit / widget / integration]

7. Decisiones y ADRs

- ADR-001: [decisión clave] — contexto, decisión, consecuencias, alternativas.

- `ADR-002` : `[decisión clave]`
-

8. Orden de Implementación

1. `[entidad]`
 2. `[contratos]`
 3. `[usecases]`
 4. `[modelos + datasources]`
 5. `[repositorios]`
 6. `[cubits]`
 7. `[páginas/widgets]`
 8. `[tablas/RLS/RPC en Supabase]`
-

Referencias del módulo: [README.md](#) · [00-alcance-feature.md](#) · [01-descomposicion-feature.md](#) · [02-mapeo-capas.md](#) · [03-contratos-primero.md](#) · [04-flujo-datos.md](#) · [05e-diseno-supabase.md](#) · [05f-criterios-acceptacion-trazabilidad.md](#) · [15-estimacion-complejidad.md](#)

Bibliografía y Fuentes

Las fuentes que dieron origen a este módulo y al proyecto completo. Libros, artículos, patrones y metodologías que fundamentan cada decisión arquitectónica.



Libros Base

Clean Architecture — Robert C. Martin (2017)

Por qué está aquí: Es la base de todo el proyecto. Define la regla de dependencia, las capas, los casos de uso, y la separación entre entidades y detalles de implementación.

Aportación concreta al módulo:

- Las 3 capas (Presentation, Domain, Data)
- La regla de dependencia: las capas internas no saben de las externas
- Los UseCases como orquestadores de lógica de negocio
- Las Entities como objetos de negocio puros

Lectura recomendada: Capítulos 1-15 (Partes I y II) para los fundamentos. Capítulos 16-22 (Parte III) para los detalles de implementación.

Domain-Driven Design — Eric Evans (2003)

Por qué está aquí: DDD enseña a descomponer problemas complejos del mundo real en modelos de software. Es la "caja de herramientas" para el paso de descomposición y entidades.

Aportación concreta al módulo:

- El **lenguaje ubicuo**: nombrar las cosas como son en el negocio
- Los **agregados**: conjuntos de entidades que se tratan como una unidad
- Las **reglas de negocio** explícitas en el dominio
- La separación entre **entidades** (tienen identidad) y **value objects** (se definen por sus atributos)

Lectura recomendada: Capítulos 1-4 (Lenguaje Ubicuo, Modelo de Dominio). Capítulo 5 (Entidades y Value Objects). Capítulo 6 (Agregados).

Growing Object-Oriented Software, Guided by Tests — Steve Freeman & Nat Pryce (2009)

Por qué está aquí: Este libro conecta el diseño con el testing. Muestra cómo dejar que los tests guíen el diseño de las interfaces y la estructura.

Aportación concreta al módulo:

- El enfoque **Contract-First** sin llamarlo explícitamente: diseñar desde afuera hacia adentro
- Los **mocks como herramientas de diseño**, no solo de testing
- Cómo el testing fuerza a tener interfaces limpias

Lectura recomendada: Capítulos 1-8 para entender la filosofía. Capítulos 20-26 para la parte práctica.

Refactoring: Improving the Design of Existing Code — Martin Fowler (1999 / 2da ed. 2018)

Por qué está aquí: No siempre se diseña perfecto desde el inicio. Fowler enseña cómo mejorar el diseño existente sin romperlo.

Aportación concreta al módulo:

- Justifica por qué vale la pena invertir tiempo en diseño: el costo de no hacerlo
- Técnicas para migrar de código espagueti a capas
- La noción de que el diseño evoluciona

Lectura recomendada: El catálogo completo de refactors. Especialmente: Extract Method, Extract Class, Move Method, Replace Conditional with Polymorphism.

Design Patterns — Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides (GoF) (1994)

Por qué está aquí: Los patrones que aparecen constantemente en Clean Architecture.

Aportación concreta al módulo:

- **Repository** (patrón): la interface que abstrae el almacenamiento
 - **Adapter**: cómo DataSource convierte datos externos a modelos internos
 - **Strategy**: cómo inyectar comportamientos alternativos
-



Patrones y Metodologías

Contract-First Design

Origen: Metodología utilizada en desarrollo de APIs (OpenAPI/Swagger), microservicios, y diseño por contratos (Bertrand Meyer, 1986).

En el módulo: Cada contrato define responsabilidades, parámetros, retornos y errores posibles. Las implementaciones vienen después.

Referencia: Meyer, B. (1986). *Design by Contract*. El principio de que las interfaces deben especificar precondiciones, postcondiciones e invariantes.

Architecture Decision Records (ADR)

Origen: Michael Nygard, 2011.

En el módulo: Los ADRs registran por qué se tomó cada decisión arquitectónica, qué alternativas se consideraron, y las consecuencias esperadas.

Referencia original: Nygard, M. (2011). *Documenting Architecture Decisions*. [Documento original](#)

Formato recomendado: El formato Nygard/Y-Statements estándar: Contexto → Decisión → Consecuencias → Alternativas.

Result Pattern (Either / Railway Oriented Programming)

Origen: Programación funcional. Popularizado en C# por Scott Wlaschin (2014).

En el módulo: Cada operación del dominio retorna `Either<Failure, Success>` en vez de lanzar excepciones. Esto hace que el manejo de errores sea explícito y tipado.

Referencia: Wlaschin, S. (2014). *Railway Oriented Programming*. [F# for Fun and Profit](#)

Implementación en Dart: fpdart (ver sección de paquetes abajo).

fpdart — Programación Funcional en Dart

Origen: Paquete Dart desarrollado por Sandro Maglione.

En el módulo: Proporciona `Either`, `Option`, `TaskEither`, y otras herramientas funcionales que se usan en los contratos del dominio.

Referencia: [fpdart en pub.dev](#) | [Documentación](#)

Behaviour Driven Development (BDD) / Gherkin

Origen: Dan North (2006), popularizado por Cucumber. El formato Dado/Cuando/Entonces nace del framework de testing RSpec.

En el módulo: La sección de criterios de aceptación (05f) usa el formato BDD para escribir criterios verificables antes de implementar.

Referencia: North, D. (2006). *Introducing BDD*. [danielnorth.net](#) | [Cucumber/Gherkin](#)

Row-Level Security (RLS) de PostgreSQL

Origen: PostgreSQL 9.5 (2016).

En el módulo: Las reglas de seguridad (RS) del diseño (02, 05e) se implementan como policies de RLS en Supabase, la fuente de verdad para "quién puede leer/escribir qué fila".

Referencia: [Documentación oficial de RLS](#) | [Guía de políticas de Supabase](#)

Transacciones y Operaciones Atómicas en PostgreSQL

Origen: ACID como propiedad fundamental de las bases relacionales (Härder & Reuter, 1983).

En el módulo: Las operaciones que deben ser indivisibles (agendar un slot, emitir factura con número secuencial, asignar repartidor) se diseñan como RPC atómicos con transacciones en Supabase (05e), en vez de múltiples updates desde el cliente.

Referencia: [Transactions en PostgreSQL](#) | [Database Functions \(RPC\) de Supabase](#)

Metodología FADER

Origen: Invención propia para este módulo (2026).

Base conceptual:

- **Formular** → Inspirado en User Story Mapping (Jeff Patton)
- **Actorizar** → Inspirado en el análisis de actores de UML y Use Case 2.0 (Ivar Jacobson)
- **Descomponer** → Inspirado en Event Storming (Alberto Brandolini) y descomposición funcional clásica
- **Entidades** → Inspirado en DDD (Eric Evans)
- **Reglas** → Inspirado en Business Rules Engine y el enfoque de DDD

La clasificación de reglas **RN** (negocio), **RT** (técnica) y **RS** (seguridad) es una extensión propia del módulo: asigna a cada tipo de regla una capa propietaria (dominio / DataSource / RLS) para la defensa en profundidad.

No inventé la descomposición de problemas. Inventé el marco con nombre y pasos claros (FADER) para que sea accionable y enseñable.

Paquetes y Herramientas Referenciadas

Paquete	Uso en el módulo	Enlace
flutter_bloc	Manejo de estado (Presentation)	pub.dev
equatable	Comparación de objetos	pub.dev

Paquete	Uso en el módulo	Enlace
get_it / injectable	Inyección de dependencias	pub.dev
fpdart	Either, Option, TaskEither	pub.dev
supabase_flutter	Backend como servicio	pub.dev
freezed (alternativa)	Sealed classes generadas	pub.dev

Nota sobre la Autoría

Este módulo `02-DISEÑO-FEATURE` es un **ensamblaje original** de conceptos existentes, no una invención desde cero de cada idea:

Concepto	Fuente	Nivel de originalidad
Capas de Clean Architecture	R. C. Martin	Adaptación
Framework FADER	Invención propia	Original
Clasificación de reglas RN/RT/RS	Invención propia	Original
Matriz de Responsabilidades	Invención propia	Original
Mapeo FADER → Clean Architecture	Invención propia	Original
Matriz de Trazabilidad (UseCase → Regla → Contrato → Test)	Invención propia	Original
Contract-First Design	Metodología establecida	Adaptación
ADRs	M. Nygard	Adaptación
Criterios de aceptación BDD	D. North / Cucumber	Adaptación
RLS de PostgreSQL	PostgreSQL	Adaptación
RPC atómicos / transacciones	PostgreSQL	Adaptación
Plantillas de contratos	Invención propia	Original
Diagramas de flujo en U	Invención propia	Original
Casos prácticos	Invención propia	Original

Última actualización: 2026-08-05